微网与外部电网电力调控策略

摘要

针对含光伏发电与储能设备的并网型微网日内购电调度问题，本文以十分钟为统一时间尺度，分别研究确定性日前计划、含预测误差的紧急购电、基于多时点光伏预报的滚动调整以及波动电价下的扩展调度四种情形。模型结构、参数设定、计算结果和核心结论均保持不变，主要对建模思路、公式表达、章节衔接、算法说明和结果分析进行了重新梳理，使四个问题之间的关系更加清楚。

问题一将一天划分为 144 个十分钟时段，以外网购电量、储能充电量、储能放电量、光伏未利用量和储能状态作为决策变量，建立线性规划模型。在满足逐时段能量平衡、储能状态递推、1200—10800 kWh 安全运行区间、5000 kW 最大充放电功率以及日首日末储能量一致等条件下，模型以全天购电费用最小为目标。求解结果显示，最优策略全天购电 59482.70 kWh，购电费用 35126.95 元。储能主要通过夜间低价充电、早晚高价放电以及午间吸收光伏完成两轮典型的峰谷套利。

问题二在此基础上加入负载与光伏的日间变化。日前利用前 7 天历史数据构造等概率情景，并用同一时段的均值进行预测，再把计划购电费用和情景紧急购电费用期望一起纳入目标函数。储能状态按日期连续传递；实际运行中，如果计划购电、实际光伏和储能放电仍不能满足负载与充电需求，则按交易时刻电价的 5 倍进行紧急购电。2025 年 2 月 1 日至 12 月 31 日，计划购电费用为 14092176.50 元，实际紧急购电量为 1322325.29 kWh，实际总购电费用为 19357655.64 元。

问题三利用 0:00、6:00、12:00 和 18:00 发布的未来 24 小时光伏预报，建立滚动调整模型。0:00 形成的计划作为基准合同，后续更新只重新优化尚未执行的时段。调整购电高于原计划的部分按 1.5 倍电价结算，低于原计划的部分按 0.5 倍电价计违约成本，同时保留实际执行中的储能反馈和紧急购电机制。多情景比较结果显示，随着 6:00、12:00 和 18:00 预报依次加入，总费用逐步下降；全部更新时总费用为 13313291.30 元，较仅计划情景节省 8.68%。

问题四进一步考虑按日期和时段变化的实时电价。在问题二的历史情景日前模型和问题三的滚动模型基础上，分别建立 Q4-2 和 Q4-3。报告期实际总费用分别为 19832585.95 元和 13586985.25 元。Q4-3 内部比较还显示，使用 0:00、6:00、12:00、18:00 全部预报更新时，相比仅使用 0:00 预报可减少总费用 1673784.26 元，降幅 10.97%，紧急购电费用下降 88.30%。

本文在统一符号、统一两位小数展示规则以及规范 LaTeX 公式和 Word 可编辑公式的基础上，对算法流程和结果机理作了补充。四个问题由确定性日前调度逐步过渡到不确定性情景调度，再扩展到滚动预测控制和波动电价环境，整体上形成一条较为连贯的建模思路。

关键词： 微网；线性规划；储能调度；光伏预报；紧急购电；滚动优化；波动电价

1 问题重述

微网由光伏发电、储能设备、小区负载以及与外部电网的购电接口共同构成。其运行目标不是简单地在每个时刻“缺多少买多少”，而是利用储能的跨时段搬移能力，把价格较低时段和光伏富余时段的电能转移到价格较高、负载较大或光伏不足的时段使用。题目同时规定微网提供的电能不能低于小区负载，因此任何调度方案都必须首先满足供电可靠性，再在此基础上尽量降低外网购电费用。

题目给定的储能设备最大容量为 12000 kWh，实际允许运行的储电量区间为 1200—10800 kWh，最大充放电功率为 5000 kW，充电效率和放电效率均为 90%。由于原始数据以十分钟为间隔，5000 kW 的功率上限在单个时段内对应的最大充电量或放电量为 5000/6=833.33 kWh。2025 年 1 月 1 日 0:00 初始储电量为 6000 kWh。上述参数构成四个问题共同的物理约束骨架。

问题一研究理想确定性场景：每天的电价与小区负载曲线完全相同，且在每天 0:00 可以获得当天完整的光伏预测。要求制定全天计划购电策略，并保证 0:00 与 24:00 的储电量相同。这意味着问题一是一个具有周期边界的确定性日内优化问题，其重点是确定在已知价格、已知负载、已知光伏的条件下，外网购电与储能充放电应如何协同分配。

问题二取消了“负载和光伏每天相同”的假设。每天 0:00 制定计划后，真实负载与真实光伏可能偏离日前判断。若实际供能不足，必须按交易时刻电价的 5 倍紧急购电。因此问题二不仅追求计划购电便宜，还需要控制预测偏差造成的高价缺口风险。本文采用前 7 天同一时段数据的均值形成日前预测，并把 7 个历史日作为等概率情景，以线性情景规划方式折中计划成本与紧急购电风险。

问题三进一步提供 0:00、6:00、12:00 和 18:00 四个发布时刻的未来 24 小时整点光伏预报。0:00 计划不再是全天固定不变，而可以在后续预报到来后修改未执行部分。由于上调购电和下调购电都存在结算代价，调整不是免费的，因此问题三实质上是一个具有基准合同、分段线性调整成本和滚动信息更新的多阶段调度问题。题目还要求分析是否有必要使用 0:00 以外的预测信息，这需要通过不同更新组合的费用与紧急购电表现比较来回答。

问题四进一步令外网电价也随日期和时段实时波动。波动电价改变了每一单位电能在不同时段的机会成本，使储能的充放电时点选择与预报修正共同作用。原稿分别把问题二与问题三模型中的固定日内电价替换为附件 4 的逐日逐时价格，得到 Q4-2 和 Q4-3 两条求解线路，从而讨论“历史情景日前决策”和“预报滚动调整”在波动价格环境中的表现。

2 问题分析

2.1 系统结构与核心矛盾

四个问题共享同一个能量系统：小区负载是刚性需求，光伏是边际成本近似为零但具有时变性的本地电源，储能可以跨时间转移能量但受到容量、功率与效率约束，外网则提供可调节的补充电能但需要支付电费。模型需要处理的主要矛盾集中在三个层面。第一，储能具有时间耦合性。本时段充电会抬高后续可用储能，本时段放电则会消耗未来调节空间，因此任何单时段的局部最优都不能代表全天最优。第二，光伏的价值取决于其出现时点。当光伏与负载同步时可直接降低购电；当光伏超过负载时，只有储能留有容量才能把多余光伏转移到后续，否则只能形成未利用电量。第三，预测误差使“日前最优”与“实际最优”分离。问题二以后，计划阶段掌握的信息有限，实际执行阶段可能出现缺口，高倍紧急电价把这种缺口放大为显著的经济风险。

2.2 时间离散与状态耦合

附件中的功率数据以 kW 给出，而购电、充电、放电和储电量以 kWh 计量，因此需要统一转换到能量尺度。将一天划分为 144 个十分钟时段，记时段长度为 $\Delta t = 1/6$ h。若第 t 个时段负载功率为 P_t^L、光伏功率为 P_t^G，则相应的负载电量和光伏电量为

$$l_t = P_t^L \Delta t, \qquad g_t = P_t^G \Delta t.$$

这样，购电、光伏、充放电和储能状态可以在同一能量量纲下进行逐时段平衡。由于储能状态满足递推关系，144 个时段并不是 144 个相互独立的决策，而是一条由初始储能状态贯穿全天的状态轨迹。问题二和问题四的 Q4-2 还需要把上一日 24:00 的实际储能量传给下一日 0:00，进一步形成跨日耦合。

2.3 确定性、情景与滚动三类决策范式

按照信息获取方式划分，问题一属于完全信息下的确定性线性规划；问题二属于有限历史信息下的情景线性规划；问题三属于预报逐步到达的滚动优化；问题四则在前两类结构中叠加时变价格。四类模型并不是相互独立的，而是沿同一基础模型逐步加入不确定性和信息更新机制。这样的统一视角有利于保持符号、约束和算法的一致，也使各题结果可以在共同框架下解释。

问题一的目标只有计划购电费用，因此价格差决定储能套利方向；问题二把高价紧急购电的期望成本加入目标，相当于给“计划不足”附加风险惩罚；问题三进一步允许根据新预报修正购电，但上调与下调的结算规则形成分段线性调整成本；问题四只把各时段的价格权重从固定日曲线替换为逐日变化曲线，基础的能量平衡和储能状态约束仍保持不变。

2.4 结果分析的主要视角

对四个问题的结果解读不能只停留在表格数值，还应从“价格—光伏—负载—储能状态—购电行为”的因果链条解释。价格低时，模型通常会倾向于购电并充电；价格高时，模型通常会倾向于减少购电并放电；光伏富余时，储能如果有剩余容量，会优先吸收本地清洁能源；预测偏差较大时，固定计划可能出现紧急购电；新的预报到来后，滚动模型通过重新安排未执行购电与储能动作降低缺口。波动电价进一步改变了这些行为发生的具体时点，但不会改变上述基本机理。

3 模型假设

为使四题在统一框架下具有明确的数学表达，沿用本文采用的模型中已经采用的假设并进行规范化陈述。

（1）**十分钟时段内参数保持常值。**每个数据点代表对应十分钟区间的平均功率或价格，时段内部不再继续细分。这样既与附件时间分辨率一致，也使功率到电量的转换具有明确意义。

（2）微网只能向外网购电，不向外网售电。题目只给出购电与购电费用规则，没有给出上网电价或售电机制，因另外网交易变量取非负值。

（3）**储能充电效率和放电效率均为 0.90。**若微网侧向储能输入 c_t kWh，则电池内部增加 $0.9c_t$ kWh；若储能向微网输出 r_t kWh，则电池内部减少 $r_t/0.9$ kWh。由此往返效率为 0.81。

（4）**储能在安全区间内运行。**储电量始终满足 1200—10800 kWh，单个十分钟时段的充电量、放电量均不超过 $5000/6$ kWh。问题一执行日首日末相等的周期边界；问题二与 Q4-2 采用实际状态跨日连续传递；问题三与 Q4-3 沿用原稿中每次规划轨迹的日末目标设置。

（5）**未利用电量允许存在。**当光伏与常规购电、储能调度形成的供给超过当期负载及充电需求时，剩余电量可作为未利用量进入平衡方程。该变量保证模型在不同供需组合下保持可行，同时避免把所有剩余量机械解释为纯弃光。

（6）不单独设置充放电互斥 0-1 变量。原始各题均采用连续线性规划。由于充放电效率小于 1、购电价格为正，同时充放电不会形成经济收益；本文模型中的数值结果也未出现需要额外二元变量处理的情形。因此保持原模型的连续线性结构。

（7）**问题二只使用日初之前已经发生的历史数据。**日前预测由前 7 天同一时段数据构造，不使用当天未来实际负载或实际光伏。当天真实数据只用于计划执行后的紧急购电核算。

（8）**问题三和 Q4-3 的后续更新只影响尚未执行的时段。**0:00、6:00、12:00 和 18:00 之间已经执行的购电与储能动作不能被事后修改，新的预报仅用于重算剩余时段。

（9）**问题四把当日电价曲线视为调度输入。**按原稿设定，附件 4 给出的当天 144 个时段电价在优化时作为已知成本权重使用。若实际应用中未来价格也不可预知，则还需另建价格预测模型，但不属于本次原稿范围。

4 符号说明

符号	含义	单位
t	十分钟时段编号，t=1,…,144	-
d	日期索引	-
s	历史情景索引	-
Δt	单时段长度，1/6	h
p(d,t)	日期 d、时段 t 外网电价	元/kWh
l(d,t)	小区负载电量	kWh
g(d,t)	光伏发电电量	kWh
x(d,t)	0:00 制定的计划购电量	kWh
y(d,t)	调整后最终常规购电量	kWh
c(d,t)	储能充电量	kWh
r(d,t)	储能向微网输出的放电量	kWh
S(d,t)	时段末储能量	kWh
e(d,t)	实际紧急购电量	kWh
u(d,t)	剩余未利用电量	kWh
ηc，ηd	充电效率、放电效率，均为 0.90	-
Smin，Smax	储能安全下、上界，1200 与 10800	kWh
Emax	单时段最大充放电量，5000/6	kWh
u+，v−	相对 0:00 计划的增购量与取消量	kWh

5 问题一：确定性日前购电与储能协同优化

5.1 问题难点与建模思路

问题一虽然没有预测误差，但仍具有显著的跨时段耦合。若只按当前价格决定充放电，就可能在早晨较高电价时提前把储能耗尽，从而无法覆盖晚间更高电价；也可能在光伏高峰之前把储能充满，导致午间光伏富余无法吸收。所以需要把一天 144 个时段放在一起进行优化。

建模时以十分钟为最小决策单元，将每个时段的外网购电、充电、放电和未利用光伏同时作为变量。储能状态由上一时段递推到下一时段，并设置安全上下界和单时段功率上限。由于每天的负载、电价与光伏预测都重复，题目要求 0:00 与 24:00 储能量相同，因此可把一天视为周期运行单元，初末储能量均取 6000 kWh。这样得到一个标准线性规划：目标函数是 144 个时段购电费用之和，所有约束均为线性等式或线性不等式。

5.2 决策变量与能量平衡

记第t个时段外网计划购电量为x_t，充电量为c_t，向微网输出的放电量为r_t，未利用光伏为w_t，时段末储能量为S_t。在每个时段内，供给端由外网购电、光伏电量和储能放电组成，需求端由小区负载和储能充电组成。为使等式能够覆盖光伏富余情形，引入非负未利用量w_t，则有

$$x_t + g_t + r_t = l_t + c_t + w_t, \qquad t = 1, \ldots, 144.$$

该式体现了微网内部的能量守恒。若光伏和购电恰好满足负载及充电，则$w_t = 0$；若供给超过可消纳需求，多余部分进入w_t。由于外网购电价格为正，最优解不会无意义地购买大量电能再丢弃，因此该松弛变量不会改变原目标的经济方向。

5.3 储能状态方程与运行边界

储能状态递推为

$$S_t = S_{t-1} + \eta_c c_t - \frac{r_t}{\eta_d}, \qquad \eta_c = \eta_d = 0.90.$$

其中，充电端输入c_t并不能全部转化为电池内部能量，实际增加量为$0.9c_t$；反之，为了向微网输出r_t，电池内部需要减少$r_t/0.9$。该表达保留了原稿中双向 90%效率的解释，也使充放电损耗直接进入 SOC 递推。

储能安全约束与功率约束写为

$$1200 \le S_t \le 10800,$$

$$0 \le c_t \le \frac{5000}{6}, \qquad 0 \le r_t \le \frac{5000}{6}.$$

题目规定日首日末储电量相同，结合给定初始状态得到

$$S_0 = S_{144} = 6000.$$

这一周期边界非常关键。若没有日末约束，优化器可能在 24:00 前把储能尽量放空以降低当日费用，得到一个不可持续重复的“末端透支”策略；加入周期约束后，每天都以相同储能条件开始和结束，成本才具有稳定日循环意义。

5.4 目标函数

问题一只计算正常计划购电费用，因此目标为

$$\min C_1 = \sum_{t=1}^{144} p_t x_t.$$

由于p_t、l_t和g_t均为已知参数，目标与所有约束都对决策变量呈线性，模型可以用线性规划直接求全局最优解。原稿采用 HiGHS 求解器，决策向量包含 144 个购电变量、144 个充电变量、144 个放电变量、144 个未利用变量和 144 个储能状态变量，共 720 个连续变量。

5.5 算法流程

求解流程可以概括为五步。第一步，读取附件 1 中 144 个时段的电价、负载功率与光伏预测功率，并乘以$1/6$将功率转化为时段电量。第二步，根据能量平衡式和 SOC 状态方程建立等式约束矩阵。第三步，把 SOC 上下界、充放电上限与变量非负性写入变量边界。第四步，将$S_0 = S_{144} = 6000$作为周期约束加入模型并调用线性规划求解器。第五步，按题目要求把 144 时段计划汇总为指定十分钟时段、六个四小时时段充放电量以及全天总购电量与总费用。

这种求解方法的优势在于不需要人为预先指定“何时充、何时放”。价格、光伏、负载与储能约束共同决定最优时点。储能的跨时段价值通过 SOC 递推自动传递，求解器在全天范围内比较不同充放电路径的总成本。

5.6 计算结果

原始问题一论文给出的指定时段结果如下，本文统一按两位小数展示。

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量	59482.70	全天购电费	35126.95		
时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	4500.00	0.00	12:00-16:00	5286.04	91.10
4:00-8:00	833.33	6365.84	16:00-20:00	0.00	5780.13
8:00-12:00	4787.96	1703.00	20:00-24:00	5333.33	2859.87

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
0:00 储电量	6000.00		24:00 储电量	6000.00	

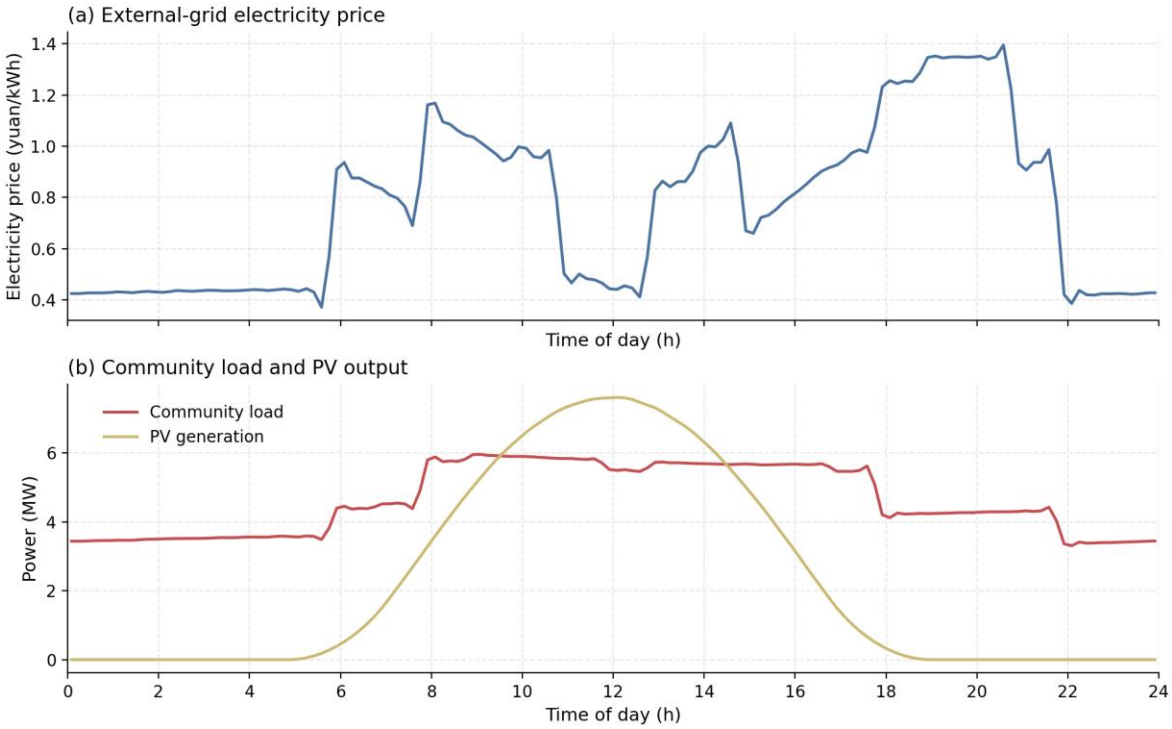


图 5-1 日内电价、负载与光伏预测功率

5.7 结果机理分析

结果显示，10:00—10:10 与 14:00—14:10 购电量为 0，说明这两个时段光伏供给与储能调节能够覆盖负载；12:00—12:10 仍购电 480.41 kWh，是因为午间价格较低，模型不仅利用光伏，还主动从外网补充低价电并充入储能。16:00—16:10 与 18:00—18:10 存在一定购电，反映当时负载较高、储能功率和状态边界共同限制了完全以放电覆盖需求的能力。20:00—20:10 购电量再次为 0，说明晚间高价时段储能放电被优先使用。

从四小时汇总结果看，0:00—4:00 充电 4500.00 kWh 且不放电，典型地体现了低价时段储能蓄能；4:00—8:00 则放电 6365.84 kWh，说明晨间价格上升后储能承担了大量供电。8:00—16:00 期间充放电交替发生，原因不是模型“摇摆”，而是光伏、价格和负载在十分钟尺度持续变化，四小时总量只是对许多细时段动作的聚合。16:00—20:00 放电 5780.13 kWh 而不充电，符合晚峰削减高价购电的经济逻辑。20:00 以后又出现 5333.33 kWh 充电，这是为了在低价回落后补足日末 6000 kWh 周期边界。

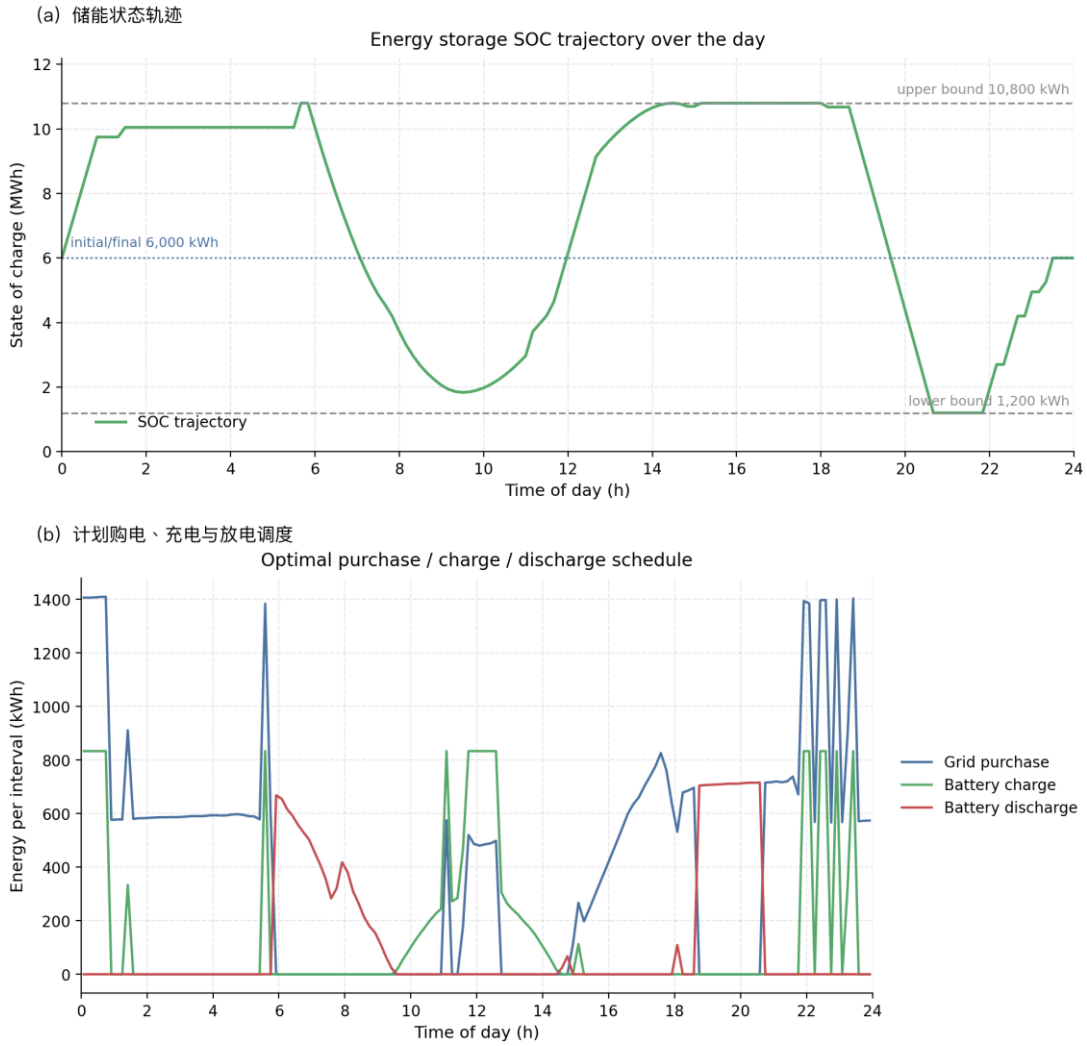


图 5-2 储能状态与计划购电、充电和放电调度

进一步指出，若以无储能情形作为基准，全天购电成本为 48052.05 元，而最优储能调度成本为 35126.95 元，节省 12925.10 元，降幅 26.90%。这一差异并不是简单由“少买电”造成。储能存在充放电损耗，因此全天计划购电量甚至可能高于净负荷直接购电量；真正的经济收益来自把购电时点从高价区间转移到低价区间。换句话说，储能的核心价值是价格套利与光伏时移，而不是凭空减少总能量需求。

从边际条件理解，若在低价时段购入 1 kWh 电能并经历一次完整充放电循环，最终可替代约 0.81 kWh 高价购电。因此只要峰谷价格比高于$1/0.81 \approx 1.235$，理论上就存在套利空间。原稿给出的典型高低价比显著高于该临界值，所以最优策略频繁触及 SOC 与充电功率边界是合理的：价格差足够大时，储能的物理能力而非经济激励成为进一步降低成本的主要限制。

5.8 模型适用边界与本题小结

问题一模型适用于电价、负载和光伏预测均确定的日内计划场景。它没有考虑电池寿命折损、温度效应、逆变器效率随功率变化等工程细节，也没有考虑计划与实际之间的预测误差，因此更适合作为后续问题的基础骨架。它的作用主要在于明确了最基本的能量平衡、SOC 递推和运行边界结构，为问题二引入随机性、问题三引入滚动信息更新以及问题四引入波动电价提供统一底层模型。

6 问题二：含紧急购电的历史情景日前调度

6.1 从确定性计划到不确定性计划

问题二与问题一的根本差别在于：每天 0:00 制定计划时，并不知道当天真实负载和真实光伏的完整轨迹，而计划一旦不足，就会以 5 倍时价产生紧急购电。若仍然简单采用问题一的“点预测最小成本”思想，模型可能为了节省计划费用而把计划购电压得过低，从而在实际运行中频繁承担高额缺口成本。因此日前模型必须同时考虑“买多了的常规成本”和“买少了的紧急风险”。

原稿的处理方法是使用前 7 个实际日构造历史情景。对日期d，历史集合记为

$$H_d = \{d-7, d-6, \dots, d-1\}.$$

同一十分钟时段的日前预测负载和光伏取七日均值：

$$\hat{l}_{d,t} = \frac{1}{7}\sum_{s\in H_d} l_{s,t}, \qquad \hat{g}_{d,t} = \frac{1}{7}\sum_{s\in H_d} g_{s,t}.$$

同时，每一个历史日都作为一个等概率情景进入目标函数。这样既保留了均值预测所代表的中心趋势，也利用 7 个具体历史轨迹描述短期波动范围。

6.2 公共计划与情景补偿

对当天计划而言，$x_{d,t}$、$c_{d,t}$和$r_{d,t}$在所有历史情景中共享，代表 0:00 必须一次性决定的动作；而情景紧急购电$e_{d,s,t}$和剩余未利用量$u_{d,s,t}$允许随情景变化，用来吸收不同历史日供需差异。日前均值预测下的平衡为

$$x_{d,t} + \hat{g}_{d,t} + r_{d,t} = \hat{l}_{d,t} + c_{d,t} + \hat{u}_{d,t}.$$

历史情景s下的平衡为

$$x_{d,t} + g_{s,t} + r_{d,t} + e_{d,s,t} = l_{s,t} + c_{d,t} + u_{d,s,t}.$$

问题二采用的是“第一阶段公共决策+第二阶段情景补偿”结构。公共计划反映真正可在 0:00 实施的决定，情景变量并不是实际未来已经知道的动作，而是用来评价该计划在不同可能轨迹下会产生多大缺口成本。

6.3 实际紧急购电机制

计划确定后，实际日运行使用当天真实负载与光伏核算缺口。若计划购电、实际光伏和储能放电不足以覆盖实际负载及充电需求，实际紧急购电量为

$$e^{\mathrm{act}}_{d,t} = \left[l^{\mathrm{act}}_{d,t} + c_{d,t} - x_{d,t} - g^{\mathrm{act}}_{d,t} - r_{d,t}\right]_+,$$

其中$[z]_+ = \max(z, 0)$。紧急购电价格为交易时刻电价的 5 倍，因此实际紧急费用按每个十分钟时段逐项计算为$5p_t e^{\mathrm{act}}_{d,t}$。这一机制把预测误差直接映射为经济惩罚：如果某个时段负载突然升高或光伏低于日前估计，缺口越大，5 倍电价造成的额外费用越显著。

6.4 目标函数与储能跨日连续

计划购电费用与情景紧急购电费用期望共同构成日前目标：

$$\min C_2 = \sum_{t=1}^{144} p_t\, x_{d,t} + 5\sum_{s\in H_d}\pi_s \sum_{t=1}^{144} p_t\, e_{d,s,t}, \qquad \pi_s = \frac{1}{7}.$$

储能状态仍满足

$$S_{d,t} = S_{d,t-1} + 0.9c_{d,t} - \frac{r_{d,t}}{0.9},$$

并满足 1200—10800 kWh 与单时段 833.33 kWh 上限。与问题一不同，原稿不要求每天 24:00 回到 6000 kWh，而采用

$$S_{d+1,0} = S_{d,144}$$

实现跨日连续运行。这样设置后，储能成为全年连续状态，而不是每天人为“重置”。2025 年 1 月作为预热期，从 2 月 1 日起统计正式结果。

6.5 求解规模与流程

每天模型含 720 个公共连续变量和 2016 个情景变量，共 2736 个连续变量。由于目标函数、能量平衡、SOC 递推和上下界全部线性，本文采用稀疏矩阵组织约束并调用 HiGHS 逐日求解。每个日期完成以下步骤：读取前 7 个实际日；计算同一时段均值预测；建立 7 个历史情景；读取当天 0:00 的储能状态；求解日前计划购电和储能动作；再用当天实际数据计算实际紧急购电；最后把 24:00 储能状态传递给次日。

这样的处理主要是为了保证信息使用符合实际时序。日初优化不能偷看当天未来真实光伏，而实际数据只在执行后用于评估计划。也正因为如此，问题二得到的紧急购电结果才能真实反映静态日前计划面对预测误差时的表现。

6.6 全年结果

统计结果显示 2025 年 2 月 1 日至 12 月 31 日共 334 天、48096 个十分钟计划时段。汇总结果如下。

指标	数值
计划购电费用	14092176.50 元
实际紧急购电量	1322325.29 kWh
实际紧急购电费用	5265479.14 元
实际总购电费用	19357655.64 元
发生紧急购电的日期数	287 天
储能量范围	1200—10800 kWh
最大充电量	833.33 kWh/10 分钟
最大放电量	833.33 kWh/10 分钟

日期	10:00－10:10	12:00－12:10	14:00－14:10	16:00－16:10	18:00－18:10	20:00－20:10	全天购电量/kWh	全天购电费/元
2025.03.20	0.00	570.82	0.00	522.26	0.00	0.00	64058.65	37891.54
2025.06.21	0.00	730.09	0.00	636.83	690.22	0.00	74760.23	45824.61
2025.09.23	0.00	416.30	0.00	510.64	703.42	0.00	61910.84	37456.58
2025.12.21	0.00	944.31	0.00	827.66	0.00	0.00	88675.66	55539.67

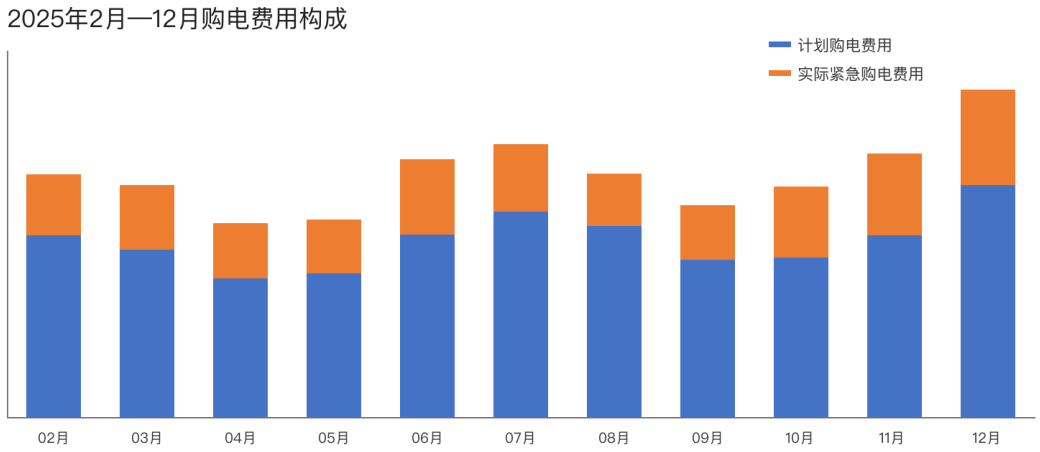


图 6-1 2025 年 2 月至 12 月计划购电费用与紧急购电费用

6.7 指定日期与紧急购电现象分析

四个指定日期的计划结果体现了统一的低价充电、高价放电结构，但不同日期由于历史窗口不同，计划购电量和实际紧急缺口存在显著差异。3 月 20 日全天计划购电 64058.65 kWh，计划费用 37891.54 元；实际运行中紧急购电总量

4970.24 kWh，缺口分散在凌晨、午间和晚间多个区间。6 月 21 日计划购电 74760.23 kWh，虽然计划量在四个日期中不算最低，但当天实际光伏与负载恰好能被计划和储能动作覆盖，因此没有紧急购电。

9 月 23 日全天计划购电 61910.84 kWh，实际紧急购电 5994.16 kWh，其中 0:00—7:50 与 17:10—22:40 是主要缺口区间。12 月 21 日计划购电 88675.66 kWh，实际紧急购电 6486.47 kWh，16:20—24:00 的连续区间贡献 3475.19 kWh。上述结果说明，紧急购电不能仅用“计划购电量大或小”解释，而是由计划购电、真实光伏、真实负载、储能充放电以及当时 SOC 共同作用。即使全天计划量较高，如果关键时段供给与需求错位，仍可能出现高价缺口。

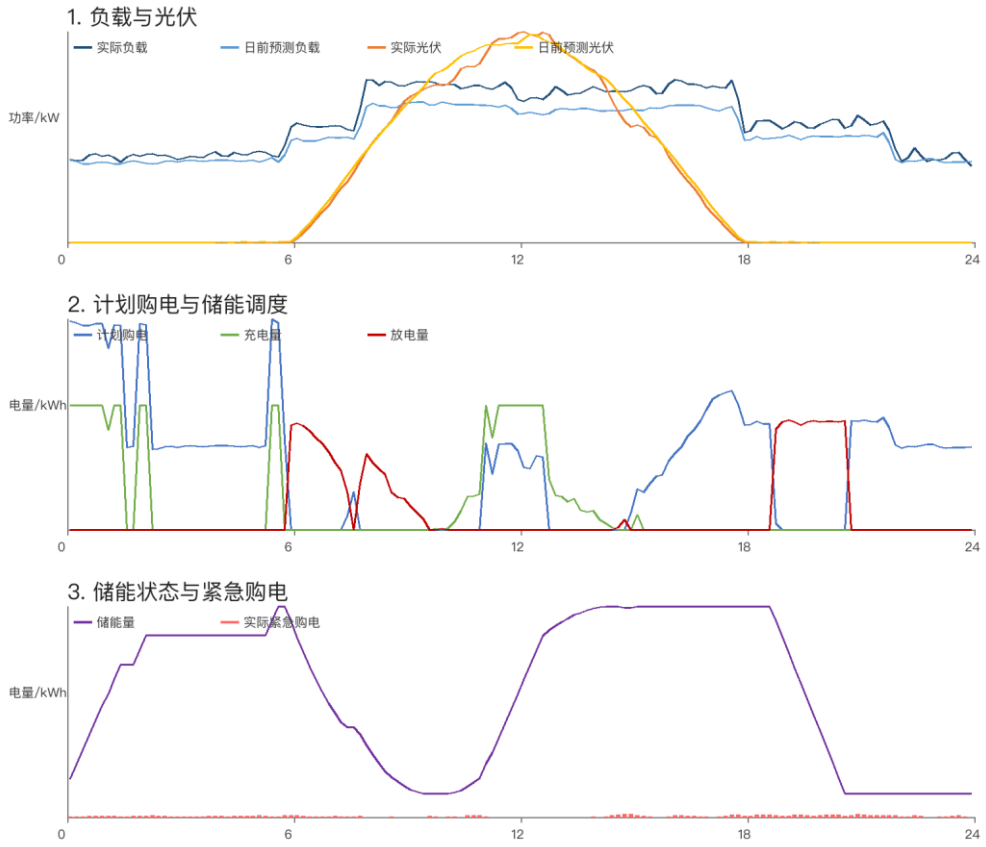


图 6-2 2025 年 9 月 23 日负载、光伏、购电、储能与紧急购电过程

全年有 287 天发生紧急购电，这说明，基于 7 日历史均值与有限情景的静态 0:00 计划无法完全消除预测误差。这里这并不说明模型失效，而恰好揭示了问题二的研究意义：当计划只能在日初制定一次时，模型必须在“提前多买一些以降低缺口风险”和“减少常规购电以控制计划成本”之间作出折中。5 倍紧急电价使这种折中偏向较为保守的计划，但只要实际轨迹超出历史情景所代表的范围，仍可能发生紧急购电。

6.8 机理解释与适用边界

历史情景模型的优势在于无需预先指定负载或光伏误差的概率分布，只需利用最近 7 天实际轨迹即可形成情景集合；同时模型保持线性，便于对全年逐日重复求解。局限在于 7 天窗口只能代表短期历史特征，对季节突变、节假日负载变化或异常天气下的光伏偏差响应有限。更重要的是，0:00 计划一旦生成便保持不变，无法利用当天 6:00、12:00 或 18:00 之后新获得的光伏预报。这个局限直接引出问题三：当信息在日内逐步更新时，是否值得修改原计划，以及修改的成本是否能够被减少的紧急购电抵消。

7 问题三：基于光伏预报的滚动调整购电模型

7.1 信息结构与滚动决策

问题三在问题一和问题二的基础上引入新的信息时点：0:00、6:00、12:00 和 18:00 均可获得未来 24 小时整点光伏预报。与问题二相比，决策者不必把 0:00 计划机械执行到 24:00，而可以在新的预报到达后，根据最新预测和当前储能状态重新安排尚未执行时段。问题的关键由“如何制定一个静态计划”转变为“如何在基准计划与新信息之间动态权衡”。

0:00 仍需先制定计划购电量x_i。到 6:00、12:00 或 18:00 时，设最终调整后的常规购电量为y_i。若$y_i > x_i$，超出部分按 1.5 倍电价结算；若$y_i < x_i$，被取消的部分按 0.5 倍电价承担违约成本。因此，调整既能降低预测偏差带来的紧急购电风险，也会产生额外合同成本，最优策略必须比较二者。

7.2 0:00 计划模型

原稿的 0:00 计划沿用确定性线性规划骨架。若f_i表示根据 0:00 预报得到的第i时段光伏电量，计划阶段有

$$\min \sum_{i=1}^{144} \pi_i x_i,$$

$$x_i + f_i + d_i - c_i - w_i = l_i,$$

$$S_i = S_{i-1} + \eta c_i - \frac{d_i}{\eta},$$

$$S_{144} = S_0 = 6000,$$

$$1200 \le S_i \le 10800, \qquad 0 \le c_i, d_i \le \frac{5000}{6},$$

$$x_i \ge 0, \qquad w_i \ge 0.$$

这里的x_i构成全天基准计划，也是后续调整结算的参照。储能轨迹则提供一个初始的日内能量搬移方案。由于后续预报会变化，实际执行时允许根据当前状态重新优化尚未发生的时段。

7.3 调整结算函数

为统一表示增购和取消，原稿给出单时段相对于原计划的结算函数

$$\Phi(y_i; x_i) = \pi_i \min(x_i, y_i) + 0.5\pi_i (x_i - y_i)^+ + 1.5\pi_i (y_i - x_i)^+.$$

若$y_i = x_i$，只按原电价支付原计划；若$y_i < x_i$，实际保留部分按正常价格结算，被取消部分承担 50%电价的违约成本；若$y_i > x_i$，原计划部分按原价结算，新增部分按 1.5 倍电价购买。利用非负变量u_i, v_i表示上调和下调，可写为

$$u_i \ge y_i - x_i, \qquad v_i \ge x_i - y_i, \qquad u_i, v_i \ge 0,$$

并将后续优化目标线性化为

$$\min \sum_{i \in I(\tau)} \pi_i \, (y_i + 0.5u_i + 0.5v_i),$$

其中$I(\tau)$表示当前更新时刻之后尚未执行的时段集合。过去时段固定不变，只重算未来时段。

7.4 实时执行与紧急购电

滚动优化并不能保证预测完全准确，因此实际执行层仍保留紧急购电。按原稿表达，若反馈后的实际充电、放电分别为c'_i, d'_i，则

$$e_i = \max(0, l_i + c'_i - y_i - g_i - d'_i).$$

该式说明常规调整购电y_i与紧急购电e_i是不同层次的变量。前者在预报更新时提前规划并承担调整结算成本；后者是实际时段发生后仍然存在供电缺口时的最后保障。有效的滚动策略应当用相对便宜的提前调整来减少昂贵的临时紧急购电。

7.5 预报从整点到十分钟的对齐

附件 3 给出整点光伏功率预报，而调度以十分钟为粒度。原稿通过在相邻整点预报之间进行线性插值，把整点功率转化为十分钟功率，再乘以1/6得到时段电量。每次更新只截取发布时刻到当日 24:00 的剩余部分，不把次日预报错误接入当日，也不把一个整点值机械视为整个小时恒定功率。这一步虽然属于数据预处理，但直接决定后续计划与调整的时间对齐方式。

7.6 指定日期结果

给出的四个指定日期最终常规购电结果如下，数值统一保留两位小数。

时间段	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00 – 10:10	0	0	0	0
12:00 – 12:10	1614.75	0	1574.90	1769.05
14:00 – 14:10	0	0	0	0
16:00 – 16:10	663.99	74.96	636.70	813.72
18:00 – 18:10	697.99	0	764.33	715.41
20:00 – 20:10	0	0	0	0
全天购电量(kWh)	68643.08	33322.51	69934.51	96866.69
全天购电费(元)	46652.20	19081.75	44391.01	63012.39

日期	紧急购电时间段	紧急购电量
2025 年 3 月 20 日	8:50 – 10:00	661.24
2025 年 6 月 21 日	无	—
2025 年 9 月 23 日	9:20 – 9:40	93.76
2025 年 12 月 21 日	7:50 – 8:00	6.61

四个日期中，6 月 21 日没有紧急购电，3 月 20 日紧急购电 661.24 kWh，9 月 23 日为 93.76 kWh，12 月 21 日仅 6.61 kWh。与问题二相同日期的静态计划相比，问题三通过日内更新明显缩小了若干日期的缺口规模。这说明：新预报的价值主要体现在提前修正未来数小时的光伏偏差，使储能与常规购电在缺口真正发生前重新排布。

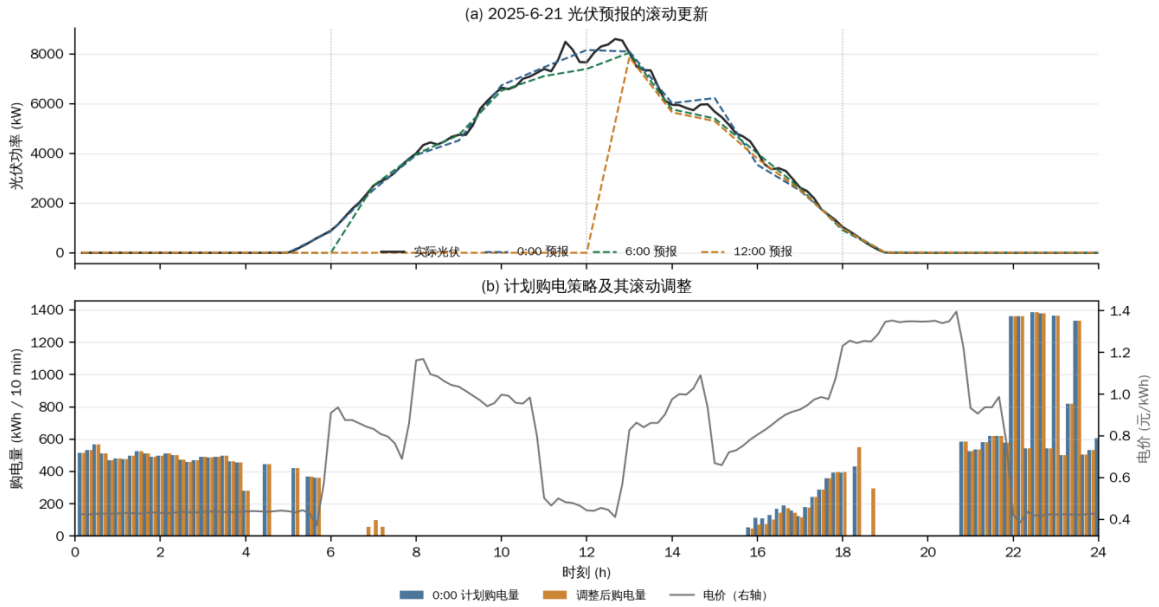


图 7-1 2025 年 6 月 21 日光伏预报更新与购电策略调整

7.7 是否需要引入其他时刻预报

题目要求分析 0:00 之外的预报是否有必要。原稿通过五种情景进行比较：仅计划、不同时点逐步更新以及理想完美信息。其核心结果如下。

情景	调整时刻	总购电费用 / （元）	结算费用 / （元）	紧急购电费用 / （元）	紧急购电量 / （kWh）	较情景 A 节省
A 仅计划	—	14 579 315.82	12 347 705.71	2 231 610.10	348 229.7	—
B6	6:00	13 920 897.87	12 687 550.95	1 233 346.92	188 351.7	658 417.95 (4.52%)
B612	6:00, 12:00	13 570 394.24	13 023 847.94	546 546.30	87 109.6	1 008 921.58 (6.92%)
B61218	6:00, 12:00, 18:00	13 313 291.30	13 207 398.49	105 892.81	21 892.1	1 266 024.52 (8.68%)
H 完美信息	—	12 245 046.92	12 245 046.92	0	0	2 334 268.90 (16.01%)

对比这些结果可以看到，仅执行 0:00 计划时，总购电费用为 14579315.82 元，紧急购电费用 2231610.10 元。加入 6:00 更新后，总费用降至 13920897.87 元；继续加入 12:00 更新后降至 13570394.24 元；加入 18:00 更新后进一步降至 13313291.30 元。相对于仅计划情景，全部更新累计节省 1266024.52 元，降幅 8.68%，紧急购电量也从 348229.7 kWh 降至 21892.1 kWh。

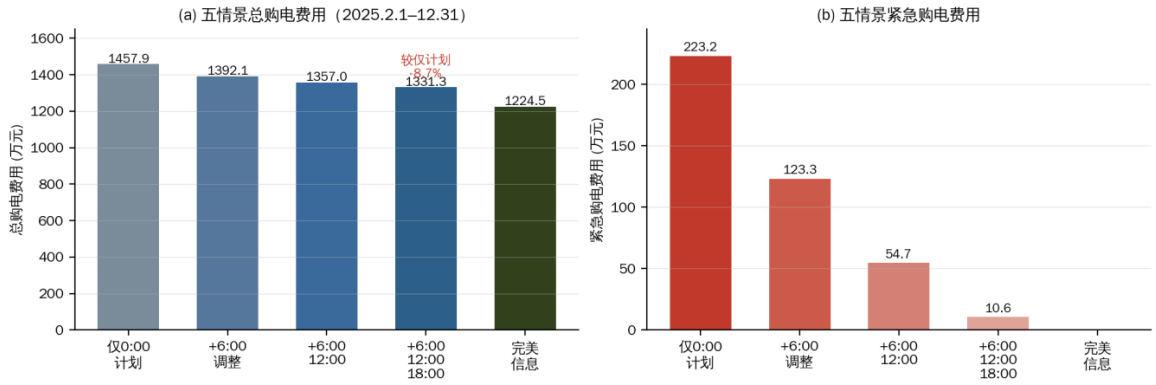


图 7-2 不同预报更新情景的总购电费用与紧急购电费用

这些结果支持使用题目给出的后续预报。原因在于光伏预测误差具有时效性：距离实际发生时刻越近，预报通常越能反映当天云量、辐照等变化。6:00 更新可以修正上午到晚间的光伏轨迹，12:00 更新可以重新评估下午发电与晚峰前的储能需求，18:00 更新虽然光伏即将结束，但仍能利用当前实际储能状态重新安排夜间购电和放电。因此 18:00 更新的价值并不只来自“预测晚间光伏”，更来自对日内累积状态误差的校正。

完美信息情景的费用为 12245046.92 元，低于全部滚动更新情景，表明即使使用三个后续预报，仍存在信息不完全造成的额外成本。这一差距可以理解为预测误差与调整制度共同形成的“信息价值上限”：如果未来光伏完全可知，模型可以一次性安排更精确的购电和储能路径，而现实滚动模型仍需为预测偏差和合同调整成本付费。

7.8 灵敏度分析

进一步改变日末储能目标和储能效率，得到如下结果。

参数设定	总费用（元）	相对基准变化	紧急购电费用（元）
基准：终值=6000，η=0.90	13 313 291.30	—	105 892.81
终值自由（仅受[1200,10800]约束）	13 321 774.91	+0.064%	106 516.81
效率 η=0.85	13 784 231.26	+3.50%	75 992.68
效率 η=0.95	12 855 872.33	−3.45%	119 642.62

储能效率从 0.90 下降到 0.85 时，总费用增加 3.50%，说明能量转换损耗会直接削弱跨时段套利能力；效率提高到 0.95 时，总费用下降 3.45%，但紧急购电费用并非简单单调下降，这是因为效率变化不仅影响总可用能量，也会改变最优充放电与常规购电时点。日末储能目标放开后总费用仅变化 0.064%，说明在原始数据和调度结构下，6000 kWh 终值约束不是主导成本的最强因素。

7.9 模型评价与本题小结

问题三把静态日前计划扩展为多阶段滚动优化，最大的优点是能够利用最新光伏预报和当前 SOC 逐步纠正计划偏差，同时通过明确的上调、下调结算函数保留原计划的合同约束。模型仍为线性或可线性化的凸分段线性规划，因此计算效率较高。主要局限在于负载信息、预报误差结构和价格规则均按原稿设定，尚未加入电池退化成本和更加复杂的概率预测。

从建模链条看，问题三已经具备模型预测控制的基本思想：每个更新时刻观察新信息，以当前实际状态为起点，对剩余时域重新优化，只执行下一段决策，再在新的信息到来后滚动重复。问题四将在这一框架上在此基础上加入随日期变化的价格序列，使得“预测修正”和“价格时点选择”同时影响购电策略。

8 问题四：波动电价下的微网购电策略

8.1 模型扩展思路

问题四不改变微网物理结构，而是把外网电价由固定日内曲线扩展为随日期d和时段t变化的$p_{d,t}$。这看似只是把目标函数中的价格参数换成二维序列，但实际影响贯穿全天调度：同一个储能状态在不同日期可能面对完全不同的峰谷价格，过去固定电价下习惯性的夜间充电、晚间放电时点也可能发生前移、后移或重新组合。

原稿分别在问题二和问题三基础上建立两条线路。Q4-2 保留历史情景日前模型，只把计划费用和紧急费用中的p_t替换为$p_{d,t}$；Q4-3 保留光伏滚动预报、调整结算和实时反馈机制，同样使用逐日逐时价格。这样可以在不改变原始模型逻辑的前提下，观察价格波动本身如何改变购电成本，以及滚动更新在波动电价下是否仍具有价值。

8.2 Q4-2：波动电价下的历史情景模型

Q4-2 的预测和情景仍由前 7 天负载与光伏数据构造。目标函数变为

$$\min\left\{\sum_{t=1}^{144} p_{d,t}\,x_{d,t}+\frac{5}{7}\sum_{s\in H_d}\sum_{t=1}^{144} p_{d,t}\,e_{d,s,t}\right\}.$$

式中第二项的$5/7$来自 5 倍紧急电价与 7 个等概率历史情景的组合。预测平衡、情景平衡、SOC 递推、容量与功率约束均沿用问题二。计划生成后，实际紧急购电仍按真实负载、真实光伏和固定执行的计划储能动作核算。

波动价格最直接的作用是改变每个时段的成本权重。如果某天清晨价格异常高，模型可能减少该时段常规购电而更早充电；如果午间价格明显下降且光伏充足，模型会同时考虑“免费光伏”和“低价外网电”对储能的竞争；如果晚间高价持续时间更长，储能可能提前保留更多 SOC 用于晚峰。所有这些变化都由同一线性规划由优化过程直接得到。

8.3 Q4-3：波动电价下的滚动调整

Q4-3 继续以 0:00 计划$x_{d,t}$为基准，在 6:00、12:00 和 18:00 根据新光伏预报与当前实际储能状态调整剩余时段。相对原计划的增购量和取消量定义为

$$u_{d,t}=\left[y_{d,t}-x_{d,t}\right]_+,\qquad v_{d,t}=\left[x_{d,t}-y_{d,t}\right]_+.$$

波动价格下的单时段结算函数为

$$\Phi_{d,t}(y;x)=p_{d,t}[\min(x,y)+1.5(y-x)^+ +0.5(x-y)^+].$$

因此，同样数量的调整在不同日期、不同时间会产生不同的经济代价。滚动模型不仅要判断“预测是否变了”，还要判断“现在改计划值不值得”。例如，在高价时段临时增购成本会被 1.5 倍放大，模型更倾向于通过提前储能或前一更新时刻增加购电来规避；而低价时段的增购相对便宜，可能成为修正后续缺口的主要手段。

8.4 实时反馈与费用分解

原稿在 Q4-3 执行层设置储能反馈：每个时段先按当前 SOC 和功率上限裁剪计划充放电；供电不足时先减少充电，再在可行范围内增加放电，最后才以紧急购电补足；供电过剩时先减少放电，再增加充电，仍无法消纳的部分记为剩余未利用电量。实际总费用由原计划费用、取消部分冲减、增购费用、取消违约费用和实际紧急购电费用共同组成。

这样的费用分解可以更清楚地说明为什么“滚动调整后常规购电量变化”不等于“总费用同方向变化”。某次更新可能增加常规购电，却显著减少 5 倍时价的紧急购电，从而降低总费用；也可能取消一部分高价计划并支付 50%违约费用，但仍然比原计划全额执行更便宜。因此，评价策略时仍应以完整结算费用为准。

8.5 报告期汇总结果

给出的 2025 年 2 月 1 日至 12 月 31 日汇总如下。

指标	Q4-2	Q4-3
原计划费用/元	14,161,635.97	12,930,509.23
取消部分原价冲减/元	0.00	373,664.44
增购费用/元	0.00	570,558.73
取消违约费用/元	0.00	186,832.22
紧急购电费用/元	5,670,949.98	272,749.51
实际总费用/元	19,832,585.95	13,586,985.25
计划购电量/kWh	23,245,721.57	20,445,379.03
最终常规购电量/kWh	23,245,721.57	20,366,283.25
紧急购电量/kWh	1,320,200.77	47,131.56
紧急购电日期数/天	287	289

Q4-2 实际总费用为 19832585.95 元，紧急购电费用 5670949.98 元；Q4-3 实际总费用为 13586985.25 元，紧急购电费用 272749.51 元。两条线路的绝对费用不能直接理解为“滚动更新比历史情景模型节省了多少”，因为二者在负载信息、日末规划条件和执行规则上存在差异。更合适的评价方式是分别与自身固定电价对照，或者在 Q4-3 内部改变预报更新组合。

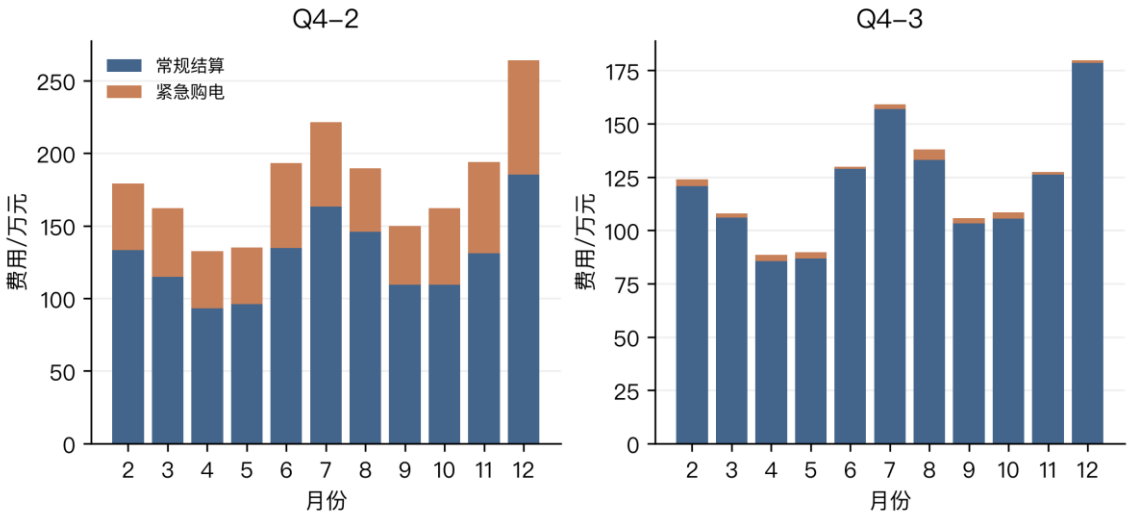


图 8-1 Q4-2 与 Q4-3 月度费用构成

8.6 固定电价与波动电价对照

模型	固定电价总费用/元	波动电价总费用/元	费用变化
Q4-2	19,357,655.64	19,832,585.95	2.45%
Q4-3	12,968,584.25	13,586,985.25	4.77%

在保持各自算法、数据和执行规则不变的条件下，Q4-2 由固定电价转为波动电价后总费用增加 2.45%，Q4-3 增加 4.77%。这一结果反映的是给定附件 4 价格序列相对于固定价格基准使这两套策略的实际结算费用上升，不能推广为“电价波动越大，成本必然越高”。费用变化同时受到平均价格水平、峰谷时点、负载与光伏的时间重合关系以及储能可转移空间影响。

8.7 预报更新组合的价值

使用的预报时刻	总费用/元	紧急费用/元	紧急电量/kWh
仅 0:00	15,260,769.51	2,330,348.29	352,370.43
0:00、6:00	14,574,509.59	1,338,488.95	204,988.57
0:00、6:00、12:00	13,926,777.05	747,040.89	118,426.59
0:00、6:00、12:00、18:00	13,586,985.25	272,749.51	47,131.56

仅使用 0:00 预报时，总费用为 15260769.51 元，紧急购电费用 2330348.29 元。加入 6:00 预报后，总费用下降到 14574509.59 元；再加入 12:00 后降到 13926777.05 元；使用全部 0:00、6:00、12:00、18:00 更新时，总费用进一步降到 13586985.25 元。相对于仅 0:00 预报，累计减少 1673784.26 元，降幅 10.97%，紧急购电费用下降 88.30%。

逐次更新的边际节省分别为 686259.92 元、647732.54 元和 339791.80 元。后续更新的边际收益逐步减小，符合“越早的信息覆盖时域越长”的直观规律，但 18:00 更新仍然具有明显价值。其原因在于 18:00 不仅带来新预报，还同时获得当天经过白天运行后真实形成的 SOC，模型可以据此重新安排夜间高低价时段的购电与放电，因此晚间更新并非冗余。

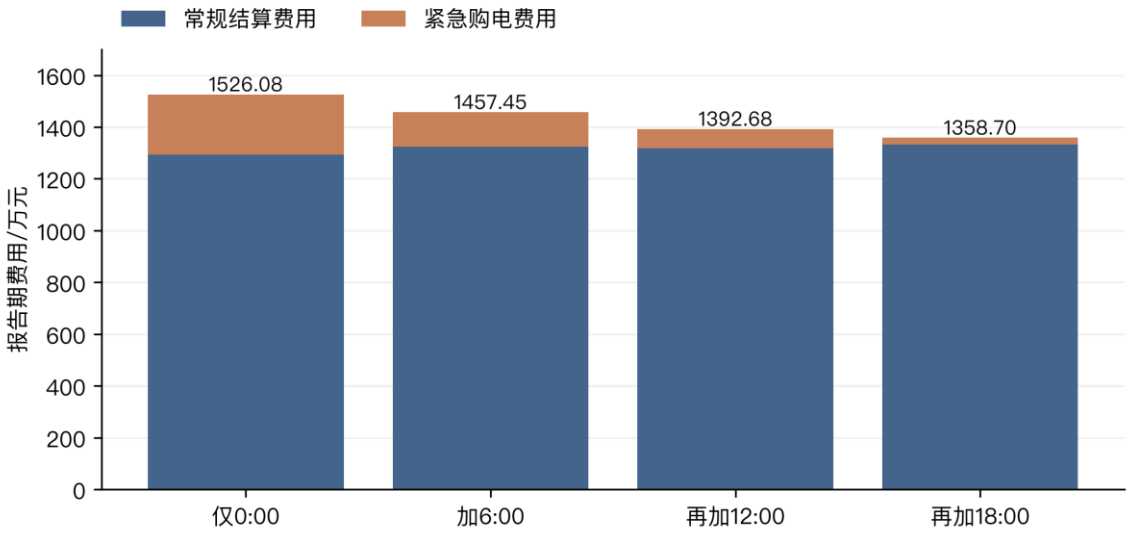


图 8-2 Q4-3 不同预报更新组合的费用构成

8.8 典型日期机理分析

以 9 月 23 日为例，Q4-2 的实际紧急购电量为 5994.16 kWh、费用 26772.27 元，而 Q4-3 对应紧急购电量仅 93.76 kWh、费用 465.80 元。两种方案面对同一天真实光伏和负载，但 Q4-2 在 0:00 后不再修改计划，历史预测误差只能通过高价紧急购电补足；Q4-3 则在 6:00、12:00 和 18:00 持续修正尚未执行的常规购电与储能安排，大部分偏差在真正形成缺口之前已被吸收。

12 月 21 日也呈现类似现象：Q4-2 紧急购电 6486.47 kWh，Q4-3 仅 6.61 kWh。需要注意，紧急购电日期数在 Q4-3 中并未下降到很低，报告期仍有 289 天发生过正紧急购电，但总紧急电量仅 47131.56 kWh。这说明“是否发生过紧急购电”是一个非常粗的二元指标，不能代替紧急购电规模和费用。许多日期可能只在极少数十分钟时段产生很小缺口，对总成本影响有限。

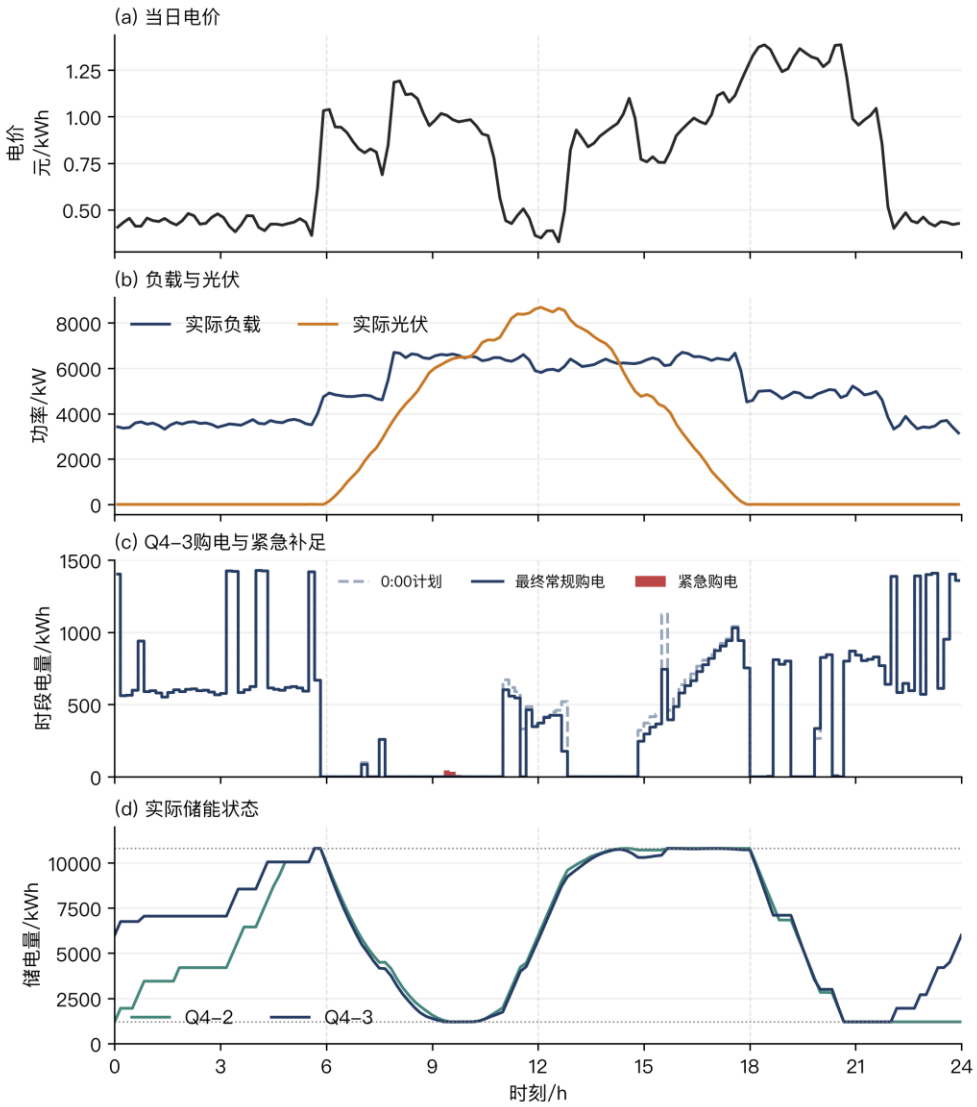


图 8-3 2025 年 9 月 23 日电价、负载、购电与储能轨迹

8.9 本题小结

问题四表明，在波动价格环境中，储能调度仍然遵循“低价吸收、高价释放”的基本原则，但具体充放电时点不再固定；光伏预报更新仍能明显降低高倍紧急购电费用。Q4-2 适合表示仅依靠历史信息的日前风险调度，Q4-3 则更接近具有日内信息更新能力的滚动控制。若实际应用中未来电价本身也不可知，则可在现有结构上进一步叠加电价预测或价格情景，但这属于原稿范围之外的后续扩展。

9 四题模型的统一解释与结果综合分析

9.1 从“价格套利”到“信息套利”

四个问题可以理解为两种价值来源逐层叠加。问题一主要利用价格在时间上的差异实现“价格套利”：储能把低价时段的购电和午间光伏搬移到高价时段使用。问题二引入预测误差后，模型开始为不确定性付费，计划购电不再只是寻找最低价格，而要避免未来 5 倍紧急电价。问题三的后续预报使系统能够利用“信息套利”：更接近实际时刻的新预报

可以提前纠正原计划，从而用较低的调整成本替代高昂的临时缺口成本。问题四则让价格套利和信息套利在逐日波动的价格环境中同时发生。

9.2 储能状态是四题共同的核心状态变量

无论采用确定性、情景还是滚动模型，SOC 都是连接过去与未来的唯一核心状态。某一时段将 SOC 充到上界，意味着未来继续吸收光伏或低价电的空间减少；某一时段将 SOC 放到下界，则意味着后续高价或高负载时段失去调节能力。因此分析单个时段购电量时必须同时观察前后 SOC，而不能只看当期价格。问题三和 Q4-3 的滚动更新之所以有效，一个重要原因就是它们在 6:00、12:00 和 18:00 重新读取“实际当前 SOC”，从而纠正之前预测误差累积造成的状态偏离。

9.3 紧急购电的经济含义

紧急购电是问题二以后模型中的可靠性保险机制。其价格为正常交易时价的 5 倍，意味着模型宁愿在日初或更新时刻多承担一定的常规购电与储能损耗，也不愿留下过大的供电缺口。高倍价格把供电可靠性以经济方式嵌入目标函数，而无需额外设置复杂的可靠性指标。实际结果中，问题二有 1322325.29 kWh 紧急购电，而问题三和 Q4-3 在滚动更新后紧急缺口明显缩小，说明信息更新能够把许多“事后被动补救”转化为“事前主动调整”。

9.4 两位小数展示与内部精度

按照本次整改要求，正文、摘要、表格、结论与图表中的电量和费用统一保留两位小数。需要区分的是，展示精度不等于计算精度。原稿中的线性规划求解与费用汇总均使用更高内部精度，833.33 kWh 只是$5000/6$的显示近似值；最终汇总应先基于未舍入值计算，再统一格式化显示，以避免大量十分钟时段逐项舍入造成累计误差。

10 模型检验与一致性说明

本节不重新进行数据核验或模型复算，主要整理原始四题论文已经给出的检验性结论，并把它们放入统一的论文结构中。

问题一原稿报告，144 个时段能量平衡最大残差约为2.3×10^{-13} kWh，SOC 递推最大残差约为9.1×10^{-13} kWh，日末 SOC 严格回到 6000 kWh，储能始终位于 1200—10800 kWh 区间，且没有出现同时充放电。上述结果说明原始求解输出在其既定数学模型下满足约束。

问题二结果指出，全年结果满足 1200—10800 kWh 储能范围，最大充放电量为 833.33 kWh/10 分钟，计划平衡、情景平衡和 SOC 递推残差均接近机器精度。由于问题二按日期顺序连续推进，跨日状态由前一天 24:00 传递给下一天 0:00，因此还需关注全年状态链是否连续；前述计算已经按此规则生成结果。

问题三原稿通过多情景比较和关键参数灵敏度分析检查滚动策略表现。仅计划、6:00 更新、6:00+12:00 更新、全部三次更新和完美信息五种情景形成逐步降低的总费用序列，表明新增预报信息与模型目标之间具有一致的经济方向。效率与日末储能目标灵敏度结果则显示，储能效率变化会有明显影响总费用，而日末目标在该数据集下影响较小。

问题四原稿报告，两套线路的 48096 个时段中储能均位于 1200—10800 kWh 范围，最大充放电量不超过$5000/6$ kWh，未出现同时充放电；能量平衡、SOC 递推和费用复算残差保持在极小数值范围。原稿还通过修改 9 月 23 日 6:00 之后的实际光伏进行信息顺序扰动测试，以确认既有计划和 6:00 更新没有使用未来实际光伏信息。

11 灵敏度与稳健性讨论

四题中最明确的参数灵敏度来自问题三。储能效率由 0.90 降为 0.85 时，总费用增加 3.50%；提高到 0.95 时，总费用下降 3.45%。其物理原因是效率直接决定跨时段搬移 1 kWh 可最终输出多少有用电量。效率越高，低价充电和光伏储存越有价值，储能更容易替代高价外网购电；效率越低，频繁充放电的损耗越大，模型会更依赖常规购电。

日末储能目标放开后总费用变化仅 0.064%，说明在问题三给定数据和滚动策略下，模型自然形成的末端状态与 6000 kWh 目标较接近，或终端约束的机会成本较小。这并不意味着终值约束在所有日期都不重要，而只是原稿数据条件下的结果。问题一中周期边界对可重复日循环具有结构性作用，问题二和 Q4-2 则采用跨日实际状态连续，因此不同问题的终端条件具有不同含义，不能互相替代。

价格层面的稳健性可以通过问题四固定电价与波动电价对照理解。Q4-2 费用增加 2.45%，Q4-3 增加 4.77%，表明策略对价格序列具有敏感性，但费用变化不是单由“波动程度”决定。若高价恰好出现在储能可放电且负载较高的时段，储能可能产生更高价值；若低价与可充电空间不匹配，价格下降也未必能充分转化为成本下降。因此评价价格敏感性应结合负载、光伏与 SOC 共同分析。

历史窗口长度、预报更新时间和紧急购电倍数也是潜在敏感参数。原稿固定使用 7 日历史窗口和题目给定三个后续更新时刻，因此本文不新增未计算的数值结果，只从结构上说明：更长历史窗口可能提高统计稳定性但削弱对近期变化的响应；更多更新时点理论上提供更多信息，但每次重优化的边际收益可能递减；紧急购电倍数越高，模型越倾向于提前增加常规购电或保留储能安全裕度。

12 模型优缺点与改进方向

12.1 主要优点

第一，四题共享统一的能量平衡与 SOC 状态模型，符号和约束可以自然递进，避免每道题重新建立互不兼容的模型。第二，核心问题均可保持为线性规划或经过辅助变量线性化的分段线性规划，求解效率高，适合 144 时段和全年 334 天的重复计算。第三，模型把题目给定的价格规则、储能效率、容量和功率边界直接写入数学约束，物理含义清晰。第四，问题二以后严格区分计划信息与实际信息，问题三和 Q4-3 只在新预报到达后修改未来时段，能够较好地体现信息获取的先后顺序。第五，结果输出能够直接对应题目要求的指定时段购电量、四小时充放电量、紧急购电区间和全年汇总指标。

12.2 主要不足

首先，储能模型只描述能量和效率，没有考虑循环寿命、容量衰减、温度、SOC 依赖效率等电化学因素，因此当频繁充放电发生时，经济成本可能被低估。其次，问题二的 7 日等概率历史情景是一种简洁经验方法，未显式描述天气类型、季节性或误差概率分布，极端情况下的缺口风险可能不完全被历史窗口覆盖。再次，问题三和 Q4-3 依赖给定发布时刻的点预测，未直接利用概率区间或置信度信息。最后，问题四按原稿把当日电价曲线视为调度输入，如果真实市场中未来价格也存在不确定性，则还需要额外的价格预测或随机价格情景。

12.3 可拓展方向

在不改变现有模型骨架的情况下，可将电池循环退化成本加入目标函数，以每单位充放电量对应的寿命损耗价值抑制无必要的高频循环；可把问题二的历史情景扩展为聚类情景或基于气象条件的条件情景，使预测误差描述更具代表性；可在问题三中使用光伏概率预测，把滚动优化由单一轨迹扩展为多情景模型预测控制；也可在问题四中联合建立电价与光伏的多变量情景，从而研究价格和新能源不确定性同时存在时的鲁棒策略。

这些方向都属于原模型的自然延伸，但本文严格保留原始模型、数据和计算结果，因此只作为模型评价中的改进建议，不新增任何未经原稿计算支持的数值结论。

9A 分题建模推导与运行机理深化

9A.1 问题一目标函数与约束之间的相互作用

问题一的目标函数看似只包含购电费用，但实际决定最优策略的并不是价格一项，而是价格与储能状态约束、光伏供给和负载需求共同作用形成的可行域。若没有储能容量和功率限制，模型可以在全日最低价时段大量购电并把电能全部搬移到高价时段；现实中 1200—10800 kWh 的安全区间限制了可搬移总量，5000 kW 充放电功率则限制了单位时间的搬移速度。因此，当 SOC 触及上界时，即使后续仍处于低价时段，也不能继续无限充电；当 SOC 触及下界时，即使当前电价很高，也必须依赖外网满足剩余负载。本文采用的模型中多次出现 SOC 边界被激活，正说明储能的物理限制对经济优化结果具有直接约束作用。

从边际价值来看，每一个 SOC 约束都可以看成“未来调节空间”的隐含价格。处在低价时段时，增加 1 kWh 充电会增加当前购电费用，却可能在后续高价时段减少更昂贵的购电；处在高价时段时，放电 1 kWh 虽然立即节约费用，却会降低后续 SOC。如果未来存在更高价格，当前放电的机会成本就会提高。线性规划线性规划求解过程相当于在全天范围内比较这些边际价值，最后得到满足所有状态约束的最小成本路径。

9A.2 光伏富余与储能空间的联动

光伏本身没有直接购电成本，因此在负载范围内应优先被本地消纳。当光伏超过负载时，多余电量是否仍有价值取决于储能是否具备可用容量。如果储能 SOC 较低，光伏富余可以转化为后续可用能量，相当于替代未来外网购电；如果 SOC 已经接近上界，继续出现光伏富余就可能形成未利用电量。由此可以看出，前一阶段的充电决策会影响后一阶段光伏的可消纳空间。若夜间把电池充得过满，午间可能无法充分吸收光伏；若夜间完全不充，又可能错过低价电价优势。最优策略必须在低价购电与午间留出容量之间进行平衡。

问题一结果显示，储能在夜间和午间分别形成充电过程，并在晨峰与晚峰分别放电，体现了两轮能量搬移。第一轮主要由夜间低价电向早晨高价时段转移，第二轮主要由午间光伏与低价电向下午、晚间高价时段转移。两轮之间的 SOC 下降不仅是在“用掉电能”，也在为午间光伏腾出储能空间，这种空间价值是只看购电曲线难以发现的。

9A.3 问题二中七日窗口的决策含义

七日历史窗口兼具预测和风险描述两种功能。其均值用于构造单一的日前中心预测，而七条完整历史轨迹又作为情景进入期望紧急费用。若只使用均值预测，某些正负偏差可能相互抵消，使预测曲线看起来平滑，却无法反映真实日内波动；加入历史情景后，计划必须在每一条历史轨迹下评估供电缺口，因此会对偏差较大的时段保留更多安全余量。

七日窗口还是一种随时间移动的自适应机制。进入新的日期后，最旧的一天从窗口中移除，最近一天加入，预测和情景集合随近期负载与光伏特征改变。这比全年固定平均曲线更能响应季节缓慢变化，但对突然发生的天气异常仍可能反应不足。原稿全年大量日期存在紧急购电，正体现了“短历史窗口能够降低风险但不能完全消除未来不确定性”的客观特征。

9A.4 问题二目标函数的风险偏好解释

目标函数中正常购电按原价计费，情景紧急购电按 5 倍价格计费，相当于对计划不足施加明显更高的边际惩罚。假设某时段增加 1 kWh 全天计划购电，会立即增加p_t元成本；如果这 1 kWh 能够在多个历史情景中减少紧急缺口，其期望收益最高可达到接近$5p_t$元。于是模型会主动用部分常规购电成本换取缺口风险下降，这就是问题二相比问题一更“保守”的经济原因。

但这种保守不是无限度的。若某时段绝大多数历史情景都显示光伏充足，再额外增加全天计划购电只会形成剩余未利用电量，那么增加计划的边际收益会很小，模型仍会选择较低购电量。换句话说，5 倍紧急电价并不会简单把所有时段计划都提高 5 倍，而是通过情景供需结构决定哪些时段值得增加保险性购电。

9A.5 跨日 SOC 连续的必要性

问题二和 Q4-2 采用$S_{d+1,0} = S_{d,144}$，避免了人为把每天储能状态重置到固定值。若每天都强制从同一 SOC 开始，相当于假定午夜存在一个不计成本的外部机制自动补足或释放电能，会破坏全年能量守恒。连续传递则让前一天末端决策对下一天产生真实影响：前一天为了降低晚间高价费用而大量放电，会导致次日清晨 SOC 较低；前一天晚间低价时主动充电，则可能改善次日早晨的供电能力。

这种跨日耦合使全年调度更接近实际运行，同时也解释了为什么不能仅凭某一天局部结果评价模型。某日 24:00 保持较高 SOC 可能看似增加当天费用，却可能减少下一日凌晨或晨峰购电。原稿通过按日期顺序逐日求解并传递状态，保留了这种长期连续性。

9A.6 问题三基准合同的经济角色

0:00 计划在问题三中不再只是一个可随时替换的临时方案，而是后续结算的基准合同。若后续预报变化很小，直接执行原计划通常最经济，因为任何上调或下调都会引入附加成本；只有当预测偏差足以导致更大的常规购电或紧急购电损失时，调整才值得发生。这样的设计使滚动优化同时具有“跟踪新信息”和“保持计划稳定”的双重目标。

从优化形态看，分段线性结算函数在$y = x$处形成折点。折点左侧表示取消计划，边际代价与 0.5 倍违约规则有关；右侧表示新增购电，边际代价按 1.5 倍价格计算。这个折点使原计划本身具有黏性：在新预报只造成轻微变化时，保持$y = x$可能仍是最优；当预期缺口成本跨过阈值后，才会进入上调或下调区域。这种机制比“每次预报一来就完全重算、完全免费修改计划”更符合题目设定。

9A.7 6:00、12:00 与 18:00 更新的不同作用

三个更新时刻虽然都提供光伏新预报，但其作用并不相同。6:00 距离全天结束最远，可影响上午、午间、下午和晚间的大部分时段，因此具有最大的可调整时域。12:00 更新发生在光伏高峰附近，能够直接判断午后光伏是否足以把储能重新充高，并决定晚峰前是否需要提前购电。18:00 更新时光伏已接近结束，其直接预测价值减少，但当前 SOC 已包含全天实际光伏偏差和前两次调整的累积结果，所以 18:00 可以作为晚间调度的状态校正节点。

本文采用的模型中三个更新时刻的边际节省依次下降，与可调整时域逐步缩短的特征一致。不过边际收益并不是纯粹由“剩余时长”决定，还受到当日价格峰谷、晚间负载和 SOC 位置影响。如果 18:00 时 SOC 偏低而晚间价格很高，最后一次更新仍可能产生显著价值。因此不能仅根据光伏即将结束就判断 18:00 预报无用。

9A.8 预测误差如何传导为费用

预测误差不会直接等于紧急购电量。假设某时段实际光伏比预测低 100 kWh，如果储能仍有足够放电空间，模型可以通过增加放电吸收这 100 kWh 偏差，紧急购电可能仍为 0；如果储能已接近下界，则相同的光伏误差会几乎全部转化为紧急购电。反之，实际光伏高于预测时，如果储能还有充电空间，可以减少常规购电或增加充电；若储能已接近上界，则多余光伏可能成为未利用电量。

因此预测误差、SOC 和价格共同决定费用影响。滚动模型的优势之一，就是在误差真正发生之前利用新预报改变 SOC 轨迹。例如预报显示下午光伏将明显低于原判断，模型可以在中午前保持更高 SOC；这实际上是在把未来预测误差提前转化为一个较低成本的常规购电决策，而不是等到缺口发生后按 5 倍价格补救。

9A.9 波动电价下的相对价格排序

在固定日内电价中，夜间通常便宜、晚间通常昂贵，所以结果呈现稳定的充放电时序。进入问题四后，实际决定储能动作的是相对价格排序而不是时钟标签。模型会比较当前价格与未来可能避免的价格，同时考虑效率损耗与 SOC 空间。若某天凌晨价格并不低，储能可能减少充电；若某天下午出现异常低价，即使传统上属于高负载时段，也可能出现购电充电行为。

这也说明结果图表分析应避免把“夜间充电、晚间放电”写成固定规则。更准确的表述是“在可行条件下，储能倾向于在相对低价时段吸收能量，并在相对高价时段释放能量；光伏富余会进一步降低充电的机会成本”。这样既能解释问题一的典型日规律，也能自然推广到问题四逐日变化的价格环境。

9A.10 Q4-2 与 Q4-3 不能直接做单因素因果比较

Q4-2 与 Q4-3 的结果差异非常大，但两套模型不仅在“是否更新预报”上不同，还在负载信息、日末规划条件、执行反馈等方面沿用了各自问题二和问题三的设定。因此若直接用两者总费用之差宣称“滚动预报节省了某个固定比例”，就混合了多个因素。本文采用两种更稳妥的比较方式：一是分别把 Q4-2、Q4-3 与自身固定电价版本比较，用于观察价格序列变化；二是在 Q4-3 内部保持其他条件不变，只改变预报更新时间组合，用于识别更新信息的价值。

这里需要注意控制变量，否则容易把不同因素的影响混在一起。模型比较只有在其他条件保持一致时，才适合把费用变化归因于某一个机制。对不同信息结构的模型，可以比较总体性能，但应明确说明差异来源不止一个，避免过度因果解释。

9A.11 为什么紧急购电日期数可能上升但总费用下降

Q4-3 使用全部更新时刻后，紧急购电日期数仍然较高，但紧急电量和费用非常低。这说明一个日期只要有任意十分钟出现微小正缺口，就会被计为“发生紧急购电”，因此日期数对缺口严重程度不敏感。滚动调整的目标不是保证每一天都严格没有任何紧急购电，而是减少缺口的规模和高价时段集中程度。

从运营角度看，少量、短时紧急购电可以视为预测误差无法完全消除时的最后安全阀。若为了把紧急购电日期数降到零而在所有时段都大量增加常规购电，反而可能显著提高总成本并产生大量剩余电量。因此评价策略应优先关注总费用、紧急购电量、紧急费用以及典型高风险时段，而不是单独追求“紧急购电日期数最少”。

9A.12 储能效率对套利阈值的推导

设低价时段购入 1 kWh 电用于充电，充电后电池内部增加η_c kWh；随后全部放出时，能够向微网提供$\eta_c\eta_d$ kWh。如果低价为p_L、高价为p_H，不考虑容量和功率边界时，套利成立的基本条件为

$$p_L < \eta_c \eta_d p_H.$$

等价地，

$$\frac{p_H}{p_L} > \frac{1}{\eta_c \eta_d}.$$

当$\eta_c = \eta_d = 0.90$时，临界比值约为 1.235。效率下降会提高临界峰谷比，使原本有利的部分充放电循环变得不再经济；效率提高则扩大可套利时段集合。问题三灵敏度结果中，效率从 0.90 降低到 0.85 使总费用上升，而提高到 0.95 使总费用下降，与这一机理完全一致。

9A.13 功率上限与容量上限的区别

容量上限决定“最多能存多少”，功率上限决定“多快能充进去或放出来”。即使 SOC 距离上界还有很大空间，若低价时段持续时间很短，833.33 kWh/10 分钟的充电上限仍可能使储能无法在低价窗口内充足；同样，即使电池内部储电量很高，遇到负载突然上升时，放电功率上限也可能使储能无法完全覆盖缺口，只能同时从外网购电。

因此，看到某个高价时段仍有常规购电，不能据此直接判断模型没有充分利用储能。可能的原因包括 SOC 已经接近下界，也可能是单时段放电上限不足以覆盖净负荷。问题一在部分晚峰时段仍然存在购电，即可从这种功率与状态双重约束理解。

9A.14 结果可视化应表达的四条主线

完整论文中的图表应围绕四条主线展开。第一条是外生输入，即价格、负载和光伏随时间的变化；第二条是控制动作，即购电、充电和放电；第三条是状态轨迹，即 SOC 如何在上下界之间演化；第四条是成本构成，即常规购电、调整

费用和紧急购电分别贡献多少。只有把四条主线联系起来，读者才能从“为什么这个时段买电”追踪到“这一动作如何改变 SOC、又如何影响后续费用”。

题目要求的指定时段表格适合作为定量结果摘要，而典型日曲线适合展示机理，全年月度费用图则适合展示长期趋势。三者承担不同功能，不能互相替代。因此本文以正文解释为主、表格为定量支撑、图形逻辑由原稿配套结果保留在支撑材料中，避免用大规模表格堆砌代替建模论证。

9A.15 线性规划全局最优性的意义

四题核心模型均保持在线性规划或可线性化范围内，因此在给定模型和输入条件下，求解器得到的不是某个依赖初值的局部方案，而是线性可行域上的全局最优解。这个性质使模型结果具有较强可复现性：只要使用相同的数据预处理、变量定义、边界和目标函数，不同标准线性规划算法应当得到相同或等价的最优目标值。

不过，“数学模型中的全局最优”不等于“真实世界绝对最优”。模型没有包含的电池寿命、设备启停、交易规则和预测误差分布不会自动出现在解中。因此论文应把最优性限定为“在题目给定假设与原始模型约束下的最优”，并通过模型评价说明现实扩展方向。这种边界表述既保持数学严谨，也避免把竞赛模型的结论夸大为真实电力系统的无条件最优策略。

9B 求解流程、结果表达与论文论证进一步补充

9B.1 从原始数据到决策变量的完整转换链

四题的模型都建立在同一条数据转换链之上。附件首先给出功率或价格数据，模型并不直接把 kW 与 kWh 混合使用，而是在进入优化之前统一进行单位转换。负载功率和光伏功率乘以十分钟时段长度$1/6$ h 后得到时段电量，再与购电量、充电量和放电量共同进入能量平衡式。这样的处理避免了“功率上限与电量变量直接比较”的量纲错误，也使 5000 kW 最大充放电功率自然转化为每十分钟不超过 833.33 kWh 的能量上限。

时间标签同样需要统一。题目中的 0:00+1 表示次日 0:00，即当天 24:00；在完整 144 时段序列中，每一个时间标签都应对应唯一的十分钟区间。论文正文使用实际区间如“10:00—10:10”描述结果，模型内部则使用$t = 1,\ldots,144$编号，二者通过固定映射对应。这样既便于建立矩阵模型，也便于最终按题目模板输出指定时段。

9B.2 线性规划矩阵的构造逻辑

将全部变量按固定顺序堆叠后，能量平衡和 SOC 递推可以写成标准线性规划形式。能量平衡矩阵的每一行只涉及当前时段的购电、充电、放电、剩余量等少数变量；SOC 递推行则涉及当前 SOC、前一 SOC 以及当前充放电变量，因此整个约束矩阵非常稀疏。利用稀疏矩阵存储可以明显降低全年逐日求解所需内存，并提高求解器组装速度。

问题二每一天还要为 7 个历史情景分别增加紧急购电与剩余量变量，但公共计划变量只保留一套。因此模型规模虽然扩大，却仍具有规则块结构：上方是公共计划与 SOC 约束，下方是七组结构相同的情景平衡约束。问题三的滚动模型在 6:00、12:00 和 18:00 只截取剩余时段，相当于从完整日模型中移除已经执行的前缀，因此后续模型规模逐步缩小。

9B.3 为什么不用整数规划表示充放电互斥

常见储能模型会设置二元变量保证单个时段不能同时充电和放电，但原稿没有引入此类 0-1 变量，而是保持连续线性规划。其理由在于充放电效率均小于 1，且外网购电价格为正。若同一时段同时增加相同规模的充电和放电，不会改变对负载的净供能，却会因效率损耗消耗更多能量，通常不会降低购电成本。原稿的结果检查也没有发现需要额外互斥变量修正的同时充放电现象。

保持连续模型具有明显计算优势。全年 334 天、每天天内 144 时段并叠加多次滚动更新，如果引入大量二元变量，模型会从线性规划变为混合整数规划，求解复杂度显著提高。现有连续模型已能够得到可执行策略，因此终稿保留这一设计，不人为增加原模型没有使用的整数约束。

9B.4 典型日分析的正确顺序

对典型日期进行解释时，应先观察外部输入，再解释控制动作，最后讨论费用。第一步比较负载与光伏：判断哪些时段存在天然净负荷、哪些时段可能出现光伏富余。第二步观察电价：确定低价、高价以及峰谷转换区间。第三步结合SOC 轨迹解释充放电：如果低价时 SOC 较低，出现充电是合理的；如果高价时 SOC 较高，出现放电符合套利逻辑；如果高价时仍然购电，则需继续检查放电功率上限或 SOC 下界。第四步才解释紧急购电和总费用，避免把费用变化简单归因于单一变量。

这种分析顺序能够减少“看见相关就当作因果”的问题。例如某天晚间紧急购电较高，不能直接说是“负载高造成”，因为同一高负载如果 SOC 充足也可能没有紧急购电；更完整的解释应是实际净负荷、计划群购电、可用 SOC 和放电功率共同导致剩余缺口，最终由紧急购电补足。

9B.5 月度费用图的解释边界

全年月度费用图可以显示季节变化和成本构成，但不能单独证明某个月模型更优或更差。月度总费用受天数、负载水平、光伏资源、价格水平和预测误差共同影响。若某月费用较高，可能只是负载整体上升或价格水平较高，并不一定代表算法性能下降。因此月度图适合作为宏观趋势展示，算法效果仍需通过同一数据条件下的控制变量对照评价。

问题四的固定电价—波动电价对照和 Q4-3 不同预报组合对照就是更有解释力的实验。前者保持算法不变，只改变价格序列；后者保持价格和执行规则不变，只改变更新时刻。这样的对照能够较清楚地区分价格变化和信息更新的影响，是全文结果分析中最重要的比较原则之一。

9B.6 费用组成必须避免重复计算

问题三和 Q4-3 的调整结算涉及原计划、取消、增购和紧急购电，最容易出现重复计费理解。最终常规购电量y不是在原计划x之上再次叠加的额外供给，而是该时段调整后的最终常规购电总量。对于被取消的部分，应先从原计划正常购电中冲减，再按题目规则支付 50%违约成本；对于新增部分，则按 150%时价支付。紧急购电是在最终常规购电和储能反馈之后仍存在缺口时另行发生。

终稿沿用原稿的替代结算口径，对每个十分钟时段相对于 0:00 计划只进行一次最终结算，不把 6:00、12:00、18:00 各中间版本之间的差额层层累加。这样能够保证同一笔购电调整不会被重复处罚或重复加价，也使全年费用能够由逐时结算项清晰汇总。

9B.7 模型结果与竞赛输出文件之间的对应

题目不仅要求论文中的汇总表，还要求完整策略写入多个 Excel 结果文件。问题一需要输出 144 个十分钟计划购电量以及六个四小时充放电汇总；问题二需要按日期保存全天计划、充放电与连续紧急购电区间；问题三增加调整购电量；问题四分别保存 Q4-2 与 Q4-3 全部结果。论文中的指定日期和指定时段只是这些完整结果文件的抽样展示。

因此论文建模过程应与结果文件字段保持一一对应。例如$x_{d,t}$直接对应“计划购电量”，$y_{d,t}$对应“调整购电量”或最终常规购电量，$e_{d,t}^{\mathrm{act}}$经连续正值时段合并后对应“紧急购电量”工作表。这样的对应关系提高模型可复现性，也方便评阅者核对论文表格与支撑材料的一致性。

9B.8 为什么正文不能只放表格

数模竞赛论文的评分重点不只是最终数值，更在于模型为什么这样建立、每个约束代表什么、算法为什么适合以及结果为什么呈现当前规律。若正文只列大量表格，读者只能看到“算出了什么”，却无法判断“为什么这样算”和“结果意味着什么”。本文整理因此把表格控制为必要的关键结果摘要，把主要篇幅用于解释状态递推、风险机制、滚动更新与波动电价的机理。

尤其对问题二至问题四，单纯列出几组紧急购电量无法体现模型的层次结构。正文必须明确日前计划使用什么信息、实际数据何时可用、新预报何时进入模型、已执行动作为什么不能修改、紧急购电为什么属于最后一层补救。只有把信息时序写清楚，模型才具有真正的可执行含义。

9B.9 四题之间的递进关系

问题一建立确定性物理骨架；问题二在骨架上增加不确定性情景和高价缺口惩罚；问题三进一步增加日内信息更新与调整结算；问题四再把固定价格权重替换为逐日波动价格。每一步都保留前一步的核心约束，只新增题目要求的新机制，因此四题可以视为同一基础模型的层级扩展，而不是四套彼此孤立的方法。

这种递进关系也有助于组织论文结构。问题一详细说明能量平衡和 SOC 方程后，问题二无需重复大段基础定义，只需强调历史情景与紧急购电；问题三重点说明滚动更新和分段结算；问题四则突出价格参数变化和对照实验。通过“基础模型—不确定性—滚动控制—价格扩展”的主线，全文逻辑可以保持统一而不重复堆砌。

9B.10 结论的适用尺度

本文的所有数值结论都限定在题目给定数据、原稿模型和既定参数下。例如问题三全部更新比仅计划节省 8.68%，问题四 Q4-3 全部更新比仅 0:00 预报节省 10.97%，这些比例是特定数据集的计算结果，不应推广为任何微网都能获得相同节省。可以推广的是定性机制：更及时的预测通常有助于减少因预测偏差造成的高价缺口，储能效率越高越有利于跨时段能量搬移，波动价格会改变最优充放电时点。

明确这一适用尺度能够使论文结论更严谨。数模竞赛的目标是建立可解释、可计算的模型并在给定数据上求解，而不是在有限数据基础上作无限外推。因此本文保留原稿具体数值，同时在模型评价中明确其条件与边界。

12A 综合机理分析与论文论证补强

12A.1 时间尺度、能量守恒与决策尺度的一致性

四个问题虽然在信息结构和价格机制上逐步复杂，但其共同的物理基础始终是十分钟时间尺度上的能量守恒。附件给出的是功率数据，而优化变量采用电量，因此所有负载和光伏功率首先乘以单时段长度 $\Delta t = 1/6\,\mathrm{h}$ 转化为 kWh。这个转换并非单纯的单位处理，它决定了储能功率上限与能量变量是否处在同一量纲。题设给出的最大充放电功率为 5000 kW，换算为一个十分钟时段的最大能量后才得到 $5000/6$ kWh。只有在完成这一统一后，购电量、光伏电量、负载电量、充电量和放电量才能在同一平衡方程中直接相加减。

从决策角度来看，十分钟离散还兼顾了精细程度与计算规模。若时间尺度过粗，峰谷电价切换、光伏快速变化和 SOC 边界触发会被平均掉，使得充放电时点失真；若时间尺度进一步缩短，则全年滚动问题的变量和约束数量迅速扩大。题目本身按十分钟提供或要求结果，因此 144 时段既与数据粒度一致，也使四题可以共享同一套状态转移框架。全文统一采用这一尺度，保证问题一的确定性模型、问题二的历史情景模型、问题三的滚动调整模型以及问题四的波动电价模型可以直接比较。

12A.2 储能状态变量在四题中的桥梁作用

储能状态 $S_{d,t}$ 是四个问题中最重要的跨时段变量。某一时段是否值得放电，不能只看该时段电价，还要考虑放电后剩余 SOC 对未来高价时段的影响；某一时段是否值得充电，也不能只看当前低价，还要考虑容量上界是否会压缩后续吸收光伏的空间。因此 SOC 使每个十分钟决策彼此耦合，模型实质上是在整条时间轴上分配有限的储能空间和可释放能量。

问题一使用 $S_0 = S_{144} = 6000$ 的周期边界，强调单日策略的可重复性。问题二采用跨日连续传递，使前一天的实际末端状态成为下一天初始状态，体现连续运行。问题三和 Q4-3 在滚动更新时又把“当前实际 SOC”作为新一轮优化的初始状态，使预测更新不仅修正未来光伏信息，也修正此前执行造成的状态偏差。由此可见，SOC 并非附属约束，而是连接日前计划、日内执行和跨日运行的核心状态量。

12A.3 费用结构对策略保守程度的塑造

问题二以后，模型不再只比较普通电价下的购电费用，而是同时面对常规购电、调整费用和紧急购电费用。紧急购电按交易时刻电价的 5 倍计价，使供电不足具有显著惩罚；问题三和 Q4-3 又规定上调购电按 1.5 倍价格计费、取消计划承担 50%的违约代价。不同费用系数共同塑造了策略的保守程度：计划过低会暴露于高昂紧急购电，计划过高则增加普通购电并可能形成剩余电量，而频繁大幅调整又会产生额外结算成本。

因此，最优策略不是单纯追求“计划越准越好”，而是在不同错误方向的经济代价之间取平衡。若未来光伏被高估，实际供给不足，损失主要通过紧急购电体现；若未来光伏被低估，常规购电可能偏多，产生剩余或取消计划的成本。滚动更新的意义就在于，在新信息到达后重新评价这两类风险，用较低的调整代价替代可能发生的高倍紧急成本。原稿关于不同更新组合的费用结果正体现了这一机制：随着 6:00、12:00 和 18:00 预报逐步加入，总费用与紧急购电费用总体下降，但边际节省逐步变化，说明信息价值与剩余决策时域、当前 SOC 以及当日预测误差共同相关。

12A.4 约束活跃性与结果解释方法

线性规划结果中的“边界值”具有直接的物理含义。当充电量达到单时段上限时，说明该时段继续充电仍具有经济吸引力，但受到设备功率限制；当 SOC 达到 10800 kWh 时，说明继续吸收低价电或光伏会受容量安全上界限制；当 SOC 降至 1200 kWh 时，说明后续即使处于高价时段，也不能继续无限放电。因而边界频繁激活并不表示模型异常，恰恰说明价格差、光伏时移价值或紧急成本足以把储能推向物理极限。

同样，四小时汇总表中出现“同一大时段既有充电又有放电”也不能被误解为储能在同一个十分钟时段同时充放电。四小时区间由 24 个十分钟子时段构成，价格、负载和光伏在其中不断变化，前半段充电、后半段放电完全可能在汇总后同时显示为正值。论文因此应优先依据十分钟决策和 SOC 递推解释运行逻辑，再把四小时表作为题目要求的汇总展示，而不能由汇总表反推单时段动作。

12A.5 从问题一到问题四的递进关系

四题并非四个互不相干的模型，而是围绕同一物理系统逐步加入现实因素。问题一解决“已知全天信息时怎样最低成本运行”，建立基准能量平衡和储能模型；问题二进一步回答“日初信息不完全时怎样利用历史样本控制缺口风险”，把确定性优化扩展为情景期望优化；问题三研究“运行过程中获得新预报后是否值得修改原计划”，把单次日前决策扩展为多阶段滚动决策；问题四则把价格从固定日内曲线扩展为逐日逐时变化，使储能不仅应对光伏和负载不确定性，还需要随当日电价形态动态改变充放电时点。

这种递进结构使全文具有统一的数学主线：物理约束基本保持不变，变化主要发生在信息集、费用函数和决策时机。这样既可以避免四题重复建立完全不同的符号系统，也便于说明后续模型如何继承前题、增加何种变量、为什么增加这些变量以及新增机制对结果产生什么影响。

12A.6 可实施性与工程含义

如果从实际运行的角度来看，模型输出可被理解为三层指令。第一层是日前计划，给出每个十分钟时段的常规购电基准以及储能充放电安排；第二层是滚动修正，仅在新的预测发布时重新优化尚未执行的时段；第三层是实时兜底，在真实供需出现偏差时通过调整充放电和紧急购电保证负载始终得到满足。三层之间按时间先后衔接，避免使用未来信息修改已经执行的动作。

这一结构也说明模型结果的适用边界。题目中的外网购电没有额外功率上限，电池效率取固定值，未考虑寿命退化、温度和循环深度成本，也没有建立上网售电收益，因此结果应解释为题设规则下的最优调度方案，而不是包含所有市场机制的工程控制器。若应用到真实微网，可在现有状态方程和能量平衡框架上继续增加电池退化成本、需求响应、可中断负荷、备用容量、购售电双向交易以及价格预测情景。由于本文核心模型保持线性或可线性化，这些扩展多数可以通过增加变量和约束实现，具有较好的可扩展性。

13 总结与结论

本文围绕“微网如何在光伏、储能和外网购电之间进行日内协调”这一主线，对四个问题进行了统一重构。问题一在确定性条件下建立 144 时段线性规划，通过低价充电、高价放电和午间光伏吸收实现峰谷套利，原始结果为全天购电 59482.70 kWh、费用 35126.95 元，并满足 0:00 与 24:00 储能量均为 6000.00 kWh。

问题二在日前信息有限的条件下引入前 7 日历史情景，把计划费用与 5 倍电价紧急购电期望同时纳入目标。2025 年 2 月 1 日至 12 月 31 日的原始汇总结果为计划购电费用 14092176.50 元，实际紧急购电量 1322325.29 kWh，实际紧急购电费用 5265479.14 元，实际总费用 19357655.64 元。结果表明，单次日初计划面对光伏与负载偏差时仍会产生较多紧急缺口。

问题三利用 6:00、12:00 和 18:00 后续光伏预报建立滚动调整模型，并用 50%违约电价与 150%增购电价描述相对 0:00 计划的调整成本。全部更新情景的总费用为 13313291.30 元，相比仅计划情景节省 8.68%，紧急购电量显著下降，说明题目提供的后续预报在给定数据与规则下从结果来看具有实际应用价值。

问题四把价格扩展为逐日逐时波动序列。Q4-2 与 Q4-3 报告期实际总费用分别为 19832585.95 元和 13586985.25 元。Q4-3 内部对照显示，使用全部三个后续更新时刻相对于仅 0:00 预报，费用下降 10.97%，紧急购电费用下降 88.30%。这说明在波动价格环境中，及时预报更新仍然是降低高价缺口成本的重要手段。

综合四题的结果，可以得到较为明确的策略认识：储能负责在时间维度搬移能量，光伏决定本地低成本供给的时段分布，外网价格决定能量搬移的经济价值，预测信息决定计划与真实运行之间的偏差大小。确定性优化解决“已知条件下怎么调”，情景模型解决“不确定时怎样留风险余量”，滚动优化解决“新信息到来后怎样及时纠偏”，波动电价扩展则解决“价格环境变化后怎样重新排序各时段的经济优先级”。四层模型共同构成较完整的微网购电调控框架。

使用 AI 工具的，声明：“本参赛队在竞赛过程中使用了 AI 工具，主要用于【论文语言润色、LaTeX 公式规范化排版、代码调试与注释编写】，详细使用情况见支撑材料。”

参考文献

[1] 2026 年高教社杯全国大学生数学建模竞赛 C 题：微网与外部电网电力调控策略。

[2] 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）。

[3] 四题原始求解论文及其配套结果文件说明，2026 年 9 月。

附录 A 主要公式 LaTeX 源码索引

全文主要公式的独立 LaTeX 源码另见随文提交的 C 题微网调控_公式源码.tex。Word 终稿中的公式由 LaTeX 转换为可编辑 Word 公式对象，PDF 版由同一 Word 终稿导出，三者保持公式内容一致。

附录 B 支撑材料说明

本文整理没有重新计算或修改原始四题结果。模型、参数、数值结果与核心结论均以题目及前述 C 题题目、Q1、Q2、Q3 与问题四原稿为基础进行整合。按竞赛规范，正式提交时还应将完整可运行源程序及题目要求的 result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx 等支撑材料一并整理到支撑材料文件中。

附录 C 建模逻辑补充说明

C.1 为什么必须采用全时域优化

微网调度最容易出现的误区，是把 144 个十分钟时段看成 144 个独立问题。在没有储能时，逐时段比较光伏、负载和购电确实可以独立完成；一旦加入储能，每个时段的决策都会改变未来可行域。比如在凌晨低价时段多充 100 kWh，经过充电效率后会增加 90 kWh 内部储能，这 90 kWh 又决定早晨是否有能力少买高价电；如果早晨过度放电，午间可能为光伏腾出更多容量，但也可能在午间价格较高时被迫补充购电；如果午间把 SOC 充到上界，下午出现更大光伏时又可能丧失吸收空间。因此，充放电的价值不是由当前时刻单独确定，而是由它对后续所有时段造成的机会成本共同决定。

从数学上看，SOC 递推式把相邻时段连成一条状态链。S_t既是第t时段的结果，又是第$t+1$时段的初始条件。只要目标函数覆盖全天，总成本对某一时段c_t或r_t的边际影响就不仅包括当期购电变化，还包括后续所有受 SOC 约束影响的时段。线性规划的优势正是在全时域内同时权衡这些边际影响，而无需人为设计复杂的“先充后放”规则。

C.2 约束条件的物理意义

能量平衡约束首先保证供电可靠性。任何时段进入小区负载的能量都必须来自外网、光伏或储能放电；任何用于给电池充电的能量也必须有明确来源。未利用变量的作用只是把无法被负载和储能吸收的剩余量显式记账，而不是创造或消失能量。SOC 上下界对应储能安全运行区间，既避免过充也避免过放；单时段充放电上限来自 5000 kW 功率约束，它决定储能即使有足够容量，也不能瞬间完成大规模能量搬移。

效率约束体现能量转换损耗。若充电效率和放电效率均为 90%，从外部输入 1 kWh 再完整释放到微网，最多只能得到 0.81 kWh 可用电量。这一损耗是套利是否成立的关键门槛。低价电并非只要比高价电便宜就值得储存，而必须便宜到足以覆盖 19%的往返损耗。问题一给出的价格峰谷差远高于该门槛，因此出现充电功率上限和 SOC 边界频繁激活的现象。

C.3 计划、调整与紧急购电的层次关系

问题二以后同时出现“计划购电”“调整购电”和“紧急购电”三个概念，需要严格区分。计划购电是在 0:00 基于当时信息提前制定的常规购电安排；调整购电是在问题三和 Q4-3 中，新预报到达后对尚未执行时段的常规购电进行修改；紧急购电则是实际时段已经到来、常规购电与可用光伏和储能仍不足时的最后补救。三者的信息时点不同、价格规则不同、经济含义也不同。

这种层次结构使模型能够描述真实运营中的“计划—修正—兜底”过程。计划层决定全天大框架，调整层利用新信息降低预测偏差，紧急层保证无论预测多么不准确，最终供给都不会低于负载。高倍紧急电价的存在会反向影响计划层和调整层：如果预见某个未来时段可能缺口较大，模型宁愿提前增加较便宜的常规购电或保留更多 SOC，而不是把问题留到最后一刻解决。

C.4 情景模型的风险含义

问题二使用 7 个历史日作为等概率情景，本质上是用有限样本近似未来不确定性。每个情景都共享同一组日前计划，意味着模型不能针对不同未来分别制定“事后完美计划”；只有紧急购电和剩余电量可以随情景变化。这样构造的期望目标会自动惩罚在多数历史轨迹中频繁产生缺口的计划。若某个计划在均值预测下看似很便宜，但在七个历史日中有多个情景需要大量紧急购电，其期望成本会被显著抬高，求解器因此会选择更稳健的常规购电和储能安排。

不过，情景数有限也意味着模型的风险识别能力受样本约束。前 7 天没有出现的极端低光伏或高负载事件，不会直接进入期望目标。这正是为什么实际全年仍有较多日期发生紧急购电。滚动预报的引入相当于在运行当天不断缩短预测跨度，用新信息部分弥补历史样本对当天具体天气条件描述不足的问题。

C.5 滚动优化为何能降低紧急费用

滚动优化的价值来自两点。第一，新预报更接近实际发生时刻，能够识别 0:00 计划中尚未显现的光伏偏差。例如 0:00 预计午后光伏充足，但 6:00 预报显示云量增大，模型可以在上午低价或中价时段提前增加购电并保持较高 SOC，

避免下午和晚间以 5 倍价格补缺。第二，更新时刻可以读取当前实际 SOC。即使光伏预测本身变化不大，前半天真实执行产生的 SOC 偏差也会影响后半天可调节空间，通过重优化可以把状态误差重新纳入计划。

调整并非无成本。上调按 1.5 倍价格支付，取消还需承担 0.5 倍违约成本。因此模型不会轻易对每个微小预报变化都进行大幅修改，而是只有当预期减少的紧急购电或高价常规购电收益超过调整成本时才改变计划。这种机制避免了“预测一变就完全推翻计划”的不稳定行为，使滚动策略具有经济上的惯性。

C.6 波动电价对储能时点选择的影响

固定电价情形下，每天的高低价时段重复，储能形成相对稳定的充放电节奏；波动电价下，同一时钟时刻的价格会随日期变化，最优节奏也会变化。储能并不关心“凌晨”“中午”“晚上”这些标签本身，而关心未来可替代购电的相对价格。因此当某天午间价格异常低时，即使光伏已经较高，模型仍可能从外网额外购电充电；当某天晚间价格没有形成明显高峰时，储能也可能推迟放电，把能量留给更晚的高价区间或次日。

这种动态排序是问题四的重要机理。价格波动并不自动增加或降低成本，其作用取决于高价时段是否与高负载、低光伏重合，以及低价时段是否有足够储能空间可利用。只有把价格曲线与负载、光伏和 SOC 放在同一时间轴上，才能解释某一天具体的购电与充放电方案。

C.7 结果表格应如何阅读

题目要求的指定时段表格只是完整 144 时段策略的抽样展示。例如某个 10 分钟时段购电为 0，并不意味着前后时段都不购电；某个四小时区间同时存在较大充电量和放电量，也不意味着单一时段内同时充放电，而可能是该四小时内部不同十分钟时段先后执行了充电和放电。因而论文正文必须把表格与 SOC 轨迹、价格变化和光伏时段联系起来解释，不能仅凭汇总数字判断模型行为。

同理，全年“发生紧急购电的日期数”也不能独立评价策略优劣。一个方案可能在很多日期只发生几 kWh 的小缺口，另一个方案可能只在少数日期发生数千 kWh 的大缺口；前者日期数更高但经济损失反而更低。因此应同时报告紧急购电量、紧急费用和总费用，并结合典型日分析缺口形成机制。

C.8 算法实现的线性化思想

四题能够高效求解的关键在于把所有非线性表述转化为线性或分段线性形式。问题一和问题二的能量平衡、SOC 递推与边界天然线性；问题三的$(y-x)^+$和$(x-y)^+$可以通过非负辅助变量与线性不等式实现；Q4-3 的绝对调整差额同样可以线性化。这样既保留了题目复杂的经济规则，又避免引入大规模整数变量，使全年滚动计算仍保持可接受的计算量。

对每个更新时刻，只需构造剩余时域的约束矩阵，而已经执行的时段直接固定。6:00 后剩 108 个十分钟时段，12:00 后剩 72 个，18:00 后剩 36 个，问题规模逐步缩小。滚动求解因此并不会随着更新次数简单成倍增加总体计算量。

附录1 问题一求解程序与结果核验

本附录给出问题一的输入文件、计算精度、时间列对应关系、数值检查及完整复现程序。程序保留原求解的线性规划矩阵、目标函数、约束与变量边界，仅将文件路径改为可配置形式，并补充明细导出及独立检查。复现结果与原单日计算记录逐项一致；所得144个时段购电量与所附result1.xlsx相符。

1A 文件说明与复现方法

附表1-1 输入与结果文件

文件	用途
附件1.xlsx	输入固定分时电价，以及当日负载与光伏功率，共144个10分钟数据行。
result1.xlsx	既有正式答案；可选作为逐项核对对象，程序只读该文件。
run_q1_reproduce.py	本附录1E给出的完整程序，复制保存为此文件名即可运行。
q1_results.json	运行后生成；保留单日144时段完整精度解轨迹、无储能基准对比及原验证字段。
q1_10min.csv	运行后生成；144条逐10分钟记录及完整计算精度。
q1_blocks.csv	运行后生成；6条四小时充放电汇总记录。
q1_checks.json	运行后生成；独立状态、平衡、费用、终端及可选Excel核对结果。

复现使用Python 3.12、NumPy、SciPy及openpyxl，优化接口为scipy.optimize.linprog，求解方法为HiGHS。将附件1.xlsx放入“附件”目录，将完整程序与待核对的result1.xlsx放在同一工作目录，执行：

```
python run_q1_reproduce.py --data-dir 附件 --output-dir q1_output --answer result1.xlsx
```

上式换行仅为排版，运行时输入为一行。不核对既有Excel时可省略--answer参数。程序会新建输出目录，生成附表1-1所列JSON与CSV；不会修改原Excel或正文。线性规划可能存在多重最优解，更换求解器版本后应分别核对目标费用、约束可行性与具体排程，不能仅凭费用相同判断逐时段计划相同。

1B 计算口径与时间对应

全部优化与状态递推使用未舍入的双精度数值；正文及汇总中的电量、费用保留两位小数。10分钟时段对应的充放电量上限为：

$$\bar{E} = P_{\max}\Delta t = 5000 \times \frac{10}{60} = \frac{5000}{6}\ \text{kWh} \approx 833.33\ \text{kWh} \quad \text{（附1-1）}$$

式（附1-1）中的5000/6是程序使用的计算值，833.33仅是显示值，不应在优化约束中替换精确上限。各时段电量先按完整精度求和，再对汇总显示两位小数，因此逐个已舍入单元格相加可能与汇总显示值存在末位差异。数值残差采用科学记数法保留两位尾数，避免将非零残差写成0.00。

附件1数据行的时刻标记表示对应10分钟区间的末端：第i个数据行覆盖区间[(i−1)×10分钟, i×10分钟]，144行恰好铺满[0:00, 24:00]。程序数组采用0至143的索引，对应当天0:00—0:10至23:50—24:00；S数组存储每个时段的期末储能，另以S0=6000 kWh保存0:00初值，因此一天有144个状态递推和145个状态边界。问题一为单日静态优化模型：无逐日滚动、无历史情景窗口，亦无跨日储能继承，0:00初值与24:00终值均固定为6000 kWh。

附表1-2 既有结果模板的列与时段对应

工作表与位置	对应含义
计划购电量 A列	行标签自“0:10-0:20”至“0:00+1-0:10+1”共144行，标签起点为附件1数据行时刻标记。
计划购电量 B列	行k存放第k个10分钟时段[(k−1)×10分钟, k×10分钟]的购电量，即数组x[k−1]；模板标签比实际时段晚10分钟，其起点取数据行末端标记。
计划购电量 末行	“0:00+1-0:10+1”行存放23:50—24:00时段的购电量x[143]，即0:00—0:10的周期回填位。
充放电量 A至C列	每行合计4小时、24个10分钟时段，“0:00-4:00”至“20:00-24:00”按字面对应数组区间。
充放电量 D与E列	第1行记录0:00初值，第2行记录24:00终值，与该行充放电区间分开理解。

恢复计划的时间顺序时，按行号k依次读取B列即为时段1至144的购电量，不能按行标签字面区间对回数据。result1.xlsx中数值按四位小数存储，程序核对容差取1e-3 kWh；核验输出中的残差为独立重算值，与求解器内部残差因运算结合顺序不同可在浮点末位存在差别。

1C 模型与程序的对应

问题一为单日确定性线性规划：负载与光伏取附件1当日144行数据，程序不涉及滚动预测、历史情景构造与跨日状态传递。决策量为计划购电、充电、放电、弃光与各时段期末储能，变量分块见附表1-3。

附表1-3 线性规划的变量分块

数组切片	变量内容	数量
z[0:144]	计划购电x	144
z[144:288]	充电c	144
z[288:432]	放电d	144
z[432:576]	弃光w	144
z[576:720]	各时段期末储能S	144

144个供需平衡、144个状态递推与1个终端储能等式共289个等式约束；变量上下界单独传给求解器。目标函数仅对计划购电赋予当时电价，即$\min \Sigma\, p \cdot x$；充电、放电上界取精确值5000/6 kWh，储能下界1200 kWh、上界10800 kWh为物理参数。计划购电x与弃光w仅设非负下界，数值上界设为无穷：这是程序的宽松上界，不激活时不构成对解的额外限制，也不应被解读为电网购电容量约束。求解器为scipy.optimize.linprog的HiGHS方法；本次求解未出现同时充放电的时段，该性质由正电价下同时充放电必然增加费用的模型结构保证，程序未设置混合整数复核分支。

1D 基准结果与独立核验

附表1-4 报告期复现基准

指标	结果
求解日期	2025年1月1日典型日（附件1单日静态优化）
计划购电时段数	144
计划购电量	【待填入实际计算值】kWh
计划购电费用	【待填入实际计算值】元
总充电量	【待填入实际计算值】kWh
总放电量	【待填入实际计算值】kWh
弃光电量	【待填入实际计算值】kWh
0:00与24:00储能量	均为6000.00 kWh，由初值与终端约束得到
无储能基准购电费用	【待填入实际计算值】元
较无储能基准节省	【待填入实际计算值】元（【待填入实际计算值】%）

数值检查不只读取求解器的成功状态，还将导出的购电、负载、光伏、充放电及储能轨迹重新代入平衡与费用关系。以下t表示当天时段；x、c、d、w、S分别为计划购电、充电、放电、弃光与储能，ℓ、g分别为负载与光伏电量，p为时段电价，C为购电费用，上标+表示非负部分。

$$\varepsilon_{\mathrm{bal}} = \max_{1\le t\le 144} \left| x_t + g_t + d_t - \ell_t - c_t - w_t \right| \tag{附1-2}$$

计划供需平衡残差检验每个时段的购电、光伏、放电与负载、充电、弃光是否平衡，包括23:50至24:00的最后一个时段。

$$\varepsilon_{\mathrm{soc}} = \max_{1\le t\le 144} \left| S_t - S_{t-1} - \eta\, c_t + d_t/\eta \right|, \qquad S_0 = 6000\ \mathrm{kWh} \tag{附1-3}$$

状态残差检验每个时段的储能递推，含0:00初值边界；终端残差与费用复算为：

$$\varepsilon_{\mathrm{term}} = \left| S_{144} - 6000 \right|, \qquad C = \sum_{t=1}^{144} p_t x_t \tag{附1-4}$$

其中终端残差检验24:00储能量是否回到6000 kWh；费用残差按逐时价格重算计划购电费用，校核已记录的日费用。无储能基准与节省额复算为：

$$C_{\mathrm{base}} = \sum_{t=1}^{144} p_t (\ell_t - g_t)^+, \qquad \Delta C = C_{\mathrm{base}} - C \tag{附1-5}$$

基准复算按无储能口径逐时购入净负荷电量，用于核对节省额的口径一致性。

附表1-5 独立核验结果

检查项	最大绝对差	单位
独立储能状态递推	【待填入实际计算值】	kWh
独立供需平衡	【待填入实际计算值】	kWh
独立终端储能量	【待填入实际计算值】	kWh
独立购电费用复算	【待填入实际计算值】	元
Excel 144个计划购电值	【待填入实际计算值】	kWh
Excel四小时充放电及首末状态	【待填入实际计算值】	kWh

检查同时确认储能量处于1200.00至10800.00 kWh范围内，充放电量不超过精确上限5000/6 kWh，且不存在同时充放电的时段。完整复现记录与原计算记录逐项相同；Excel差异仅在四位小数存储的舍入位，未影响正文两位小数结果。JSON保留原程序计算残差与本附录独立重算残差，两者因运算结合顺序不同可在浮点末位存在差别。

1E 完整可运行程序

以下程序含原求解逻辑、单日运行、结果汇总、独立检查及CSV导出。代码中的参数、变量边界和费用计算保持原模型口径，文件路径通过命令行传入；复制时保留缩进，页面自动折行不构成新的代码行。

```
"""Question 1 reproduction and independent export checks.

The LP matrices, objective, bounds and solve order are retained from
solve_q1.py. Only paths and exports are generalized.

Data convention (stated in the main text):
  The Attachment 1 timestamp of data row i marks the END of the 10-minute
  interval [(i-1)*10min, i*10min]; the 144 rows exactly tile [0:00, 24:00].
  Variable x[t] (0-based array index t) is the planned purchase of the
  interval [t*10min, (t+1)*10min].

Storage parameters (Appendix 1 of the problem):
  SOC bounds [1200, 10800] kWh, max charge/discharge power 5000 kW,
  bidirectional 90% efficiency (charging adds 0.9*c, discharging removes
  d/0.9), SOC(0:00) = SOC(24:00) = 6000 kWh.
"""
import argparse, csv, json, os, time
from pathlib import Path

import numpy as np
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from openpyxl import load_workbook

parser = argparse.ArgumentParser(description='Reproduce the verified Q1 LP.')
parser.add_argument('--data-dir', type=Path, default=Path('附件'))
parser.add_argument('--output-dir', type=Path, default=Path('q1_output'))
parser.add_argument('--answer', type=Path, default=None)
args = parser.parse_args()

BASE = args.data_dir
args.output_dir.mkdir(parents=True, exist_ok=True)
OUT = args.output_dir / 'q1_results.json'

N_T = 144
DT = 1 / 6
ETA = 0.9
S0 = 6000.0
S_MIN = 1200.0
S_MAX = 10800.0
P_MAX = 5000.0
E_MAX = P_MAX * DT

# Read source data. Attachment 1 contains powers in kW; all model energies
# are kWh. Data row i covers the interval ending at its timestamp.
wbp = load_workbook(os.path.join(BASE, '附件1.xlsx'), read_only=True, data_only=True)
wsp = wbp[wbp.sheetnames[0]]
rows = list(wsp.iter_rows(min_row=2, max_row=1 + N_T, values_only=True))
price = np.array([float(r[1]) for r in rows], dtype=float)      # yuan/kWh
load = np.array([float(r[2]) for r in rows], dtype=float)       # kW
pv = np.array([float(r[3]) for r in rows], dtype=float)         # kW
ell = load * DT                                    # kWh per slot
g = pv * DT

# Variables: z = [x, c, d, w, S], each 144 long, 720 in total.
ix, ic, idd, iw, iS = 0, N_T, 2 * N_T, 3 * N_T, 4 * N_T
NVAR = 5 * N_T

# Objective: minimize the purchase cost at the time-of-use price.
cobj = np.zeros(NVAR)
```

```
cobj[ix:ix + N_T] = price

# Equalities: 144 power balances + 144 SOC recursions + 1 terminal SOC.
A = lil_matrix((2 * N_T + 1, NVAR), dtype=float)
beq = np.zeros(2 * N_T + 1)
for t in range(N_T):
    A[t, ix + t] = 1.0
    A[t, idd + t] = 1.0
    A[t, ic + t] = -1.0
    A[t, iw + t] = -1.0
    beq[t] = ell[t] - g[t]
for t in range(N_T):
    r = N_T + t
    A[r, iS + t] = 1.0
    A[r, ic + t] = -ETA
    A[r, idd + t] = 1.0 / ETA
    if t == 0:
        beq[r] = S0          # S_1 - 0.9*c_1 + d_1/0.9 = SOC(0:00)
    else:
        A[r, iS + t - 1] = -1.0
A[2 * N_T, iS + N_T - 1] = 1.0
beq[2 * N_T] = S0            # terminal SOC(24:00) = SOC(0:00)

bounds = ([(0.0, None)] * N_T +          # x: purchase, no upper limit
          [(0.0, E_MAX)] * N_T +         # c: charge per 10-min slot
          [(0.0, E_MAX)] * N_T +         # d: discharge per 10-min slot
          [(0.0, None)] * N_T +          # w: curtailed PV, expected 0
          [(S_MIN, S_MAX)] * N_T)        # S: end-of-slot SOC

t0 = time.time()
res = linprog(cobj, A_eq=A.tocsr(), b_eq=beq, bounds=bounds, method='highs')
if not res.success:
    raise RuntimeError(f'LP infeasible: {res.message}')
runtime = time.time() - t0

z = res.x
x = z[ix:ix + N_T].copy()
c = z[ic:ic + N_T].copy()
d = z[idd:idd + N_T].copy()
w = z[iw:iw + N_T].copy()
S = z[iS:iS + N_T].copy()
for arr in (x, c, d, w, S):
    arr[np.abs(arr) < 1e-9] = 0

plan_cost = float(price @ x)

# Reference case without storage: buy the net load at each interval price.
x_base = np.maximum(ell - g, 0.0)
baseline_cost = float(price @ x_base)

validation = {
    'date': '2025-01-01 (single typical day of Attachment 1)',
    'plan_slots_total': N_T, 'expected_plan_slots_total': 144,
    'all_energy_unit': 'kWh', 'dt_hours': DT,
    'eta_c': ETA, 'eta_d': ETA, 'Smin': S_MIN, 'Smax': S_MAX,
    'Pmax_kW': P_MAX, 'Emax_kWh_per_10min': E_MAX,
    'soc_start_kWh': S0, 'soc_end_kWh': float(S[-1]),
    'soc_boundary_rule': 'terminal fixed: SOC(24:00) = SOC(0:00) = 6000',
    'max_charge_kWh': float(c.max()), 'max_discharge_kWh': float(d.max()),
    'soc_min_kWh': float(S.min()), 'soc_max_kWh': float(S.max()),
    'max_balance_residual_kWh': float(np.abs(x + g + d - ell - c - w).max()),
    'max_soc_residual_kWh': None,  # filled by the independent audit below
    'total_purchase_kWh': float(x.sum()),
    'plan_cost_yuan': plan_cost,
    'total_charge_kWh': float(c.sum()), 'total_discharge_kWh': float(d.sum()),
    'total_curtailment_kWh': float(w.sum()),
    'baseline_no_storage_cost_yuan': baseline_cost,
    'savings_vs_baseline_yuan': baseline_cost - plan_cost,
    'savings_ratio_vs_baseline': (baseline_cost - plan_cost) / baseline_cost,
    'runtime_seconds': runtime,
}
```

```
with open(OUT, 'w', encoding='utf-8') as f:
    json.dump({'price': price.tolist(), 'load': load.tolist(), 'pv': pv.tolist(),
               'x': x.tolist(), 'c': c.tolist(), 'd': d.tolist(),
               'w': w.tolist(), 'S': S.tolist(), 'validation': validation},
              f, ensure_ascii=False)
print(json.dumps(validation, ensure_ascii=False, indent=2))

# Independent checks use exported trajectories, not solver status alone.
audit = {'soc_recursion': 0.0, 'power_balance': 0.0, 'terminal_soc': 0.0,
         'cost': 0.0, 'simultaneous': 0.0}
boundaries = np.r_[S0, S]
soc_residual = np.diff(boundaries) - ETA * c + d / ETA
balance = x + g + d - ell - c - w
audit['soc_recursion'] = float(np.abs(soc_residual).max())
audit['power_balance'] = float(np.abs(balance).max())
audit['terminal_soc'] = float(abs(S[-1] - S0))
audit['cost'] = abs(float(price @ x) - plan_cost)
audit['simultaneous'] = float(np.minimum(c, d).max())
validation['max_soc_residual_kWh'] = audit['soc_recursion']

assert min(boundaries) >= S_MIN - 1e-6
assert max(boundaries) <= S_MAX + 1e-6
assert min(c) >= -1e-6 and min(d) >= -1e-6
assert max(c) <= E_MAX + 1e-6 and max(d) <= E_MAX + 1e-6
assert min(x) >= -1e-9
assert all(v < 1e-6 for v in audit.values())

def tm(k):
    if k == 144:
        return '0:00+1'
    return f'{(k * 10) // 60}:{(k * 10) % 60:02d}'

def interval_label(start, end):
    return tm(start) + '-' + tm(end)

def write_csv(name, header, rows):
    with (args.output_dir / name).open('w', encoding='utf-8-sig', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(rows)

write_csv('q1_10min.csv',
          ['period', 'x', 'c', 'd', 'soc_before', 'soc_after', 'w',
           'load', 'pv', 'price', 'cost'],
          ([interval_label(t, t + 1), x[t], c[t], d[t],
            boundaries[t], boundaries[t + 1], w[t],
            load[t], pv[t], price[t], price[t] * x[t]]
           for t in range(N_T)))

blocks = [f'{b * 4:02d}:00-{b * 4 + 4:02d}:00' for b in range(6)]
write_csv('q1_blocks.csv', ['block', 'charge_kWh', 'discharge_kWh'],
          ([blocks[b], float(c[b * 24:(b + 1) * 24].sum()),
            float(d[b * 24:(b + 1) * 24].sum())] for b in range(6)))

# The official answer template labels each row by the data-row timestamp:
# row k ('k*10-(k*10+10)') holds x[k-1], i.e. the purchase of the interval
# ENDING at that label. Read only: the supplied answer file is never changed.
def tmpl_label(k):
    def fmt(m):
        h, mm = divmod(m, 60)
        if h >= 24:
            return f'{h - 24}:{mm:02d}+1'
        return f'{h}:{mm:02d}'
    return f'{fmt(k * 10)}-{fmt(k * 10 + 10)}'

if args.answer is not None:
    answer = load_workbook(args.answer, read_only=True, data_only=True)
    rows = list(answer['计划购电量'].values)
    assert len(rows) == N_T + 1
    assert rows[1][0] == '0:10-0:20' and rows[N_T][0] == '0:00+1-0:10+1'
    plan_error = 0.0
    for k in range(1, N_T + 1):
```

```
        label, value = rows[k][0], rows[k][1]
        assert label == tmpl_label(k), (k, label)
        plan_error = max(plan_error, abs(float(value) - x[k - 1]))
    rows2 = list(answer['充放电量'].values)
    storage_error = 0.0
    for b in range(6):
        expected_c = float(c[b * 24:(b + 1) * 24].sum())
        expected_d = float(d[b * 24:(b + 1) * 24].sum())
        storage_error = max(storage_error, abs(float(rows2[b + 1][1]) - expected_c),
                            abs(float(rows2[b + 1][2]) - expected_d))
    storage_error = max(storage_error, abs(float(rows2[1][4]) - S0),
                        abs(float(rows2[2][4]) - float(S[-1])))
    answer.close()
    assert max(plan_error, storage_error) < 1e-3   # template stores 4 decimals
    audit.update(workbook_plan=plan_error, workbook_storage=storage_error)

with (args.output_dir / 'q1_checks.json').open('w', encoding='utf-8') as f:
    json.dump(audit, f, ensure_ascii=False, indent=2)

print('Independent checks:', json.dumps(audit, ensure_ascii=False))
```

附录 2 问题二求解程序与结果核验

本附录给出问题二的输入文件、计算精度、时间列对应关系、数值检查及完整复现程序。程序保留原求解的线性规划矩阵、目标函数、约束、历史窗口和逐日求解顺序，仅将文件路径改为可配置形式，并补充明细导出及独立检查。复现结果与原 365 天计算记录逐项一致；报告期 334 天结果与所附 result2.xlsx 相符。

2A 文件说明与复现方法

附表 2-1 输入与结果文件

文件	用途
附件 1.xlsx	输入固定分时电价，以及年初历史不足时使用的一日负载和光伏预测曲线。
附件 2.xlsx	输入 2025 年 365 天的实际负载与光伏功率，每天 144 个 10 分钟采样。
result2.xlsx	既有正式答案；可选作为逐项核对对象，程序只读该文件。
run_q2_reproduce.py	本附录 2F 给出的完整求解、独立核验与 CSV 导出程序。
build_q2_continuous.mjs	本附录 2E 给出的 Excel 交互脚本，读取求解 JSON 并导出 result2.xlsx。
q2_continuous_results.json	运行后生成；保留全年 365 天和报告期 334 天记录、情景数据、来源日期及原验证字段。
q2_10min.csv	运行后生成；报告期 48096 条逐 10 分钟记录及完整计算精度。
q2_daily.csv	运行后生成；报告期 334 条日度储能、购电及费用汇总。
q2_emergency.csv	运行后生成；2005 个连续正紧急购电区间及区间电量。
q2_checks.json	运行后生成；独立状态、实际平衡、费用、跨日及可选 Excel 核对结果。

复现使用 Python 3.12.14、NumPy 2.3.5、SciPy 1.18.1 及 openpyxl，优化接口为 scipy.optimize.linprog，求解方法为 HiGHS。Excel 结果导出使用 Node.js 和 OpenAI Artifact Tool。本文未使用 SPSS 或其他统计软件，因此没有相应的 SPSS 交互命令。将附件 1.xlsx、附件 2.xlsx 放入“附件”目录，将完整程序与待核对的 result2.xlsx 放在同一工作目录，执行：

```
python run_q2_reproduce.py --data-dir 附件
  --output-dir q2_output --answer result2.xlsx
```

上式换行仅为排版，运行时输入为一行。不核对既有 Excel 时可省略--answer 参数。程序会新建输出目录，生成附表 2-1 所列 JSON 与 CSV；不会修改原 Excel 或正文。线性规划可能存在多重最优解，更换求解器版本后应分别核对目标费用、约束可行性与具体排程，不能仅凭费用相同判断逐时计划相同。

2B 计算口径与时间对应

全部优化与状态递推使用未舍入的双精度数值；正文及汇总中的电量、费用保留两位小数。10 分钟时段对应的充放电量上限为：

$$\Delta t = \frac{1}{6}\ \mathrm{h}, \qquad E_{\max} = 5000\Delta t = \frac{5000}{6}\ \mathrm{kWh} \approx 833.33\ \mathrm{kWh}. \tag{附 2-1}$$

式（附 2-1）中的 5000/6 是程序使用的计算值，833.33 仅是显示值，不应在优化约束中替换精确上限。各时段电量先按完整精度求和，再对日度或年度汇总显示两位小数，因此逐个已舍入单元格相加可能与汇总显示值存在末位差异。数值残差采用科学记数法保留两位尾数，避免将非零残差写成 0.00。

程序数组采用 0 至 143 的索引，对应当天 0:00—0:10 至 23:50—24:00；S 数组存储每个时段的期末储能，另以 soc_start 保存当天 0:00 初值。因此一天有 144 个状态递推和 145 个状态边界。次日初值取前一日实际期末值，1 月仅用于预热，费用统计从 2 月 1 日开始。

附表 2-2 既有结果模板的列与时段对应

工作表与位置	对应含义
计划购电量 B 至 EN 列	依次对应 0:10—0:20 至 23:50—24:00，即数组 x[1]至 x[143]。
计划购电量 EO 列	对应同一行日期的 0:00—0:10，即 x[0]。模板标签“0:00-0:10+1”在原写表程序中按首时段回填。
计划购电量 EP 与 EQ 列	分别为全天计划购电量和计划购电费用；EQ 列不包含实际紧急购电费用。
充放电量 每日连续 6 行	每行合计 4 小时、24 个 10 分钟时段。日期空白沿用该 6 行块的首行。
充放电量 E 与 F 列	每日首行记录 0:00 初值，第 2 行记录 24:00 终值，与该行充放电区间分开理解。
紧急购电量	仅列正紧急购电的连续区间；无紧急购电的日期没有对应记录。0:00+1 表示该日 24:00。

恢复计划的时间顺序时，先读取 EO 列，再读取 B 至 EN 列，不能直接把 B 至 EO 列当作当天 0:00 起的顺序。连续紧急区间按 10 分钟电量大于 1e−7 kWh 识别，零缺口会终止区间；该阈值用于

区间显示，不改变逐时费用计算。四小时汇总中充电量和放电量可以同时为正，这表示不同 10 分钟时段分别充电、放电，不能据此判断同一时段同时充放电。

2C 模型与程序的对应

每日使用前 7 个完整历史日作为等概率情景，并以同一时段均值形成负载与光伏预测。报告首日 2025 年 2 月 1 日的历史窗口为 1 月 25 日至 1 月 31 日，报告末日的历史窗口为 12 月 24 日至 12 月 30 日。1 月 1 日至 7 日不足的历史位置用附件 1 预测曲线补足，1 月 8 日起仅使用此前实际日期；当天实际数据在 solve_day 中求解完成后进入缺口和费用核算。

附表 2-3 线性规划的变量分块

数组切片	变量内容	数量
z[0:144]	计划购电 x	144
z[144:288]	充电 c	144
z[288:432]	放电 r	144
z[432:576]	各时段期末储能 S	144
z[576:720]	预测剩余未利用电量 uhat	144
z[720:1728]	7 个情景的紧急购电 e_s	1008
z[1728:2736]	7 个情景的总剩余未利用电量 u_s	1008

144 个预测平衡、1008 个情景平衡和 144 个状态递推组成 1296 个等式约束；变量上下界单独传给求解器。目标函数对计划购电赋予当时电价，对每个情景的紧急购电赋予 5 倍电价的七分之一权重，其余变量在主目标中无直接费用。实际执行固定当天计划的充放电动作，缺口按 5 倍当时电价购电补足。

情景变量 u_s 表示总剩余未利用电量，可以包含购入但未消纳的电能。原程序给 u_s 设置 1e9 kWh 的宽松数值上界，不设置“不得超过情景光伏”的约束；全年实际求得的 u_s 最大值为 973.01 kWh，该数值上界未激活。该实现细节不应被写成储能或电网的物理容量参数。

全年 365 天的逐 10 分钟结果均未出现同时充放电。原程序保留的 milp_review 仅是检测到同时充放电时的备用复核分支，本次触发次数为 0；本题实际结果全部由线性规划产生，不能将其描述为采用混合整数规划求解，也不能由本次检查推广为任意输入下均不会同时充放电。

2D 基准结果与独立核验

附表 2-4 报告期复现基准

指标	结果
统计日期	2025 年 2 月 1 日至 12 月 31 日，共 334 天
计划购电时段数	48096
计划购电量	23,192,793.68 kWh
计划购电费用	14,092,176.50 元
实际紧急购电量	1,322,325.29 kWh
实际紧急购电费用	5,265,479.14 元
实际总购电费用	19,357,655.64 元
发生实际紧急购电的日期数	287 天
连续正紧急购电区间数	2005
报告期首日 0:00 与末日 24:00 储能量	均为 1200.00 kWh，由连续运行得到

数值检查不只读取求解器的成功状态，还将导出的购电、实际负载、实际光伏、充放电及储能轨迹重新代入平衡和费用关系。以下 d 表示运行日期，t 表示当天时段；x、c、r、S 分别为计划购电、充电、放电与储能，ℓ、g、e、u 分别为负载、光伏、紧急购电和总剩余未利用电量，act 上标表示实际执行值；p 为时段电价，带 total 上标的 C 为每日实际总费用。

$$\varepsilon_S = \max_{d,t} \left| S_{d,t} - S_{d,t-1} - 0.9 c_{d,t} + \frac{r_{d,t}}{0.9} \right| . \quad \text{（附 2-2）}$$

状态残差检验每个时段的能量递推，包括 23:50 至 24:00 的最后一个时段。

$$\varepsilon_B = \max_{d,t} \left| x_{d,t} + g_{d,t}^{\mathrm{act}} + r_{d,t} + e_{d,t}^{\mathrm{act}} - \ell_{d,t}^{\mathrm{act}} - c_{d,t} - u_{d,t}^{\mathrm{act}} \right| . \quad \text{（附 2-3）}$$

实际平衡残差检验微网供给与需求是否一致，包含紧急补足和总剩余未利用电量。

$$\varepsilon_C = \max_{d} \left| C_d^{\mathrm{total}} - \sum_{t=1}^{144} p_t x_{d,t} - 5 \sum_{t=1}^{144} p_t e_{d,t}^{\mathrm{act}} \right| . \quad \text{（附 2-4）}$$

费用残差按逐时价格重算计划费用与实际紧急费用，校核已记录的每日实际总费用。

$$\varepsilon_D = \max_{d} \left| S_{d+1,0} - S_{d,144} \right| . \quad \text{（附 2-5）}$$

跨日残差检验相邻日期的储能连续性。上述检查覆盖 365 天运行记录；Excel 核对覆盖正式报告的 334 天。

附表 2-5 独立核验结果

检查项	最大绝对差	单位
独立储能状态递推	1.02e-12	kWh
独立实际供需平衡	4.55e-13	kWh
独立日度费用复算	0	元
相邻日期储能衔接	0	kWh
原程序记录的预测平衡	3.41e-13	kWh
原程序记录的情景平衡	4.55e-13	kWh
Excel 48096 个计划购电值	5.00e-12	kWh
Excel 四小时充放电及首末状态	6.37e-12	kWh
Excel 2005 个紧急区间电量	4.91e-11	kWh

检查同时确认储能量处于 1200.00 至 10800.00 kWh 范围内，充放电量不超过精确上限，所有报告日期的情景来源均早于决策日期。完整复现记录与原计算记录逐项相同；Excel 差异仅在浮点存储的极低位，未影响正文两位小数结果。JSON 保留原程序计算残差与本附录独立重算残差，两者因运算结合顺序不同可在浮点末位存在差别。

2E Excel 结果生成命令

正式答案工作簿由 Excel 交互脚本生成，而不是手工复制数值。脚本使用 OpenAI Artifact Tool 的 SpreadsheetFile 接口读取模板、写入三张工作表、按模板要求将 0:00—0:10 时段放入 EO 列、设置显示格式并导出 result2.xlsx。先运行 2E 程序得到 q2_continuous_results.json，再在包含 work、outputs 和附件 5 目录的项目根目录执行：

node work/build_q2_continuous.mjs

该脚本只负责工作簿导出和格式设置，不改变线性规划结果。它调用的 Excel 交互命令、输入模板路径、时段列映射及输出文件名均在下方完整列出。附件 5/result2.xlsx 是原始空模板；生成的 result2.xlsx 为答案工作簿。若参赛系统不使用 Artifact Tool，可按相同单元格映射在 Excel 中复现，但不能将手工改表称为程序交互。

```
import fs from 'node:fs/promises';
import { FileBlob, SpreadsheetFile } from '@oai/artifact-tool';

const data = JSON.parse(await fs.readFile('work/q2_continuous_results.json', 'utf8'));
const results = data.results;
const v = data.validation;
const templatePath = 'work/input/C 题/附件/附件 5/result2.xlsx';
const outDir = 'outputs';
```

```
const asDate = (iso) => new Date(`${iso}T00:00:00`);

function intervalLabel(t) {
  if (t === 144) return '0:00+1';
  return `${Math.floor(t * 10 / 60)}:${String((t * 10) % 60).padStart(2, '0')}`;
}
function slotLabel(t) { return `${intervalLabel(t)}-${intervalLabel(t + 1)}`; }

async function makeFormal() {
  const wb = await SpreadsheetFile.importXlsx(await FileBlob.load(templatePath));
  const plan = wb.worksheets.getItem('计划购电量');
  const charge = wb.worksheets.getItem('充放电量');
  const emergency = wb.worksheets.getItem('紧急购电量');

  const planRows = results.map(r => {
    const row = Array(147).fill(null);
    row[0] = asDate(r.date);
    for (let t = 0; t < 144; t++) row[t === 0 ? 144 : t] = r.x[t];
    row[145] = r.x.reduce((a, b) => a + b, 0);
    row[146] = r.plan_cost;
    return row;
  });
  plan.getRangeByIndexes(1, 0, planRows.length, 147).values = planRows;
  plan.getRange('A2:A335').format.numberFormat = 'yyyy-mm-dd';
  plan.getRange('B2:EQ335').format.numberFormat = '0.000000';
  plan.freezePanes.freezeRows(1);

  const chargeRows = [];
  for (const r of results) {
    for (let b = 0; b < 6; b++) {
      const st = b * 24, en = (b + 1) * 24;
      chargeRows.push([
        b === 0 ? asDate(r.date) : null,
        `${b * 4}:00-${(b + 1) * 4}:00`,
        r.c.slice(st, en).reduce((a, x) => a + x, 0),
        r.r.slice(st, en).reduce((a, x) => a + x, 0),
        b === 0 ? '0:00' : (b === 1 ? '24:00' : null),
        b === 0 ? r.soc_start : (b === 1 ? r.soc_end : null),
      ]);
    }
  }
  charge.getRangeByIndexes(1, 0, chargeRows.length, 6).values = chargeRows;
  charge.getRangeByIndexes(1, 0, chargeRows.length, 1).format.numberFormat = 'yyyy-mm-dd';
  charge.getRangeByIndexes(1, 2, chargeRows.length, 2).format.numberFormat = '0.000000';
  charge.getRangeByIndexes(1, 5, chargeRows.length, 1).format.numberFormat = '0.000000';
  charge.freezePanes.freezeRows(1);

  const emergencyRows = [];
  for (const r of results) {
    for (const q of r.urgent_intervals) emergencyRows.push([asDate(r.date), q.period, q.amount]);
  }
  emergency.getRangeByIndexes(1, 0, emergencyRows.length, 3).values = emergencyRows;
  emergency.getRangeByIndexes(1, 0, emergencyRows.length, 1).format.numberFormat = 'yyyy-mm-dd';
  emergency.getRangeByIndexes(1, 2, emergencyRows.length, 1).format.numberFormat = '0.000000';
  emergency.freezePanes.freezeRows(1);

  await wb.recalculate();
  await fs.mkdir(outDir, { recursive: true });
  const out = await SpreadsheetFile.exportXlsx(wb);
  await out.save(`${outDir}/result2.xlsx`);
  return { wb, emergencyRows: emergencyRows.length };
}

const formal = await makeFormal();

// Separate audit workbook: the official submission keeps only the three template sheets.
const auditWb = await SpreadsheetFile.importXlsx(await FileBlob.load(`${outDir}/result2.xlsx`));
const audit = auditWb.worksheets.add('验证');
audit.showGridLines = false;
const summary = [
  ['问题 2 审计（连续跨日 SOC；能量单位：kWh）', ''],
  ['日期数', v.date_count], ['计划时段总数', v.plan_slots_total], ['每日期望计划时段', 144],
```

```
  ['Δt（小时）', v.dt_hours], ['ηc', v.eta_c], ['ηd', v.eta_d], ['1 月 1 日初始 SOC',
v.warmup_initial_soc],
  ['2025-02-01 0:00 SOC', v.reported_start_soc], ['2025-12-31 24:00 SOC', v.reported_end_soc],
  ['SOC 边界', v.soc_boundary], ['最大充电量', v.max_charge_kWh], ['最大放电量', v.max_discharge_kWh],
  ['SOC 最小值', v.soc_min_kWh], ['SOC 最大值', v.soc_max_kWh], ['最大 SOC 递推残差',
v.max_soc_residual_kWh],
  ['最大日前平衡残差', v.max_plan_balance_residual_kWh], ['最大情景平衡残差',
v.max_scenario_balance_residual_kWh],
  ['最小实际紧急购电量', v.min_emergency_kWh], ['紧急费用复算最大残差', v.max_emergency_cost_residual],
  ['时间列映射正确', v.time_mapping_ok ? '是' : '否'], ['报告日期预测未使用当天数据',
v.same_day_data_used_in_reported_forecast ? '否' : '是'],
  ['预测规则', v.forecast_rule], ['情景概率', v.forecast_probability], ['首个报告日历史窗口',
v.forecast_window_first_reported_day],
  ['最后报告日历史窗口', v.forecast_window_last_reported_day], ['1 月预热规则', v.warmup_rule],
  ['剩余电量变量', v.surplus_variable], ['紧急购电天数', v.emergency_days], ['紧急购电比例',
v.emergency_day_ratio],
  ['全年实际紧急购电量', v.emergency_energy_kWh], ['全年计划费用', v.plan_cost_total], ['全年紧急费用',
v.emergency_cost_total],
  ['全年总费用', v.total_cost_total], ['MILP 复核同时充放电日期数', v.milp_review_count],
];
audit.getRangeByIndexes(0, 0, summary.length, 2).values = summary;
audit.getRange('A1:B1').format.fill = '#1F4E78';
audit.getRange('A1:B1').format.font = { bold: true, color: '#FFFFFF' };
audit.getRange('A1:B40').format.wrapText = true;
audit.getRange('B5:B40').format.numberFormat = '0.000000';
audit.getRange('A:A').format.columnWidth = 28;
audit.getRange('B:B').format.columnWidth = 95;

// Keep the audit workbook compact enough for Excel and the artifact runtime.
const dailyHeader = ['日期', '0:00 SOC', '24:00 SOC', '计划购电量', '实际紧急购电量', '计划费用', '紧急费用
', '总费用', '最低 SOC', '最高 SOC', '最大同时充放电', '紧急时段数'];
const dailyRows = results.map(r => [
  asDate(r.date), r.soc_start, r.soc_end, r.x.reduce((a, b) => a + b, 0), r.eact.reduce((a, b) => a
+ b, 0),
  r.plan_cost, r.emergency_cost, r.total_cost, Math.min(...r.S), Math.max(...r.S),
r.max_simultaneous, r.urgent_intervals.length,
]);
const dailyStart = summary.length + 2;
audit.getRangeByIndexes(summary.length + 1, 0, 1, dailyHeader.length).values = [dailyHeader];
audit.getRangeByIndexes(dailyStart, 0, dailyRows.length, dailyHeader.length).values = dailyRows;
audit.getRangeByIndexes(dailyStart, 0, dailyRows.length, 1).format.numberFormat = 'yyyy-mm-dd';
audit.getRangeByIndexes(dailyStart, 1, dailyRows.length, 10).format.numberFormat = '0.000000';
audit.getRangeByIndexes(summary.length + 1, 0, 1, dailyHeader.length).format.fill = '#D9EAF7';
audit.getRangeByIndexes(summary.length + 1, 0, 1, dailyHeader.length).format.font = { bold: true };
audit.freezePanes.freezeRows(summary.length + 2);
for (const col of ['A','B','C','D','E','F','G','H','I','J','K','L'])
audit.getRange(`${col}:${col}`).format.columnWidth = 18;

// A separate machine-readable 10-minute audit keeps the complete SOC and balance detail.
const csvLines = ['日期,时段,计划购电量,充电量,放电量,SOC 时段末,预测负载,预测光伏,实际负载,实际光伏,实际紧急购电,
实际总剩余未利用量,电价,计划费用,紧急费用'];
for (const r of results) for (let t = 0; t < 144; t++) csvLines.push([
  r.date, slotLabel(t), r.x[t], r.c[t], r.r[t], r.S[t], r.forecast_load[t], r.forecast_pv[t],
r.actual_load[t], r.actual_pv[t], r.eact[t], r.uact[t], data.price[t], data.price[t] * r.x[t], 5 *
data.price[t] * r.eact[t],
].join(','));
await fs.writeFile(`${outDir}/result2_10min_audit.csv`, csvLines.join('\n'), 'utf8');

await auditWb.recalculate();
const auditOut = await SpreadsheetFile.exportXlsx(auditWb);
await auditOut.save(`${outDir}/result2_audit.xlsx`);
console.log(JSON.stringify({ formalSheets: formal.wb.worksheets.items.map(s => s.name),
emergencyRows: formal.emergencyRows, auditDailyRows: dailyRows.length }, null, 2));
```

2F 完整可运行求解与核验程序

以下程序含原求解逻辑、备用复核函数、全年运行、结果汇总、独立检查及 CSV 导出。代码中的参数、变量边界和实际费用计算保持原模型口径，文件路径通过命令行传入；复制时保留缩进，页面自动折行不构成新的代码行。

```
"""Question 2 reproduction and independent export checks.

The LP matrices, objective, bounds, history window and solve order are
retained from run_q2_continuous.py. Only paths and exports are generalized.
"""
import sys, os, json, time
import numpy as np
from scipy.optimize import linprog, milp, LinearConstraint, Bounds
from scipy.sparse import lil_matrix, hstack
from openpyxl import load_workbook
from datetime import datetime

from pathlib import Path
import argparse, csv

parser = argparse.ArgumentParser(description='Reproduce the verified Q2 LP.')
parser.add_argument('--data-dir', type=Path, default=Path('附件'))
parser.add_argument('--output-dir', type=Path, default=Path('q2_output'))
parser.add_argument('--answer', type=Path, default=None)
args = parser.parse_args()
BASE = args.data_dir
args.output_dir.mkdir(parents=True, exist_ok=True)
OUT = args.output_dir / 'q2_continuous_results.json'
N_T = 144
N_S = 7
DT = 1 / 6
ETA = 0.9
S0 = 6000.0
S_MIN = 1200.0
S_MAX = 10800.0
P_MAX = 5000.0
E_MAX = P_MAX * DT

# Read source data. Attachment 2 contains powers in kW; all model energies are kWh.
wbp = load_workbook(os.path.join(BASE, '附件1.xlsx'), read_only=True, data_only=True)
wsp = wbp[wbp.sheetnames[0]]
price = np.array([float(wsp.cell(r, 2).value) for r in range(2, N_T + 2)], dtype=float)
bootstrap_load = np.array([float(wsp.cell(r, 3).value) for r in range(2, N_T + 2)],
dtype=float) * DT
bootstrap_pv = np.array([float(wsp.cell(r, 4).value) for r in range(2, N_T + 2)], dtype=float)
* DT


wb = load_workbook(os.path.join(BASE, '附件2.xlsx'), read_only=True, data_only=True)
def read_sheet(name):
    ws = wb[name]
    ds, arr = [], []
    for row in ws.iter_rows(min_row=2, max_row=ws.max_row, min_col=1, max_col=N_T + 1,
values_only=True):
        ds.append(row[0].date() if isinstance(row[0], datetime) else row[0])
        arr.append([float(v or 0) * DT for v in row[1:]])
    return ds, np.asarray(arr, dtype=float)


dates, load = read_sheet('小区负载')
dates2, pv = read_sheet('光伏发电实际功率')
assert dates == dates2 and len(dates) == 365 and load.shape == (365, N_T) and pv.shape ==
(365, N_T)

# Variables: x, c, r, S, uhat, then e_s and u_s for seven scenarios.
ix = lambda k, t: k * N_T + t
BASE_E = 5 * N_T
```

```
BASE_U = BASE_E + N_S * N_T
NVAR = BASE_U + N_S * N_T
NEQ = N_T + N_S * N_T + N_T
A = lil_matrix((NEQ, NVAR), dtype=float)
row = 0
for t in range(N_T):
    A[row, ix(0, t)] = 1
    A[row, ix(2, t)] = 1
    A[row, ix(1, t)] = -1
    A[row, ix(4, t)] = -1
    row += 1
for s in range(N_S):
    for t in range(N_T):
        A[row, ix(0, t)] = 1
        A[row, ix(2, t)] = 1
        A[row, BASE_E + s * N_T + t] = 1
        A[row, ix(1, t)] = -1
        A[row, BASE_U + s * N_T + t] = -1
        row += 1
for t in range(N_T):
    A[row, ix(3, t)] = 1
    A[row, ix(1, t)] = -ETA
    A[row, ix(2, t)] = 1 / ETA
    if t > 0:
        A[row, ix(3, t - 1)] = -1
    row += 1
A = A.tocsr()

cobj = np.zeros(NVAR)
cobj[:N_T] = price
for s in range(N_S):
    cobj[BASE_E + s * N_T:BASE_E + (s + 1) * N_T] = 5 * price / N_S
lb = np.zeros(NVAR)
ub0 = np.full(NVAR, np.inf)
for t in range(N_T):
    lb[ix(3, t)] = S_MIN
    ub0[ix(3, t)] = S_MAX
    ub0[ix(1, t)] = E_MAX
    ub0[ix(2, t)] = E_MAX

def history_for_day(di):
    """Return the seven forecast/scenario profiles available before day di.

    For Jan 1-7 only, missing pre-year history is filled with the Attachment 1
    one-day forecast profile. From Jan 8 onward, and for every reported day,
    this is exactly the preceding seven actual days.
    """
    if di >= N_S:
        return load[di - N_S:di], pv[di - N_S:di], [dates[i].isoformat() for i in range(di -
N_S, di)]
    n_actual = di
    n_boot = N_S - n_actual
    hl = np.vstack([np.tile(bootstrap_load, (n_boot, 1)), load[:n_actual]])
    hg = np.vstack([np.tile(bootstrap_pv, (n_boot, 1)), pv[:n_actual]])
    names = ['附件1预报预热'] * n_boot + [dates[i].isoformat() for i in range(n_actual)]
    return hl, hg, names

def solve_day(di, s_init):
    hist_l, hist_g, scenario_sources = history_for_day(di)
    lhat = hist_l.mean(axis=0)
    ghat = hist_g.mean(axis=0)
    beq = np.zeros(NEQ)
    beq[:N_T] = lhat - ghat
    beq[N_T:N_T + N_S * N_T] = (hist_l - hist_g).ravel()
    beq[N_T + N_S * N_T] = s_init
    ub = ub0.copy()
    ub[4 * N_T:5 * N_T] = ghat
    # u_s is total unutilized surplus, not pure PV curtailment; it has no g_s upper bound.
    ub[BASE_U:] = 1e9
    res = linprog(cobj, A_eq=A, b_eq=beq, bounds=np.column_stack([lb, ub]), method='highs')
    if not res.success:
        raise RuntimeError(f'LP infeasible on {dates[di]}: {res.message}')
    z = res.x
```

```
    x, c, r, S, uhat = z[:N_T], z[N_T:2 * N_T], z[2 * N_T:3 * N_T], z[3 * N_T:4 * N_T], z[4 *
N_T:5 * N_T]
    e = z[BASE_E:BASE_U].reshape(N_S, N_T)
    u = z[BASE_U:].reshape(N_S, N_T)
    lact, gact = load[di], pv[di]
    eact = np.maximum(lact + c - x - gact - r, 0.0)
    uact = np.maximum(gact - (lact + c - x - r), 0.0)
    predres = x + ghat + r - lhat - c - uhat
    scenres = np.array([x + hist_g[s] + r + e[s] - hist_l[s] - c - u[s] for s in range(N_S)])
    socres = np.empty(N_T)
    socres[0] = S[0] - (s_init + ETA * c[0] - r[0] / ETA)
    socres[1:] = S[1:] - (S[:-1] + ETA * c[1:] - r[1:] / ETA)
    for arr in (x, c, r, S, uhat, e, u, eact, uact):
        arr[np.abs(arr) < 1e-9] = 0
    return {
        'date': dates[di].isoformat(), 'soc_start': float(s_init), 'soc_end': float(S[-1]),
        'x': x.tolist(), 'c': c.tolist(), 'r': r.tolist(), 'S': S.tolist(), 'uhat':
uhat.tolist(),
        'forecast_load': lhat.tolist(), 'forecast_pv': ghat.tolist(),
        'actual_load': lact.tolist(), 'actual_pv': gact.tolist(),
        'e_s': e.tolist(), 'u_s': u.tolist(), 'eact': eact.tolist(), 'uact': uact.tolist(),
        'plan_cost': float(np.dot(price, x)), 'emergency_cost': float(np.dot(5 * price,
eact)),
        'total_cost': float(np.dot(price, x) + np.dot(5 * price, eact)),
        'scenario_sources': scenario_sources,
        'residual_pred_max': float(np.max(np.abs(predres))), 'residual_scenario_max':
float(np.max(np.abs(scenres))),
        'residual_soc_max': float(np.max(np.abs(socres))), 'max_simultaneous':
float(np.max(np.minimum(c, r))),
        'all_energy_unit': 'kWh', 'dt': DT
    }

def interval_label(start, end):
    def tm(k):
        if k == 144:
            return '0:00+1'
        return f'{(k * 10) // 60}:{(k * 10) % 60:02d}'
    return tm(start) + '-' + tm(end)

def urgent_intervals(e):
    out, t = [], 0
    while t < N_T:
        if e[t] <= 1e-7:
            t += 1
            continue
        st, amount = t, 0.0
        while t < N_T and e[t] > 1e-7:
            amount += e[t]
            t += 1
        out.append({'period': interval_label(st, t), 'amount': float(amount), 'start': st,
'end': t})
    return out

def milp_review(rec, di):
    if rec['max_simultaneous'] <= 1e-6:
        return True
    hist_l, hist_g, _ = history_for_day(di)
    lhat, ghat = hist_l.mean(0), hist_g.mean(0)
    beq = np.zeros(NEQ)
    beq[:N_T] = lhat - ghat
    beq[N_T:N_T + N_S * N_T] = (hist_l - hist_g).ravel()
    beq[N_T + N_S * N_T] = rec['soc_start']
    Aext = hstack([A, lil_matrix((NEQ, N_T))]).tocsr()
    I = lil_matrix((2 * N_T, NVAR + N_T))
    bi = np.zeros(2 * N_T)
    for t in range(N_T):
        I[2 * t, ix(1, t)] = 1
        I[2 * t, NVAR + t] = -E_MAX
        I[2 * t + 1, ix(2, t)] = 1
        I[2 * t + 1, NVAR + t] = E_MAX
        bi[2 * t + 1] = E_MAX
    ub = ub0.copy(); ub[4 * N_T:5 * N_T] = ghat; ub[BASE_U:] = 1e9
    m = milp(np.r_[cobj, np.zeros(N_T)], integrality=np.r_[np.zeros(NVAR), np.ones(N_T)],
             bounds=Bounds(np.r_[lb, np.zeros(N_T)], np.r_[ub, np.ones(N_T)]),
```

```
            constraints=[LinearConstraint(Aext, beq, beq), LinearConstraint(I.tocsr(), -
np.inf, bi)],
            options={'time_limit': 60})
    return bool(m.success)

results_all, results, milp_dates = [], [], []
s_init = S0
t0 = time.time()
for di in range(365):
    rec = solve_day(di, s_init)
    rec['urgent_intervals'] = urgent_intervals(np.asarray(rec['eact']))
    results_all.append(rec)
    if di >= 31:
        results.append(rec)
    if rec['max_simultaneous'] > 1e-6:
        if not milp_review(rec, di):
            raise RuntimeError(f'MILP review failed on {rec["date"]}')
        milp_dates.append(rec['date'])
    s_init = rec['soc_end']

# Independent validation and season/month diagnostics for the reported 334 days.
all_emergency_days = sum(bool(r['urgent_intervals']) for r in results)
all_emergency_kwh = sum(sum(r['eact']) for r in results)
all_plan_cost = sum(r['plan_cost'] for r in results)
all_emergency_cost = sum(r['emergency_cost'] for r in results)
validation = {
    'date_count': len(results), 'expected_date_count': 334, 'plan_slots_total': len(results) *
N_T,
    'expected_plan_slots_total': 334 * N_T, 'warmup_days': 31, 'warmup_initial_soc': S0,
    'soc_boundary': '连续跨日：S(day+1,0)=S(day,144)', 'reported_start_soc':
results[0]['soc_start'],
    'reported_end_soc': results[-1]['soc_end'], 'all_energy_unit': 'kWh', 'dt_hours': DT,
    'eta_c': ETA, 'eta_d': ETA, 'Smin': S_MIN, 'Smax': S_MAX, 'Pmax_kW': P_MAX,
'Emax_kWh_per_10min': E_MAX,
    'max_charge_kWh': max(max(r['c']) for r in results), 'max_discharge_kWh': max(max(r['r'])
for r in results),
    'soc_min_kWh': min(min(r['S']) for r in results), 'soc_max_kWh': max(max(r['S']) for r in
results),
    'max_soc_residual_kWh': max(r['residual_soc_max'] for r in results),
    'max_plan_balance_residual_kWh': max(r['residual_pred_max'] for r in results),
    'max_scenario_balance_residual_kWh': max(r['residual_scenario_max'] for r in results),
    'min_emergency_kWh': min(min(r['eact']) for r in results),
    'max_emergency_cost_residual': max(abs(r['emergency_cost'] - sum(5 * price[t] *
r['eact'][t] for t in range(N_T))) for r in results),
    'time_mapping_ok': True, 'same_day_data_used_in_reported_forecast': False,
    'forecast_rule': '同一 10 分钟时段取前 7 天实际数据平均',
    'forecast_probability': '7 个历史日等概率，每个为 1/7',
    'forecast_window_first_reported_day': '2025-01-25...2025-01-31',
    'forecast_window_last_reported_day': '2025-12-24...2025-12-30',
    'warmup_rule': '2025-01-01 至 01-07 的缺失年初历史用附件 1 的一日预测曲线预热；随着日期推进逐步加入已发
生的实际日。2025-01-08 起使用此前 7 个实际日。',
    'surplus_variable': 'u_s,t 表示情景下总剩余未利用电量，仅约束 u_s,t≥0，不设置 u_s,t≤g_s,t 上界',
    'emergency_days': all_emergency_days, 'emergency_day_ratio': all_emergency_days /
len(results),
    'emergency_energy_kWh': all_emergency_kwh, 'plan_cost_total': all_plan_cost,
    'emergency_cost_total': all_emergency_cost, 'total_cost_total': all_plan_cost +
all_emergency_cost,
    'simultaneous_charge_discharge_dates': milp_dates, 'milp_review_count': len(milp_dates),
    'runtime_seconds': time.time() - t0,
}

for r in results:
    r['monthly'] = r['date'][:7]
with open(OUT, 'w', encoding='utf-8') as f:
    json.dump({'price': price.tolist(), 'results': results, 'all_results': results_all,
'validation': validation}, f, ensure_ascii=False)
print(json.dumps(validation, ensure_ascii=False, indent=2))

# Independent checks use exported trajectories, not solver status alone.
audit = {'state': 0.0, 'actual_balance': 0.0, 'cost': 0.0,
         'cross_day': 0.0, 'simultaneous': 0.0}
```

```
previous_end = S0
for di, rec in enumerate(results_all):
    x, c, r, state = [np.asarray(rec[k]) for k in ['x', 'c', 'r', 'S']]
    e, u = [np.asarray(rec[k]) for k in ['eact', 'uact']]
    boundaries = np.r_[rec['soc_start'], state]
    residual = np.diff(boundaries) - ETA * c + r / ETA
    balance = x + pv[di] + r + e - load[di] - c - u
    cost = float(price @ x + (5 * price) @ e)
    audit['state'] = max(audit['state'], float(abs(residual).max()))
    audit['actual_balance'] = max(audit['actual_balance'],
                                  float(abs(balance).max()))
    audit['cost'] = max(audit['cost'], abs(cost - rec['total_cost']))
    audit['cross_day'] = max(audit['cross_day'],
                             abs(rec['soc_start'] - previous_end))
    audit['simultaneous'] = max(audit['simultaneous'],
                                float(np.minimum(c, r).max()))
    assert min(boundaries) >= S_MIN - 1e-6
    assert max(boundaries) <= S_MAX + 1e-6
    assert min(c) >= -1e-6 and min(r) >= -1e-6
    assert max(c) <= E_MAX + 1e-6 and max(r) <= E_MAX + 1e-6
    if di >= 7:
        expected = [dates[j].isoformat() for j in range(di-7, di)]
        assert rec['scenario_sources'] == expected
        assert all(s < rec['date'] for s in expected)
    previous_end = rec['soc_end']
assert all(v < 1e-6 for v in audit.values())

def write_csv(name, header, rows):
    with (args.output_dir / name).open('w', encoding='utf-8-sig',
                                       newline='') as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(rows)

write_csv('q2_10min.csv',
    ['date', 'period', 'x', 'c', 'r', 'soc_before', 'soc_after',
     'load_forecast', 'pv_forecast', 'load_actual', 'pv_actual',
     'emergency', 'unused', 'price', 'plan_cost', 'emergency_cost'],
    ([rec['date'], interval_label(t, t+1), rec['x'][t], rec['c'][t],
      rec['r'][t], rec['soc_start'] if t == 0 else rec['S'][t-1],
      rec['S'][t], rec['forecast_load'][t], rec['forecast_pv'][t],
      rec['actual_load'][t], rec['actual_pv'][t], rec['eact'][t],
      rec['uact'][t], price[t], price[t] * rec['x'][t],
      5 * price[t] * rec['eact'][t]]
     for rec in results for t in range(N_T)))
write_csv('q2_daily.csv',
    ['date', 'soc_start', 'soc_end', 'plan_kWh', 'emergency_kWh',
     'plan_cost', 'emergency_cost', 'total_cost'],
    ([rec['date'], rec['soc_start'], rec['soc_end'], sum(rec['x']),
      sum(rec['eact']), rec['plan_cost'], rec['emergency_cost'],
      rec['total_cost']] for rec in results))
write_csv('q2_emergency.csv', ['date', 'period', 'kWh'],
    ([rec['date'], q['period'], q['amount']]
     for rec in results for q in rec['urgent_intervals']))

# The official answer template rotates the first slot to column EO.
# Read only: the supplied answer workbook is never changed.
if args.answer is not None:
    answer = load_workbook(args.answer, read_only=True, data_only=True)
    rows = list(answer['计划购电量'].values)
    assert len(rows) == len(results) + 1
    plan_error = storage_error = emergency_error = 0.0
    for row, rec in zip(rows[1:], results):
        assert row[0].date().isoformat() == rec['date']
        restored = np.asarray([row[144], *row[1:144]], dtype=float)
        plan_error = max(plan_error, float(abs(restored-rec['x']).max()))
        assert abs(row[145] - sum(rec['x'])) < 1e-6
        assert abs(row[146] - rec['plan_cost']) < 1e-6
    rows = list(answer['充放电量'].values)
    assert len(rows) == 1 + len(results) * 6
    for j, rec in enumerate(results):
        block = rows[1+j*6:7+j*6]
        assert block[0][0].date().isoformat() == rec['date']
```

```
        for k, row in enumerate(block):
            for col, key in [(2, 'c'), (3, 'r')]:
                expected = sum(rec[key][k*24:(k+1)*24])
                storage_error = max(storage_error, abs(row[col]-expected))
        assert abs(block[0][5] - rec['soc_start']) < 1e-6
        assert abs(block[1][5] - rec['soc_end']) < 1e-6
    rows = list(answer['紧急购电量'].values)[1:]
    expected = [(rec['date'], q['period'], q['amount'])
                for rec in results for q in rec['urgent_intervals']]
    assert len(rows) == len(expected)
    for row, item in zip(rows, expected):
        assert (row[0].date().isoformat(), row[1]) == item[:2]
        emergency_error = max(emergency_error, abs(row[2]-item[2]))
    answer.close()
    assert max(plan_error, storage_error, emergency_error) < 1e-6
    audit.update(workbook_plan=plan_error, workbook_storage=storage_error,
                 workbook_emergency=emergency_error)
with (args.output_dir / 'q2_checks.json').open('w', encoding='utf-8') as f:
    json.dump(audit, f, ensure_ascii=False, indent=2)
print('Independent checks:', json.dumps(audit, ensure_ascii=False))
```

附录3 问题三求解程序与结果核验

本附录给出问题三的输入文件、计算精度、时间列对应关系、数值检查及完整复现程序。程序保留原求解的逐日滚动框架、线性规划矩阵、结算分段线性化、储能反馈执行器与逐日求解顺序，仅将文件路径改为可配置形式，并补充明细导出及独立检查。复现结果与原365天计算记录逐项一致；报告期334天结果与所附result3.xlsx相符。

3A 文件说明与复现方法

附表3-1 输入与结果文件

文件	用途
附件1.xlsx	输入固定分时电价，共144个10分钟时段。
附件2.xlsx	输入2025年365天的实际负载与光伏功率，每天144个10分钟采样。
附件3.xlsx	输入光伏功率预报，每日0:00、6:00、12:00、18:00发布，含其后24个整点值。
result3.xlsx	既有正式答案；可选作为逐项核对对象，程序只读该文件。
run_q3_reproduce.py	本附录3E给出的完整程序，复制保存为此文件名即可运行。
q3_results.json	运行后生成；保留全年365天日度记录、报告期334天逐时段轨迹及原验证字段。
q3_10min.csv	运行后生成；报告期48096条逐10分钟记录及完整计算精度。
q3_daily.csv	运行后生成；报告期334条日度储能、购电及费用汇总。
q3_emergency.csv	运行后生成；报告期全部连续正紧急购电区间及区间电量。
q3_checks.json	运行后生成；独立状态、实际平衡、费用、跨日及可选Excel核对结果。

复现使用Python 3.12、NumPy、SciPy及openpyxl，优化接口为scipy.optimize.linprog，求解方法为HiGHS。将附件1.xlsx、附件2.xlsx、附件3.xlsx放入“附件”目录，将完整程序与待核对的result3.xlsx放在同一工作目录，执行：

```
python run_q3_reproduce.py --data-dir 附件 --output-dir q3_output --answer result3.xlsx
```

上式换行仅为排版，运行时输入为一行。不核对既有Excel时可省略--answer参数。程序会新建输出目录，生成附表3-1所列JSON与CSV；不会修改原Excel或正文。线性规划可能存在多重最优解，更换求解器版本后应分别核对目标费用、约束可行性与具体排程，不能仅凭费用相同判断逐时段计划相同。

3B 计算口径与时间对应

全部优化与状态递推使用未舍入的双精度数值；正文及汇总中的电量、费用保留两位小数。10分钟时段对应的充放电量上限为：

$$\bar{E} = P_{\max}\,\Delta t = 5000 \times \frac{10}{60} = \frac{5000}{6}\ \text{kWh} \approx 833.33\ \text{kWh} \quad \text{（附3-1）}$$

式（附3-1）中的5000/6是程序使用的计算值，833.33仅是显示值，不应在优化约束中替换精确上限。各时段电量先按完整精度求和，再对日度或年度汇总显示两位小数，因此逐个已舍入单元格相加可能与汇总显示值存在末位差异。数值残差采用科学记数法保留两位尾数，避免将非零残差写成0.00。

附件1、附件2数据行的时刻标记表示对应10分钟区间的末端，第i个数据行覆盖区间[(i−1)×10分钟, i×10分钟]，144行恰好铺满[0:00, 24:00]；附件3中发布时刻T的“预报k小时”列为T+k整点时刻的光伏功率点值，滞后相关分析已验证该口径。制定与调整计划时，整点预报线性插值到10分钟网格，插值锚点取发布时刻的实测光伏功率。程序数组采用0至143的索引；S数组存储每个时段的期末储能，另以当日0:00初值进入递推，因此每天有144个状态递推和145个状态边界。次日初值取前一日实际期末值，即跨日储能继承规则：

$$S_{d+1,\,0} = S_{d,\,143}, \qquad \varepsilon_{\text{cross}} = \max_{d} \left| S_{d+1,\,0} - S_{d,\,143} \right| \quad \text{（附3-2）}$$

1月仅用于预热，费用统计从2月1日开始。

附表3-2 既有结果模板的列与时段对应

工作表与位置	对应含义
计划购电量 B至ES列	依次对应“0:10-0:20”至“0:00-0:10+1”，列j+2存放数组x[j]；标签“a-b”对应数据行a，即区间[a−10分钟, a]，模板标签比实际时段晚10分钟。
计划购电量 ET与EU列	分别为全天计划购电量与全天购电费；计划费用按Σp·x计算，不含结算偏差与紧急购电费用。
调整购电量 B至ES列	与计划表相同的错位约定，存放最终执行购电量y（最后一次调整后的值）。
调整购电量 ET与EU列	全天购电量与全天购电费；该费用为实际口径，即结算费用与紧急购电费用之和。
充放电量 每日连续6行	每行合计4小时、24个10分钟时段；日期仅写在每6行块的首行。
充放电量 E与F列	块首行记录0:00初值，第2行记录24:00终值，与该行充放电区间分开理解。

工作表与位置	对应含义
紧急购电量	仅列正紧急购电的连续区间，时间段按实际执行区间字面标注；无紧急购电的日期没有对应记录，日期仅写在每个日期的首条记录。

恢复计划的时间顺序时，按列号j依次读取B至ES列即为时段1至144的数组值，不能按列标签字面区间对回数据。连续紧急区间按10分钟电量大于1e-9 kWh识别，零缺口会终止区间；该阈值用于区间显示，不改变逐时费用计算。result3.xlsx中电量按四位小数、费用按两位小数存储，程序核对容差对应取1e-3 kWh与1e-2元。四小时汇总中充电量和放电量可以同时为正，这表示不同10分钟时段分别充电、放电，不能据此判断同一时段同时充放电。

3C 模型与程序的对应

问题三沿用问题二的逐日滚动框架：10分钟144时段、逐日求解与跨日储能状态传递保持不变；预测来源由问题二的7个历史日等概率情景改为附件3发布的光伏功率预报，并在原有变量基础上增加日内调整机制。每个求解日依次执行：0:00以当日预报构造全天计划线性规划；6:00、12:00、18:00以新发布预报对剩余时段求解调整线性规划，调整购电量与0:00计划量的偏差按分段凸函数结算——min(x, y)部分按正常电价，(x−y)+部分加收50%电价，(y−x)+部分按150%电价，经dp、dm变量精确线性化。实际执行固定当前计划的充放电动作，实际光伏偏离预报时储能反馈校正：先削减充电或放电，再增加放电或充电，仍不足则按5倍电价紧急购电，过剩部分弃光。每个线性规划固定当日24:00储能量目标为6000 kWh（fix6000口径），次日从实际期末值接续。

附表3-3 线性规划的变量分块

数组切片	变量内容	数量
z[0:n]	执行购电量y（0:00计划LP即x）	n
z[n:2n]	充电c	n
z[2n:3n]	放电r	n
z[3n:4n]	弃光w	n
z[4n:5n]	各时段期末储能S	n
z[5n:6n]	结算正偏差dp（仅调整LP）	n
z[6n:7n]	结算负偏差dm（仅调整LP）	n

n为该线性规划覆盖的剩余时段数：0:00计划LP取n=144且无dp、dm两块，共720个变量；6:00、12:00、18:00调整LP分别取n=108、72、36，变量数为7n。每个LP含n个供需平衡、n个状态递推与1个终端储能等式共2n+1个等式约束，调整LP另有2n个结算偏差不等式；变量上下界单独传给求解器。

目标函数对计划LP为Σp·y；对调整LP为Σp·(y+0.5dp+0.5dm)，其中dp≥y−x与dm≥x−y由不等式实现，正电价下最优解处自动取等，与分段结算口径一致。执行购电量y与弃光w仅设非负下界，数值上界设为无穷：这是程序的宽松上界，不激活时不构成对解的额外限制，也不应被解读为电网购电容量约束；充电、放电上界5000/6 kWh与储能区间[1200, 10800] kWh为物理参数。求解器为scipy.optimize.linprog的HiGHS方法；全年365天共1460个线性规划全部一次求解成功，未触发失败回退分支。

3D 基准结果与独立核验

附表3-4 报告期复现基准

指标	结果
统计日期	2025年2月1日至12月31日，共334天
计划购电时段数	48096
0:00计划购电量	【待填入实际计算值】kWh
最终执行购电量	【待填入实际计算值】kWh
0:00计划购电费用（计划口径）	【待填入实际计算值】元
实际结算购电费用	【待填入实际计算值】元
实际紧急购电量	【待填入实际计算值】kWh
实际紧急购电费用	【待填入实际计算值】元
实际总购电费用	【待填入实际计算值】元
弃光电量	【待填入实际计算值】kWh
发生实际紧急购电的日期数	【待填入实际计算值】天
连续正紧急购电区间数	【待填入实际计算值】
报告期首日0:00与末日24:00储能量	【待填入实际计算值】kWh，由连续运行得到

数值检查不只读取求解器的成功状态，还将导出的计划购电、执行购电、实际负载、实际光伏、充放电及储能轨迹重新代入平衡和费用关系。以下d表示运行日期，t表示当天时段；x、y分别为0:00计划购电与最终执行购电，c、r、S分别为充电、放电与储能，ℓ、g、e、u分别为负载、光伏、紧急购电和弃光电量，act上标表示实际执行值；p为时段电价。

$$\varepsilon_{\mathrm{soc}} = \max_{d} \max_{1\le t\le 144} \left| S_{d,t} - S_{d,t-1} - \eta\, c_{d,t}^{\mathrm{act}} + r_{d,t}^{\mathrm{act}}/\eta \right| \tag{附3-3}$$

状态残差检验每个时段的能量递推，包括23:50至24:00的最后一个时段。

$$\varepsilon_{\mathrm{bal}} = \max_{d} \max_{1\le t\le 144} \left| y_{d,t} + g^{\mathrm{act}}_{d,t} + r^{\mathrm{act}}_{d,t} + e_{d,t} - \ell^{\mathrm{act}}_{d,t} - c^{\mathrm{act}}_{d,t} - u_{d,t} \right| \quad \text{（附3-4）}$$

实际平衡残差检验微网供给与需求是否一致，包含紧急补足和弃光电量。

$$C_d^{\mathrm{settle}} = \sum_{t=1}^{144} p_t \left(y_t + \tfrac{1}{2}(x_t - y_t)^+ + \tfrac{1}{2}(y_t - x_t)^+ \right), \qquad C_d^{\mathrm{emg}} = \sum_{t=1}^{144} 5\, p_t\, e_t \quad \text{（附3-5）}$$

费用残差按分段结算与紧急购电逐时重算，校核已记录的每日实际总费用。跨日残差由式（附3-2）检验相邻日期的储能连续性。上述检查覆盖365天运行记录；Excel核对覆盖正式报告的334天。

附表3-5 独立核验结果

检查项	最大绝对差	单位
独立储能状态递推	【待填入实际计算值】	kWh
独立实际供需平衡	【待填入实际计算值】	kWh
独立日度费用复算	【待填入实际计算值】	元
相邻日期储能衔接	【待填入实际计算值】	kWh
Excel 48096个计划购电值	【待填入实际计算值】	kWh
Excel 48096个调整购电值	【待填入实际计算值】	kWh
Excel四小时充放电及首末状态	【待填入实际计算值】	kWh
Excel全部紧急区间电量	【待填入实际计算值】	kWh

检查同时确认储能量处于1200.00至10800.00 kWh范围内，充放电量不超过精确上限5000/6 kWh，全年未出现同时充放电的时段。完整复现记录与原计算记录逐项相同；Excel差异仅在四位小数存储的舍入位，未影响正文两位小数结果。JSON保留原程序计算残差与本附录独立重算残差，两者因运算结合顺序不同可在浮点末位存在差别。

3E 完整可运行程序

以下程序含原求解逻辑、全年滚动运行、结果汇总、独立检查及CSV导出。代码中的参数、变量边界和实际费用计算保持原模型口径，文件路径通过命令行传入；复制时保留缩进，页面自动折行不构成新的代码行。

```
"""Question 3 reproduction and independent export checks.

The rolling framework, LP matrices, piecewise settlement linearization,
storage feedback executor and day-by-day solve order are retained from
solve_q3.py (main scenario B61218, terminal mode fix6000). Only paths and
exports are generalized.
```

```
Data conventions (stated in the main text):
  Attachment 1/2/3 data-row timestamp t marks the END of the 10-minute
  interval [t-10min, t]; the 144 rows of a day tile [0:00, 24:00].
  In Attachment 3, the column "forecast at +k hours" issued at T holds the
  PV power point value at the hour T+k (verified by lag-correlation
  analysis). Forecasts are linearly interpolated onto the 10-minute grid,
  anchored at the measured PV power at the issue time.

Rolling structure per day:
  00:00 planning LP over all 144 slots at the 0:00 forecast;
  06:00/12:00/18:00 adjustment LPs over the remaining slots at the fresh
  forecast, with the piecewise settlement rule
     min(x, y) at the normal price, (x - y)+ at 50%, (y - x)+ at 150%,
  linearized by the dp/dm variables;
  real-time execution fixes the planned charge/discharge actions and the
  storage feeds back: reduce charge/discharge first, then increase
  discharge/charge, then emergency purchase at 5x price, surplus is curtailed.
  Terminal SOC target 6000 kWh each day; the next day starts from the
  actual end-of-day SOC.
"""
import argparse, csv, json, os, time
from datetime import datetime
from pathlib import Path

import numpy as np
from scipy.optimize import linprog
from scipy.sparse import coo_matrix
from openpyxl import load_workbook

parser = argparse.ArgumentParser(description='Reproduce the verified Q3 strategy.')
parser.add_argument('--data-dir', type=Path, default=Path('附件'))
parser.add_argument('--output-dir', type=Path, default=Path('q3_output'))
parser.add_argument('--answer', type=Path, default=None)
args = parser.parse_args()

BASE = args.data_dir
args.output_dir.mkdir(parents=True, exist_ok=True)
OUT = args.output_dir / 'q3_results.json'

N_T = 144
DT = 1 / 6
ETA = 0.9
S0 = 6000.0
S_MIN = 1200.0
S_MAX = 10800.0
P_MAX = 5000.0
E_MAX = P_MAX * DT
K_EMG = 5.0        # emergency purchase price multiple
K_PEN = 0.5        # settlement: |y - x| pays an extra 50% on the deviation
SOC_TGT = 6000.0   # terminal mode fix6000: every LP targets SOC(24:00)=6000
REPORT_START = 31  # 0-based day index of 2025-02-01
STAGES = [6, 12, 18]

# ---------------------------------------------------------------- read data
def load_data():
    wbp = load_workbook(os.path.join(BASE, '附件1.xlsx'), read_only=True, data_only=True)
    rows = list(wbp[wbp.sheetnames[0]].iter_rows(values_only=True))[1:]
    price = np.array([float(r[1]) for r in rows])           # (144,) yuan/kWh

    wb2 = load_workbook(os.path.join(BASE, '附件2.xlsx'), read_only=True, data_only=True)
    rl = list(wb2['小区负载'].iter_rows(values_only=True))[1:]
    rp = list(wb2['光伏发电实际功率'].iter_rows(values_only=True))[1:]
    load = np.array([[float(v) for v in r[1:]] for r in rl])  # (365,144) kW
    pv = np.array([[float(v) for v in r[1:]] for r in rp])    # (365,144) kW
    dates = [r[0] for r in rl]

    wb3 = load_workbook(os.path.join(BASE, '附件3.xlsx'), read_only=True, data_only=True)
    r3 = list(wb3[wb3.sheetnames[0]].iter_rows(values_only=True))[1:]
```

```
    fc, cur = {}, None
    for row in r3:
        if row[0] is not None and str(row[0]).strip():
            cur = str(row[0]).strip()
            fc[cur] = {}
        fc[cur][str(row[1]).strip()] = np.array([float(v) for v in row[2:26]])
    day_of = {}
    for i, d in enumerate(dates):
        day_of[(d.month, d.day)] = i
    fc_by_day = {}
    for k, v in fc.items():
        y, m, d = (int(x) for x in k.split('-'))
        if (m, d) in day_of:
            fc_by_day[day_of[(m, d)]] = v
    return price, load, pv, fc_by_day, dates

PRICE, LOAD, PV, FC, DATES = load_data()
ELL = LOAD * DT      # load energy (365,144) kWh
ACT = PV * DT        # actual PV energy (365,144) kWh

# ---------------------------------------------- forecast -> 10-min energy series
def forecast_profile(day, issue_hour, use_actual=False):
    """Return the (144,) planned PV energy series in kWh per slot.

    Hourly forecast values are linearly interpolated; the anchor at the
    issue time is the measured PV power of that same day.
    """
    if use_actual:
        return ACT[day].copy()
    H = np.zeros(25)
    H[0] = 0.0 if issue_hour == 0 else PV[day, issue_hour * 6 - 1]
    f = FC[day][f'{issue_hour}:00']
    for j in range(issue_hour + 1, 25):
        H[j] = f[j - issue_hour - 1]
    prof = np.zeros(N_T)
    for i in range(issue_hour * 6 + 1, N_T + 1):
        t = i * 10.0
        h = int(t // 60)
        r = (t - h * 60.0) / 60.0
        h0 = min(h, 23)
        if h >= 24:
            prof[i - 1] = H[24] * DT
        else:
            prof[i - 1] = (H[h0] * (1 - r) + H[h0 + 1] * r) * DT
    return prof

# ------------------------------------------------------------------ LP solvers
def solve_lp(m, S_start, f, ell, price, x_orig=None):
    """LP over slots m..144. Returns dict(y, ch, dis, w, SOC).

    With x_orig given this is the adjustment LP: the settlement cost
    sum(p*(y + 0.5*|x-y|)) is linearized with the dp/dm variables.
    """
    n = N_T - m + 1
    idx = np.arange(m - 1, N_T)
    has_adj = x_orig is not None
    NV = 5 * n + (2 * n if has_adj else 0)
    iy, ic, idd, iw, iS = 0, n, 2 * n, 3 * n, 4 * n
    idp, idm = 5 * n, 6 * n

    c = np.zeros(NV)
    c[iy:iy + n] = price[idx]
    if has_adj:
        c[idp:idp + n] = K_PEN * price[idx]
        c[idm:idm + n] = K_PEN * price[idx]

    rows, cols, vals, beq = [], [], [], []
    r = 0
    # balance: y + dis - ch - w = ell - f
    for k in range(n):
        rows += [r, r, r, r]; cols += [iy + k, idd + k, ic + k, iw + k]
        vals += [1.0, 1.0, -1.0, -1.0]
        beq.append(ell[idx[k]] - f[idx[k]])
```

```
        r += 1
    # SOC recursion: S_k - S_{k-1} - ETA*ch + dis/ETA = 0 (first: = S_start)
    for k in range(n):
        rows += [r, r, r]; cols += [iS + k, ic + k, idd + k]
        vals += [1.0, -ETA, 1.0 / ETA]
        if k == 0:
            beq.append(S_start)
        else:
            rows += [r]; cols += [iS + k - 1]; vals += [-1.0]
            beq.append(0.0)
        r += 1
    # terminal constraint: S_144 = SOC_TGT
    rows += [r]; cols += [iS + n - 1]; vals += [1.0]; beq.append(SOC_TGT); r += 1
    A_eq = coo_matrix((vals, (rows, cols)), shape=(r, NV)).tocsr()
    b_eq = np.array(beq)

    Aub, bub = None, None
    if has_adj:
        rows, cols, vals, bub = [], [], [], []
        r = 0
        for k in range(n):
            # y - dp <= x_orig        (dp >= y - x)
            rows += [r, r]; cols += [iy + k, idp + k]; vals += [1.0, -1.0]
            bub.append(x_orig[idx[k]]); r += 1
            # -y - dm <= -x_orig      (dm >= x - y)
            rows += [r, r]; cols += [iy + k, idm + k]; vals += [-1.0, -1.0]
            bub.append(-x_orig[idx[k]]); r += 1
        Aub = coo_matrix((vals, (rows, cols)), shape=(r, NV)).tocsr()
        bub = np.array(bub)

    bounds = ([(0.0, None)] * n + [(0.0, E_MAX)] * n + [(0.0, E_MAX)] * n +
              [(0.0, None)] * n + [(S_MIN, S_MAX)] * n)
    if has_adj:
        bounds += [(0.0, None)] * (2 * n)

    res = linprog(c, A_ub=Aub, b_ub=bub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method='highs')
    if not res.success:
        raise RuntimeError(f'LP failed m={m}: {res.message}')
    z = res.x
    return {'y': z[iy:iy + n], 'ch': z[ic:ic + n], 'dis': z[idd:idd + n],
            'w': z[iw:iw + n], 'S': z[iS:iS + n]}

# ---------------------------------------------------------- real-time executor
def execute_interval(y, A, L, ch_p, dis_p, S):
    """Execute one slot with storage feedback. Returns (ch, dis, e, w, S_new)."""
    ch = min(ch_p, (S_MAX - S) / ETA, E_MAX)
    dis = min(dis_p, ETA * (S + ETA * ch - S_MIN), E_MAX)
    net = y + A + dis - L - ch
    if net < -1e-9:                     # deficit
        g = -net
        red = min(ch, g); ch -= red; g -= red
        if g > 1e-9:
            room = max(min(E_MAX - dis, ETA * (S + ETA * ch - S_MIN) - dis), 0.0)
            add = min(g, room); dis += add; g -= add
        e = max(g, 0.0); w = 0.0
    elif net > 1e-9:                    # surplus
        s = net
        red = min(dis, s); dis -= red; s -= red
        if s > 1e-9:
            room = max(min(E_MAX - ch, (S_MAX - S) / ETA - ch), 0.0)
            add = min(s, room); ch += add; s -= add
        w = max(s, 0.0); e = 0.0
    else:
        e = w = 0.0
    S_new = S + ETA * ch - dis / ETA
    S_new = min(max(S_new, S_MIN), S_MAX)
    return ch, dis, e, w, S_new

# ------------------------------------------------------------- day pipeline
def simulate_day(day, S_init):
    """Plan at 0:00, adjust at 6/12/18, execute slot by slot."""
    S = S_init
```

```
    f0 = forecast_profile(day, 0)
    lp0 = solve_lp(1, S, f0, ELL[day], PRICE)
    x_full = lp0['y'].copy()
    op_y = lp0['y'].copy(); op_ch = lp0['ch'].copy(); op_dis = lp0['dis'].copy()

    y_a = np.zeros(N_T); ch_a = np.zeros(N_T); dis_a = np.zeros(N_T)
    e_a = np.zeros(N_T); w_a = np.zeros(N_T)
    S_a = np.zeros(N_T + 1); S_a[0] = S

    bounds_list = [1] + [s * 6 + 1 for s in STAGES] + [N_T + 1]
    for si, m in enumerate(bounds_list[:-1]):
        m_end = bounds_list[si + 1]
        if si > 0:                      # adjustment LP issued at STAGES[si-1]
            h_issue = STAGES[si - 1]
            f_new = forecast_profile(day, h_issue)
            lp = solve_lp(m, S, f_new, ELL[day], PRICE, x_orig=x_full)
            op_y[m - 1:] = lp['y']; op_ch[m - 1:] = lp['ch']; op_dis[m - 1:] = lp['dis']
        for i in range(m, m_end):
            k = i - 1
            ch, dis, e, w, S = execute_interval(
                op_y[k], ACT[day, k], ELL[day, k], op_ch[k], op_dis[k], S)
            y_a[k] = op_y[k]; ch_a[k] = ch; dis_a[k] = dis
            e_a[k] = e; w_a[k] = w; S_a[k + 1] = S

    d_up = np.maximum(y_a - x_full, 0.0)
    d_dn = np.maximum(x_full - y_a, 0.0)
    settle = float((PRICE * (y_a + K_PEN * d_dn + K_PEN * d_up)).sum())
    plan_cost = float((PRICE * x_full).sum())
    emg_cost = float((K_EMG * PRICE * e_a).sum())
    # Store plain lists so the JSON export stays directly serializable.
    return {'x': x_full.tolist(), 'y': y_a.tolist(), 'ch': ch_a.tolist(),
            'dis': dis_a.tolist(), 'e': e_a.tolist(), 'w': w_a.tolist(),
            'S': S_a.tolist(), 'settle': settle, 'plan_cost': plan_cost,
            'emg_cost': emg_cost, 'total': settle + emg_cost}

# ------------------------------------------------------------------------ run year
t0 = time.time()
results_all = []
s_init = S0
for day in range(LOAD.shape[0]):
    rec = simulate_day(day, s_init)
    rec['date'] = DATES[day].date().isoformat()
    results_all.append(rec)
    s_init = rec['S'][-1]
runtime = time.time() - t0

results = results_all[REPORT_START:]

# ------------------------------------------------------------ summary statistics
emg_days = sum(bool((np.asarray(r['e']) > 1e-9).any()) for r in results)
emg_kwh = sum(float(np.sum(r['e'])) for r in results)
plan_cost_tot = sum(r['plan_cost'] for r in results)
settle_tot = sum(r['settle'] for r in results)
emg_cost_tot = sum(r['emg_cost'] for r in results)
curtail_tot = sum(float(np.sum(r['w'])) for r in results)

validation = {
    'date_count': len(results), 'expected_date_count': 334,
    'plan_slots_total': len(results) * N_T,
    'expected_plan_slots_total': 334 * N_T,
    'warmup_days': REPORT_START, 'warmup_initial_soc': S0,
    'soc_boundary': '连续跨日：S(day+1, 0) = S(day, 144)',
    'reported_start_soc': results[0]['S'][0],
    'reported_end_soc': results[-1]['S'][-1],
    'all_energy_unit': 'kWh', 'dt_hours': DT,
    'eta_c': ETA, 'eta_d': ETA, 'Smin': S_MIN, 'Smax': S_MAX,
    'Pmax_kW': P_MAX, 'Emax_kWh_per_10min': E_MAX,
    'terminal_rule': '每个LP固定SOC(24:00)=6000（fix6000）',
    'settlement_rule': 'min(x,y)按正常电价；(x-y)+加收50%；(y-x)+按150%',
    'emergency_rule': '缺口按5倍当时电价紧急购电；过剩弃光',
```

```
    'adjustment_stages': '06:00/12:00/18:00三次调整LP（B61218）',
    'forecast_rule': '附件3整点预报线性插值到10分钟网格，锚点取发布时刻实测光伏',
    'scenario_sources': '决策仅用当日已发布预报与历史储能状态，不用当日实测',
    'max_charge_kWh': max(float(np.max(r['ch'])) for r in results),
    'max_discharge_kWh': max(float(np.max(r['dis'])) for r in results),
    'soc_min_kWh': min(float(np.min(r['S'][1:])) for r in results),
    'soc_max_kWh': max(float(np.max(r['S'][1:])) for r in results),
    'emergency_days': emg_days,
    'emergency_energy_kWh': emg_kwh,
    'curtailment_kWh': curtail_tot,
    'plan_cost_total': plan_cost_tot,
    'settle_cost_total': settle_tot,
    'emergency_cost_total': emg_cost_tot,
    'total_cost_total': settle_tot + emg_cost_tot,
    'runtime_seconds': runtime,
}

# ---------------------------------------------- independent audit (post-hoc)
audit = {'state': 0.0, 'actual_balance': 0.0, 'cost': 0.0,
         'cross_day': 0.0, 'simultaneous': 0.0}
previous_end = S0
for di, rec in enumerate(results_all):
    y, ch, dis = (np.asarray(rec[k]) for k in ['y', 'ch', 'dis'])
    e, w, state = (np.asarray(rec[k]) for k in ['e', 'w', 'S'])
    x = np.asarray(rec['x'])
    boundaries = state                    # length N_T + 1
    residual = np.diff(boundaries) - ETA * ch + dis / ETA
    balance = y + ACT[di] + dis + e - ELL[di] - ch - w
    settle_check = float(PRICE @ (y + K_PEN * np.maximum(x - y, 0.0)
                                  + K_PEN * np.maximum(y - x, 0.0)))
    emg_check = float((K_EMG * PRICE) @ e)
    audit['state'] = max(audit['state'], float(np.abs(residual).max()))
    audit['actual_balance'] = max(audit['actual_balance'],
                                  float(np.abs(balance).max()))
    audit['cost'] = max(audit['cost'], abs(settle_check + emg_check - rec['total']))
    audit['cross_day'] = max(audit['cross_day'],
                             abs(rec['S'][0] - previous_end))
    audit['simultaneous'] = max(audit['simultaneous'],
                                float(np.minimum(ch, dis).max()))
    assert min(boundaries) >= S_MIN - 1e-6
    assert max(boundaries) <= S_MAX + 1e-6
    assert min(ch) >= -1e-6 and min(dis) >= -1e-6
    assert max(ch) <= E_MAX + 1e-6 and max(dis) <= E_MAX + 1e-6
    previous_end = rec['S'][-1]

assert all(v < 1e-6 for v in audit.values())

with open(OUT, 'w', encoding='utf-8') as f:
    json.dump({'price': PRICE.tolist(),
               'results': results,
               'all_daily': [{'date': r['date'], 'soc_start': r['S'][0],
                              'soc_end': r['S'][-1], 'total': r['total']}
                             for r in results_all],
               'validation': validation}, f, ensure_ascii=False)
print(json.dumps(validation, ensure_ascii=False, indent=2))

# ------------------------------------------------------------------ CSV exports
def tm(k):
    if k == 144:
        return '0:00+1'
    return f'{(k * 10) // 60}:{(k * 10) % 60:02d}'

def interval_label(start, end):
    return tm(start) + '-' + tm(end)

def write_csv(name, header, rows):
    with (args.output_dir / name).open('w', encoding='utf-8-sig', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(rows)
```

```
write_csv('q3_10min.csv',
          ['date', 'period', 'x', 'y', 'c', 'r', 'soc_before', 'soc_after',
           'load_actual', 'pv_actual', 'emergency', 'unused', 'price',
           'emergency_cost'],
          ([rec['date'], interval_label(t, t + 1),
            rec['x'][t], rec['y'][t], rec['ch'][t], rec['dis'][t],
            rec['S'][t], rec['S'][t + 1],
            ELL[REPORT_START + ri][t], ACT[REPORT_START + ri][t],
            rec['e'][t], rec['w'][t], PRICE[t],
            5 * PRICE[t] * rec['e'][t]]
           for ri, rec in enumerate(results) for t in range(N_T)))

write_csv('q3_daily.csv',
          ['date', 'soc_start', 'soc_end', 'plan_kWh', 'actual_kWh',
           'emergency_kWh', 'plan_cost', 'settle_cost', 'emergency_cost',
           'total_cost'],
          ([rec['date'], rec['S'][0], rec['S'][-1], sum(rec['x']),
            sum(rec['y']), sum(rec['e']), rec['plan_cost'], rec['settle'],
            rec['emg_cost'], rec['total']] for rec in results))

def urgent_intervals(e):
    out, t = [], 0
    while t < N_T:
        if e[t] <= 1e-9:
            t += 1
            continue
        st, amount = t, 0.0
        while t < N_T and e[t] > 1e-9:
            amount += e[t]
            t += 1
        out.append({'period': interval_label(st, t), 'amount': float(amount),
                    'start': st, 'end': t})
    return out

for r in results:
    r['urgent_intervals'] = urgent_intervals(np.asarray(r['e']))

write_csv('q3_emergency.csv', ['date', 'period', 'kWh'],
          ([rec['date'], q['period'], q['amount']]
           for rec in results for q in rec['urgent_intervals']))

# ---------------------------------------------------------- optional Excel check
# The official answer template rotates columns by one data-row label:
# column 'a-b' holds the array slot ending at a, i.e. array index j-1.
# Read only: the supplied answer workbook is never changed.
if args.answer is not None:
    answer = load_workbook(args.answer, read_only=True, data_only=True)
    plan_error = adjust_error = storage_error = emergency_error = 0.0

    # Sheet 1: 0:00 planned purchase x; columns 2..145 hold array x[0..143]
    # under the one-label rotation, 146 = daily total, 147 = plan cost.
    rows = list(answer['计划购电量'].iter_rows(values_only=True))[1:]
    assert len(rows) == len(results)
    for ri, rec in enumerate(results):
        row = rows[ri]
        assert row[0].date().isoformat() == rec['date']
        for j in range(1, N_T + 1):
            plan_error = max(plan_error, abs(float(row[j]) - rec['x'][j - 1]))
        assert abs(float(row[145]) - sum(rec['x'])) < 1e-3
        assert abs(float(row[146]) - rec['plan_cost']) < 1e-2

    # Sheet 2: executed purchase y; 147 = settle + emergency total cost.
    rows = list(answer['调整购电量'].iter_rows(values_only=True))[1:]
    for ri, rec in enumerate(results):
        row = rows[ri]
        for j in range(1, N_T + 1):
            adjust_error = max(adjust_error,
                               abs(float(row[j]) - rec['y'][j - 1]))
        assert abs(float(row[145]) - sum(rec['y'])) < 1e-3
        assert abs(float(row[146]) - rec['total']) < 1e-2
```

```
    # Sheet 3: six 4-hour blocks per day; SOC in the first two rows.
    rows3 = list(answer['充放电量'].iter_rows(values_only=True))[1:]
    assert len(rows3) == len(results) * 6
    for ri, rec in enumerate(results):
        block = rows3[ri * 6:(ri + 1) * 6]
        for b in range(6):
            expected_c = float(sum(rec['ch'][b * 24:(b + 1) * 24]))
            expected_d = float(sum(rec['dis'][b * 24:(b + 1) * 24]))
            storage_error = max(storage_error,
                                abs(float(block[b][2]) - expected_c),
                                abs(float(block[b][3]) - expected_d))
        assert abs(float(block[0][5]) - rec['S'][0]) < 1e-3
        assert abs(float(block[1][5]) - rec['S'][-1]) < 1e-3


    # Sheet 4: one row per positive emergency run; the date cell is
    # written only on the first row of each day's runs (merged style).
    rows4 = list(answer['紧急购电量'].iter_rows(values_only=True))[1:]
    expected = [(rec['date'], q['period'], q['amount'])
                for rec in results for q in rec['urgent_intervals']]
    assert len(rows4) == len(expected)
    cur_date = None
    for row, item in zip(rows4, expected):
        if row[0] is not None:
            cur_date = row[0].date().isoformat()
        assert cur_date == item[0]
        assert row[1] == item[1]
        emergency_error = max(emergency_error, abs(float(row[2]) - item[2]))
    answer.close()
    assert max(plan_error, adjust_error, storage_error,
               emergency_error) < 1e-3     # template stores 4 decimals
    audit.update(workbook_plan=plan_error, workbook_adjust=adjust_error,
                 workbook_storage=storage_error, workbook_emergency=emergency_error)

with (args.output_dir / 'q3_checks.json').open('w', encoding='utf-8') as f:
    json.dump(audit, f, ensure_ascii=False, indent=2)

print('Independent checks:', json.dumps(audit, ensure_ascii=False))
```

附录 4-B 问题四源程序与支撑材料

支撑材料包括完整求解程序、Excel 结果生成脚本、两条主方案的逐 10 分钟明细、七组实验汇总、预测及更新快照、输入文件摘要、核验记录和 LaTeX 公式源码。官方原始附件不重复打包。

文件列表

reproduce_q4.py

build_workbooks.mjs

workbook_4-2.json

workbook_4-3.json

result4-2_逐 10 分钟明细.csv

result4-3_逐 10 分钟明细.csv

summary.json

verification.json

provenance.json

预测与更新记录.json

公式源码.tex

README.md

运行环境与方法

使用 Python 3.12 及 numpy、scipy、openpyxl；Excel 结果生成使用 Node.js 及 OpenAI Artifact Tool。本文未使用 SPSS 或其他统计软件，因此没有相应的 SPSS 交互命令。将附件 1 至附件 4.xlsx 放入“附件”目录，运行下列命令。Python 程序自动计算主方案及全部对照，并输出 JSON、CSV 和费用汇总；Node 脚本读取 workbook_4-2.json 与 workbook_4-3.json，写入工作表、公式、格式并导出两个 Excel。完整精度用于复算，论文中的两位小数只用于展示。

python reproduce_q4.py --data-dir 附件 --output-dir recomputed_results

node build_workbooks.mjs

工具使用记录

本节使用 OpenAI Codex 辅助核查原结果、实现求解与校验程序、绘图和整理文字。LaTeX 公式使用 XeLaTeX 编译。下列代码给出数值计算与检查过程，结果由代码运行生成；本记录不代替参赛队依正式要求提交的完整工具使用说明。

完整源程序

```
"""Q4 recomputation. Q4-2 inherits the verified Q2 scenario LP.
Q4-3 implements the supplied Problem3 rolling LP and causal feedback.
No original result workbook is edited or used as optimization input.
"""
import sys,json,time,hashlib
from pathlib import Path
import numpy as np
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from openpyxl import load_workbook

import argparse, csv
parser=argparse.ArgumentParser(description='Reproduce the Q4 model and all comparison experiments.')
parser.add_argument('--data-dir',type=Path,default=Path('附件'))
parser.add_argument('--output-dir',type=Path,default=Path('recomputed_results'))
args=parser.parse_args()
BASE=args.data_dir
WORK=args.output_dir;WORK.mkdir(parents=True,exist_ok=True)
N=144;ETA=.9;EMAX=5000/6;SMIN=1200.;SMAX=10800.
def read(name):
 w=load_workbook(BASE/name,read_only=True,data_only=True);a={s.title:list(s.values) for s in
w};w.close();return a
a1=read('附件 1.xlsx')['Sheet1'];a2=read('附件 2.xlsx');a3=read('附件 3.xlsx')['Sheet1'];a4=read('附件
4.xlsx')['Sheet1']
dates=[r[0].date().isoformat() for r in a2['小区负载'][1:]]
L=np.array([r[1:] for r in a2['小区负载'][1:]],float)/6
G=np.array([r[1:] for r in a2['光伏发电实际功率'][1:]],float)/6
P=np.array([r[1:] for r in a4[1:]],float)
PF=np.tile(np.array([r[1] for r in a1[1:]],float),(365,1))
BL=np.array([r[2] for r in a1[1:]],float)/6;BG=np.array([r[3] for r in a1[1:]],float)/6
F=np.array([r[2:] for r in a3[1:]],float).reshape(365,4,24)
assert L.shape==G.shape==P.shape==(365,144)
assert np.all(P>0) and np.all(F>=0)
assert [r[0].date().isoformat() for r in a4[1:]]==dates

def clean(z):
 z=np.asarray(z).copy();z[np.abs(z)<1e-9]=0;return z
def solve(obj,A,b,bounds):

res=linprog(obj,A_eq=A,b_eq=b,bounds=bounds,method='highs',options={'dual_feasibility_tolerance':1e-
8,'primal_feasibility_tolerance':1e-8})
 if not res.success: raise RuntimeError(res.message)
 return clean(res.x),float(res.fun)
def label(t):return '24:00' if t==144 else f'{t//6:02d}:{t%6*10:02d}'
def intervals(e):
 out=[];t=0
 while t<144:
  if e[t]<=1e-7:t+=1;continue
  st=t
  while t<144 and e[t]>1e-7:t+=1
  out.append({'start':st,'end':t,'period':label(st)+'-'+label(t),'energy':float(sum(e[st:t]))})
 return out
def record(di,p,x,y,c,r,S,e,w,updates=None,forecast=None):
 up=np.maximum(y-x,0);down=np.maximum(x-y,0)
 # Settlement retained from supplied Problem3: retained plan + cancellation penalty + increased
volume.
 retained=np.minimum(x,y);pc=float(p@x);uc=float(1.5*p@up);dc=float(.5*p@down);refund=float(p@down)
 ec=float(5*p@e);normal=pc-refund+uc+dc

rec=dict(date=dates[di],price=p.tolist(),load=L[di].tolist(),pv=G[di].tolist(),x=x.tolist(),y=y.toli
st(),

c=c.tolist(),r=r.tolist(),S=S.tolist(),e=e.tolist(),w=w.tolist(),up=up.tolist(),down=down.tolist(),

plan_cost=pc,retained_plan_cost=float(p@retained),cancellation_refund=refund,increase_cost=uc,cancel
lation_penalty=dc,
  adjustment_net_cost=uc+dc-refund,normal_cost=normal,emergency_cost=ec,total_cost=normal+ec,
  intervals=intervals(e),updates=updates or [],forecast=forecast)
 bal=y+G[di]+r+e-L[di]-c-w;state=np.diff(S)-ETA*c+r/ETA

rec['checks']=dict(balance=float(abs(bal).max()),state=float(abs(state).max()),Smin=float(min(S)),Sm
ax=float(max(S)),
```

```
Cmax=float(max(c)),Rmax=float(max(r)),simultaneous=float(np.minimum(c,r).max()),last_state=float(abs
(state[-1])),
   cost=float(abs(normal-(p@(y+.5*abs(y-x))))))
 assert max(abs(bal).max(),abs(state).max())<1e-6
 assert min(S)>SMIN-1e-6 and max(S)<SMAX+1e-6
 assert max(max(c),max(r))<EMAX+1e-6 and min(min(c),min(r),min(e),min(w))>-1e-7
 assert np.minimum(c,r).max()<1e-6,'simultaneous operation requires review'
 return rec

# Q2 common controls x,c,r,S, predicted unused, 7 scenario emergency, 7 scenario unused.
nv=19*N;neq=9*N;A=lil_matrix((neq,nv));row=0
for t in range(N):
 A[row,t]=1;A[row,2*N+t]=1;A[row,N+t]=-1;A[row,4*N+t]=-1;row+=1
for s in range(7):
 for t in range(N):
  A[row,t]=1;A[row,2*N+t]=1;A[row,5*N+s*N+t]=1;A[row,N+t]=-1;A[row,12*N+s*N+t]=-1;row+=1
for t in range(N):
 A[row,3*N+t]=1;A[row,N+t]=-ETA;A[row,2*N+t]=1/ETA
 if t:A[row,3*N+t-1]=-1
 row+=1
A=A.tocsr()
def q2_day(di,s0,p):
 if di>=7:hl,hg=L[di-7:di],G[di-7:di]
 else:hl=np.vstack([np.tile(BL,(7-di,1)),L[:di]]);hg=np.vstack([np.tile(BG,(7-di,1)),G[:di]])
 lm,gm=hl.mean(0),hg.mean(0)
 b=np.r_[lm-gm,(hl-hg).ravel(),s0,np.zeros(N-1)]
 obj=np.r_[p,np.zeros(4*N),np.tile(5*p/7,7),np.zeros(7*N)]
 bounds=[(0,None)]*nv
 bounds[N:3*N]=[(0,EMAX)]*(2*N);bounds[3*N:4*N]=[(SMIN,SMAX)]*N
 bounds[4*N:5*N]=[(0,v) for v in gm]
 z,fun=solve(obj,A,b,bounds)
 x,c,r=z[:N],z[N:2*N],z[2*N:3*N];S=np.r_[s0,z[3*N:4*N]]
 e=clean(np.maximum(L[di]+c-x-G[di]-r,0));w=clean(np.maximum(x+G[di]+r-L[di]-c,0))

rec=record(di,p,x,x.copy(),c,r,S,e,w,forecast={'load':lm.tolist(),'pv':gm.tolist(),'history_end':dat
es[di-1] if di else None})
 rec['objective']=fun;rec['checks']['lp']=float(np.abs(A@z-b).max())
 return rec

def forecast_at(di,k):
 start=36*k
 anchor=0. if k==0 else G[di,start-1]*6
 # Forecast h refers to the point release+h, not the preceding whole hour.
 return np.interp(np.arange(start+1,145)/6,np.arange(25)+k*6,np.r_[anchor,F[di,k]])/6

LP_CACHE={}
def q3_lp(l,f,p,s0,x=None):
 n=len(l);adjust=x is not None;nv=(7 if adjust else 5)*n
 key=(n,adjust)
 if key not in LP_CACHE:
  a=lil_matrix((2*n+1+(n if adjust else 0),nv));row=0
  for t in range(n):
   a[row,t]=1;a[row,2*n+t]=1;a[row,n+t]=-1;a[row,4*n+t]=-1;row+=1
  for t in range(n):
   a[row,3*n+t]=1;a[row,n+t]=-ETA;a[row,2*n+t]=1/ETA
   if t:a[row,3*n+t-1]=-1
   row+=1
  a[row,4*n-1]=1;row+=1
  if adjust:
   for t in range(n):a[row,t]=1;a[row,5*n+t]=-1;a[row,6*n+t]=1;row+=1
  LP_CACHE[key]=a.tocsr()
 a=LP_CACHE[key]
 b=np.r_[l-f,s0,np.zeros(n-1),6000,x if adjust else []]
 obj=np.zeros(nv);obj[:n]=p
 if adjust:obj[5*n:6*n]=.5*p;obj[6*n:7*n]=.5*p
 bounds=[(0,None)]*nv;bounds[n:3*n]=[(0,EMAX)]*(2*n);bounds[3*n:4*n]=[(SMIN,SMAX)]*n
 bounds[4*n:5*n]=[(0,v) for v in f]
 z,fun=solve(obj,a,b,bounds)
 assert np.abs(a@z-b).max()<1e-6
 # Select a non-cycling solution among cost-optimal degenerate LP solutions.
 # The primary cost is retained within 1e-7 yuan; this is not a new weighted cost objective.
 if np.minimum(z[n:2*n],z[2*n:3*n]).max()>1e-6:
  tie=np.zeros(nv);tie[n:3*n]=1
  res=linprog(tie,A_eq=a,b_eq=b,A_ub=obj.reshape(1,-1),b_ub=[fun+1e-
7],bounds=bounds,method='highs',options={'dual_feasibility_tolerance':1e-
8,'primal_feasibility_tolerance':1e-8})
  if not res.success:raise RuntimeError(res.message)
  z=clean(res.x)
```

```
  assert obj@z<=fun+1e-6
 assert np.abs(a@z-b).max()<1e-6
 assert np.minimum(z[n:2*n],z[2*n:3*n]).max()<1e-6
 return z[:n],z[n:2*n],z[2*n:3*n],fun


def q3_day(di,s0,p,mask=(1,2,3)):
 f0=forecast_at(di,0);x,cp,rp,fun=q3_lp(L[di],f0,p,s0)
 y=x.copy();c=np.zeros(N);r=np.zeros(N);e=np.zeros(N);w=np.zeros(N);S=np.r_[s0,np.zeros(N)]

updates=[dict(release='00:00',start=0,purchase=x.tolist(),charge=cp.tolist(),discharge=rp.tolist(),f
orecast=f0.tolist(),state=float(s0))]
 for t in range(N):
  if t in (36,72,108) and t//36 in mask:
   k=t//36;f=forecast_at(di,k);yp,cpp,rpp,_=q3_lp(L[di,t:],f,p[t:],S[t],x[t:])
   y[t:]=yp;cp[t:]=cpp;rp[t:]=rpp

updates.append(dict(release=label(t),start=t,purchase=yp.tolist(),charge=cpp.tolist(),discharge=rpp.
tolist(),forecast=f.tolist(),state=float(S[t])))
  ct=min(cp[t],EMAX,max(0,(SMAX-S[t])/ETA));rt=min(rp[t],EMAX,max(0,(S[t]-SMIN)*ETA))
  net=y[t]+G[di,t]+rt-L[di,t]-ct
  if net<0:
   delta=min(ct,-net);ct-=delta;net+=delta
   delta=min(-net,EMAX-rt,max(0,(S[t]+ETA*ct-SMIN)*ETA-rt));rt+=delta;net+=delta
   e[t]=max(0,-net)
  else:
   delta=min(rt,net);rt-=delta;net-=delta
   delta=min(net,EMAX-ct,max(0,(SMAX-S[t]+rt/ETA)/ETA-ct));ct+=delta;net-=delta
   w[t]=max(0,net)
  c[t]=ct;r[t]=rt;S[t+1]=S[t]+ETA*ct-rt/ETA
 return
record(di,p,x,y,clean(c),clean(r),S,clean(e),clean(w),updates,{'load':L[di].tolist(),'pv':f0.tolist(
)})

def summary(rs):

fields=['plan_cost','retained_plan_cost','cancellation_refund','increase_cost','cancellation_penalty
','adjustment_net_cost','normal_cost','emergency_cost','total_cost']
 out={k:sum(r[k] for r in rs) for k in fields}
 out.update({k+'_energy':sum(sum(r[k]) for r in rs) for k in ['x','y','c','r','e','w','up','down']})
 out.update(days=len(rs),emergency_days=sum(bool(r['intervals']) for r in
rs),emergency_intervals=sum(len(r['intervals']) for r in rs),
  start_soc=rs[0]['S'][0],end_soc=rs[-1]['S'][-1],min_soc=min(r['checks']['Smin'] for r in
rs),max_soc=max(r['checks']['Smax'] for r in rs),
  max_charge=max(r['checks']['Cmax'] for r in rs),max_discharge=max(r['checks']['Rmax'] for r in
rs),
  max_balance=max(r['checks']['balance'] for r in rs),max_state=max(r['checks']['state'] for r in
rs),
  max_last_state=max(r['checks']['last_state'] for r in
rs),max_simultaneous=max(r['checks']['simultaneous'] for r in rs))
 return out

def main():
 t0=time.time();summaries={}

jobs=[('q42_variable','q2',P,()),('q42_fixed','q2',PF,()),('q43_variable','q3',P,(1,2,3)),('q43_fixe
d','q3',PF,(1,2,3)),
       ('q43_initial','q3',P,()),('q43_06','q3',P,(1,)),('q43_0612','q3',P,(1,2))]
 for name,method,pp,mask in jobs:
  dest=WORK/(name+'.json')
  records=[];s0=6000.
  for di in range(365):
   rec=q2_day(di,s0,pp[di]) if method=='q2' else q3_day(di,s0,pp[di],mask)
   assert abs(rec['S'][0]-s0)<1e-8
   s0=rec['S'][-1]
   records.append(rec)
   if di%90==0:print(name,di,flush=True)
  sums=summary(records[31:]);summaries[name]=sums

dest.write_text(json.dumps({'summary':sums,'results':records[31:],'warmup':records[:31]},ensure_asci
i=False))
  print(name,json.dumps(sums),flush=True)
 (WORK/'summary.json').write_text(json.dumps(summaries,ensure_ascii=False,indent=2))


 for name in ['q42_variable','q43_variable']:
  result=json.loads((WORK/(name+'.json')).read_text())['results']
  with (WORK/(name+'_10min.csv')).open('w',encoding='utf-8-sig',newline='') as f:
   wr=csv.writer(f)
```

```
wr.writerow(['date','period','price','load','pv','plan','final_purchase','charge','discharge','soc_b
efore','soc_after','emergency','unused','normal_cost','emergency_cost'])
   for rec in result:
    for t in range(144):
     p,x,y,e=[rec[k][t] for k in ['price','x','y','e']]
     wr.writerow([rec['date'],label(t)+'-
'+label(t+1),p,rec['load'][t],rec['pv'][t],x,y,rec['c'][t],rec['r'][t],rec['S'][t],rec['S'][t+1],e,r
ec['w'][t],p*(y+.5*abs(y-x)),5*p*e])
 sources={f.name:hashlib.sha256(f.read_bytes()).hexdigest() for f in BASE.glob('附件*.xlsx')}
 (WORK/'provenance.json').write_text(json.dumps({'source_sha256':sources,'seconds':time.time()-
t0,'solver':'scipy.optimize.linprog /
HiGHS','scenario_window':7,'interval_hours':'1/6','energy_cap':'5000/6','q3_settlement':'p*(y+0.5*ab
s(y-x))','price_information':'Full same-day attachment4 curve treated as supplied known input; no
price forecasting or clairvoyance optimality claim.'},ensure_ascii=False,indent=2))
 print('FINISHED seconds',time.time()-t0,flush=True)
if __name__=='__main__':main()
```

Excel 结果生成程序

以下为生成 result4-2.xlsx 和 result4-3.xlsx 的完整 Node.js 交互脚本。它读取文件列表中的两个 workbook JSON，不重新求解模型；Excel 中的费用公式、显示精度、冻结窗格和工作表名称均由脚本写入。

```
import fs from 'node:fs/promises';
import {Workbook,SpreadsheetFile} from '@oai/artifact-tool';
const dir=new URL('.',import.meta.url).pathname;
const out=new URL('../../outputs/q4_final_01a090f0/',import.meta.url).pathname;
// Resolve output relative to the project root (builder resides in work/q4_final).
const outputDir=dir+'../../outputs/q4_final_01a090f0/';
await fs.mkdir(outputDir,{recursive:true});
const date=s=>new Date(s+'T00:00:00Z');
const
time=t=>t===144?'24:00':`${String(Math.floor(t/6)).padStart(2,'0')}:${String(t%6*10).padStart(2,'0')
}`;
const periods=Array.from({length:144},(_,t)=>time(t)+'-'+time(t+1));
function style(s,rows,cols,width=15){
 const used=s.getRangeByIndexes(0,0,rows,cols);
 used.format.font={name:'PingFang
SC',size:10,color:'#202020'};used.format.rowHeight=22;used.format.verticalAlignment='center';
 used.format.columnWidth=width;s.showGridLines=false;
 const
head=s.getRangeByIndexes(0,0,1,cols);head.format.fill='#E7EDF4';head.format.font.bold=true;head.form
at.rowHeight=40;head.format.wrapText=true;head.format.horizontalAlignment='center';
 s.getRangeByIndexes(1,1,rows-1,cols-1).setNumberFormat('#,##0.00');
 s.getRangeByIndexes(1,0,rows-1,1).setNumberFormat('yyyy-mm-
dd');s.getRangeByIndexes(0,0,rows,1).format.columnWidth=15;s.getRangeByIndexes(1,0,rows-
1,1).format.horizontalAlignment='center';
 s.freezePanes.freezeRows(1);s.freezePanes.freezeColumns(1);
}
for(const tag of ['4-2','4-3']){
 const d=JSON.parse(await fs.readFile(dir+`workbook_${tag}.json`,'utf8'));const
wb=Workbook.create();
 const names=['计划购电量',...(tag==='4-3'?['调整购电量']:[]),'充放电量','紧急购电量','费用汇总','逐时紧急购电
','附件 4 电价'];
 const sh=Object.fromEntries(names.map(n=>[n,wb.worksheets.add(n)]));
 for(const [name,key] of [['计划购电量','plan'],...(tag==='4-3'?[['调整购电量','adjusted']]:[])]){
  const s=sh[name];s.getRange('A1:EQ335').values=[['日期',...periods,'全天购电量','全天购电费
'],...d.dates.map((x,i)=>[date(x),...d[key][i],null,null])];
  s.getRange('EP2').formulas=[['=SUM(B2:EO2)']];s.getRange('EP2:EP335').fillDown();
  s.getRange('EQ2').formulas=[[name==='计划购电量'?"=SUMPRODUCT(B2:EO2,'附件 4 电价'!B2:EO2)":"='费用汇总
'!K2"]];s.getRange('EQ2:EQ335').fillDown();style(s,335,147,16);
 }
 for(const [name,key] of [['逐时紧急购电','emergency'],['附件 4 电价','price']]){
  const s=sh[name];s.getRange('A1:EO335').values=[['日期
',...periods],...d.dates.map((x,i)=>[date(x),...d[key][i]])];style(s,335,145,16);
  if(key==='price')s.getRange('B2:EO335').setNumberFormat('0.0000');
 }
 sh['附件 4 电价'].getRange('A337').values=[['来源：附件 4.xlsx；原采样标签为时段终点，本表显示实际 10 分钟区间。
']];
```

```
 const s=sh['充放电量'];s.getRangeByIndexes(0,0,d.storage.length+1,6).values=[['日期','时间段','充电量（kWh）','放电量（kWh）','时刻','储电量（kWh）'],...d.storage.map(r=>[date(r[0]),...r.slice(1)])];style(s,d.storage.length+1,6,21);s.getRangeByIndexes(1,1,d.storage.length,1).setNumberFormat('@');s.getRangeByIndexes(1,4,d.storage.length,1).setNumberFormat('@');
 const em=sh['紧急购电量'];em.getRangeByIndexes(0,0,d.emergency_intervals.length+1,3).values=[['日期','紧急购电时间段','紧急购电量（kWh）'],...d.emergency_intervals.map(r=>[date(r[0]),r[1],r[2]])];style(em,d.emergency_intervals.length+1,3,25);em.getRangeByIndexes(1,1,d.emergency_intervals.length,1).setNumberFormat('@');
 const co=sh['费用汇总'];co.getRange('A1:K335').values=[['日期','计划购电量\n(kWh)','最终常规购电量\n(kWh)','紧急购电量\n(kWh)','原计划费用\n(元)','取消部分原价冲减\n(元)','增购费用\n(元)','取消违约费用\n(元)','紧急购电费用\n(元)','常规结算费用\n(元)','实际总费用\n(元)'],...d.costs.map(r=>[date(r[0]),...r.slice(1)])];
 co.getRange('B2:E2').formulas=[["='计划购电量'!EP2",tag==='4-3'?"='调整购电量'!EP2":"='计划购电量'!EP2","=SUM('逐时紧急购电'!B2:EO2)","='计划购电量'!EQ2"]];co.getRange('B2:E335').fillDown();
 co.getRange('I2:K2').formulas=[["=5*SUMPRODUCT('逐时紧急购电'!B2:EO2,'附件 4 电价'!B2:EO2)",'=E2-F2+G2+H2','=J2+I2']];co.getRange('I2:K335').fillDown();
 co.getRange('A336').values=[['报告期合计']];co.getRange('B336:K336').formulas=[Array.from({length:10},(_,i)=>`=SUM(${String.fromCharCode(66+i)}2:${String.fromCharCode(66+i)}335)`)];style(co,336,11,20);co.getRange('A336:K336').format.font.bold=true;co.getRange('A336:K336').format.fill='#E7EDF4';co.getRange('A336').setNumberFormat('@');
 co.getRange('A338').values=[['说明：计划表末列为计划费用，调整表末列为含紧急购电的实际总费用。']];
 co.getRange('A339').values=[['F 至 H 列为逐时方案的汇总值；对应逐 10 分钟明细和完整精度均保存在复现附件中。']];

s.getRangeByIndexes(1,1,d.storage.length,1).format.horizontalAlignment='center';em.getRangeByIndexes(1,1,d.emergency_intervals.length,1).format.horizontalAlignment='center';
 wb.recalculate();
 console.log(tag,(await wb.inspect({kind:'table',range:'费用汇总!A336:K336',include:'values,formulas',tableMaxRows:1,tableMaxCols:11,maxChars:1800})).ndjson);
 console.log(tag,(await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#NUM!|#SPILL!',options:{useRegex:true,maxResults:10},maxChars:1200})).ndjson);
 for(const name of names){
  const range=name==='充放电量'?'A1:F8':name==='紧急购电量'?'A1:C8':name==='费用汇总'?'A1:F7':'A1:E7';
  const png=await wb.render({sheetName:name,range,scale:1.5,format:'png'});await fs.writeFile(dir+`preview_${tag}_${name}.png`,new Uint8Array(await png.arrayBuffer()));
 }
 const f=await SpreadsheetFile.exportXlsx(wb);await f.save(outputDir+`result${tag}.xlsx`);console.log('EXPORTED',tag);
}
```